BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC VĂN HIẾN

ThS. Lương Thái Hiền

CƠ SỞ DỮ LIỆU

TẬP BÀI GIẢNG

(Tài liệu lưu hành nội bộ)

Năm 2023

A member of HungHau Education

LỜI NÓI ĐẦU

"Cơ sở dữ liệu" là học phần cơ sở ngành đối với sinh viên thuộc các chuyên ngành Công nghệ Thông tin. Ngày nay, cơ sở dữ liệu đã có nhiều ứng dụng trong mọi hoạt động của xã hội. Muốn thiết kế và sử dụng cơ sở dữ liệu chúng ta phải nắm được các kỹ thuật cơ bản của cơ sở dữ liệu. đó là mô hình liên kết thực thể, mô hình cơ sở dữ liệu quan hệ, cách sử dụng các phép toán đại số quan hệ để tạo, cập nhật và truy vấn cơ sở dữ liệu và khái niệm phụ thuộc hàm ứng dụng trong lý thuyết thiết kế và chuẩn hóa cơ sở dữ liệu quan hệ.

Bài giảng cần thiết cho tất cả các đối tượng muốn tìm hiểu và thiết kế các cơ sở dữ liệu quan hệ ứng dụng trong công tác quản lý.

Với mong muốn sinh viên có tài liệu học tập và có thể hiểu được các nội dung trong học phần, tác giả đã biên soạn tập bài giảng này với cách thức trình bày cô đọng cùng các ví dụ minh họa đi kèm.

Tập bài giảng "Cơ sở dữ liệu" trình bày những kiến thức liên quan đến các khái niệm về cơ sở dữ liệu, cách thiết kế và tổ chức, cài đặt các lược đồ cơ sở dữ liệu; khai thác dữ liệu thông qua các cấu trúc lệnh SQL,…

Tập bài giảng bao gồm 7 chương:

Chương 1: trình bày tổng quan về cơ sở dữ liệu, các mô hình cơ sở dữ liệu, hệ quản trị cơ sở dữ liệu.

Chương 2: trình bày quy trình thiết kế cơ sở dữ liệu, thiết kế mô hình dữ liệu ở mức quan niệm.

Chương 3: trình bày các khái niệm liên quan đến mô hình dữ liệu quan hệ, cách chuyển đổi mô hình quan niệm sang mô hình quan hệ.

Chương 4: trình bày các khái niệm về đại số quan hệ, các phép tính đại số quanhệ, phép tính trên biến bộ,…

Chương 5: trình bày các cấu trúc lệnh SQL tạo cơ sở dữ liệu, tạo bảng, khaithác dữ liệu,…

Chương 6: trình bày các khái niệm liên quan đến ràng buộc toàn vẹn, cài đặt ràngbuộc toàn vẹn cho cơ sở dữ liệu.

Chương 7: trình bày các khái niệm liên quan đến phụ thuộc hàm, các thuật toánliên quan đến phụ thuộc hàm; các dạng chuẩn của cơ sở dữ liệu, …

MỤC LỤC

Chương 1 8

TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU 8

1.1. Giới thiệu 8

1.2. Quá trình phát triển 8

1.3. Một số đặc tính của cơ sở dữ liệu 10

1.4. Người sử dụng cơ sở dữ liệu 12

1.5. Khái niệm và tính năng của hệ quản trị cơ sở dữ liêu 13

1.6. Kiến trúc của hệ quản trị cơ sở dữ liêu 14

1.7. Một số ứng dụng cơ sở dữ liệu 16

BÀI TẬP CHƯƠNG 1 18

Chương 2 19

MÔ HÌNH THỰC THỂ KẾT HỢP 19

2.1. Quá trình thiết kế cơ sở dữ liệu 19

2.2. Mô hình thực thể kết hợp 22

2.2.1. Thực thể (Entity) 22

2.2.2. Thuộc tính (Attribute) 24

2.2.3. Mối kết hợp 28

2.2.4. Lược đồ thực thể kết hợp 31

2.2.5. Các loại lược đồ thực thể kết hợp 34

2.3. Các vấn đề về thiết kế ở mức khái niệm 37

BÁI TẬP CHƯƠNG 2 38

Chương 3 42

MÔ HÌNH DỮ LIỆU QUAN HỆ 42

3.1. Giới thiệu 42

3.2. Các khái niệm của mô hình quan hệ 42

3.2.1. Quan hệ (Relation) 42

3.2.2. Schema 43

3.2.3. Thuộc tính 44

3.2.4. Lược đồ 44

3.2.5. Bộ 44

3.2.6. Miền giá trị 45

3.3. Các ràng buộc của mô hình quan hệ 45
3.4. Các đặc trưng của mô hình quan hệ 46
3.5. Chuyển lược đồ cơ sở dữ liệu theo mô hình thực thể kết hợp sang mô hình quan hệ 47
3.1.1. Qui tắc chung 47
3.1.2. Qui tắc chuyển mối quan hệ thành khóa ngoại 47
BÀI TẬP CHƯƠNG 3 48
Chương 4 49
ĐẠI SỐ QUAN HỆ VÀ PHÉP TÍNH QUAN HỆ 49
3.1. Giới thiệu 49
3.2. Phép toán một ngôi 49
4.2.1. Phép chọn 49
4.2.2. Phép chiếu 50
4.2.3. Phép đổi tên 50
3.3. Phép toán hai ngôi 51
4.3.1. Phép toán tập hợp 51
4.3.2. Phép kết 53
4.3.3. Phép chia 54
3.4. Phép toán khác 55
3.4.1. Hàm kết hợp 55
3.4.2. Gom nhóm 55
3.4.3. Kết ngoài 56
3.5. Phép tính quan hệ biến bộ 56
4.5.1. Biến bộ 57
4.5.2. Biểu thức và phép tính quan hệ 57
3.6. Phép tính quan hệ biến miền 59
BÀI TÂP CHƯƠNG 4 60
Chương 5 61
NGÔN NGỮ TRUY VẤN SQL 61
5.1. Giới thiệu 61
5.2. Schema là gì? 61
5.2.1. Tạo Schema với lệnh Create Schema 62

5.2.2. Cách xóa Schema với lệnh DROP SCHEMA 62
5.2.3. Phân quyền cho Schema 62
5.3. Định nghĩa dữ liệu 63
5.3.1. Kiểu dữ liệu 63
5.3.2. Các lệnh định nghĩa dữ liệu 63
5.4. Cập nhật dữ liệu 70
5.4.1. Thêm dữ liệu 70
5.4.2. Xóa dữ liệu 75
5.4.3. Sửa dữ liệu 76
5.5. Truy vấn dữ liệu 78
5.5.1. Truy vấn cơ bản 78
5.5.2. Tập hợp, so sánh tập hợp và truy vấn lồng 80
5.5.3. Hàm kết hợp và gom nhóm 82
5.5.4. Một số dạng truy vấn khác 83
BÀI TẬP CHƯƠNG 5 88
Chương 6 90
RÀNG BUỘC TOÀN VẸN 90
6.2. Các yếu tố của ràng buộc toàn vẹn 90
6.3.1. Bối cảnh 90
6.3.2. Nội dung 90
6.3.3. Bảng tầm vực ảnh hưởng 91
6.3. Phân loại ràng buộc toàn vẹn 91
6.3.1. Ràng buộc toàn vẹn liên quan đến một quan hệ 93
6.3.2. Ràng buộc toàn vẹn liên quan đến nhiều quan hệ 94
6.4. Cài đặt các ràng buộc toàn vẹn 98
6.4.1. Assertion 98
6.4.2. Trigger 99
BÀI TẬP CHƯƠNG 6 102
Chương 7 104
PHỤ THUỘC HÀM VÀ CHUẨN HÓA CƠ SỞ DỮ LIỆU 104
7.1. Sự dư thừa thông tin 104
7.2. Phụ thuộc hàm 105

7.2.1. Định nghĩa 105
7.2.2. Các tính chất của phụ thuộc hàm 105
7.3. Hệ luật dẫn Armstrong 106
7.4. Bao đóng và thuật toán tìm bao đóng 108
7.4.1. Định nghĩa bao đóng 108
7.4.2. Thuật toán tìm bao đóng 108
7.5. Khóa và thuật toán tìm khóa của lược đồ quan hệ 109
7.5.1. Định nghĩa khóa của lược đồ quan hệ 109
7.5.2. Thuật toán tìm khóa của lược đồ quan hệ 109
7.6. Các dạng chuẩn 112
7.6.1. Dạng Chuẩn Một (First Normal Form) 112
7.6.2. Dạng Chuẩn 2 (Second Normal Form) 113
7.6.3. Dạng Chuẩn 3 (Third Normal Form) 115
7.6.4. Định lý 116
7.6.5. Dạng Chuẩn BC (Boyce-Codd Normal Form) 118
BÀI TẬP CHƯƠNG 7 122
TÀI LIỆU THAM KHẢO 124

DANH MỤC KÝ HIỆU – TỪ VIẾT TẮT

Từ viết tắt	Cụm từ đầy đủ
CSDL	Cơ sở dữ liệu
ERD	Entity Relationship Diagram
HQTCSDL	Hệ quản trị cơ sở dữ liệu
DBMS	Database Management System
IBM	International Business Machines
NASA	National Aeronautics and Space Administration
IMS	Information Management System
IDS	Integrated Data Store
CODASYL	Conference of Data System Language
SEQUEL	Structured English Query Language
E-R	Entity-Relationship
RSI	Relational Software Inc
SQL	Structured Query Language
CAD	Computer Aided Design
GPL	General Public License
DBA	Database Administrator
DBD	Database Designer
ERP	Enterprise Resource Planning

DANH MỤC HÌNH ẢNH

Hình 1.1. Mô hình cơ sở dữ liệu quan hệ *12*
Hình 1.2. Các thành phần chính của hệ quản trị cơ sở dữ liệu *14*
Hình 2.1. Mô hình thực thể nhân viên *23*
Hình 2.2. Mô hình thực thể nhân viên *27*
Hình 2.3. Sơ đồ mô hình dữ liệu mạng *29*
Hình 2.4. Sơ đồ mô hình mối kết hợp [n:n] *30*
Hình 2.5. Sơ đồ mối kết hợp Chuyên biệt hóa *30*
Hình 2.6. Sơ đồ mối kết hợp đệ quy *31*
Hình 2.7. Sơ đồ mối kết hợp mở rộng có nhiều loại mối kết hợp với nhau *31*
Hình 3.1. Lược đồ quan hệ *43*
Hình 3.2. Các bộ dữ liệu trong mô hình quan hệ *45*
Hình 3.3. Mô hình ràng buộc toàn vẹn tham chiếu *46*
Hình 3.4. Mô hình ERD quản lý Sinh viên *48*
Hình 6.1. Lược đồ cơ sở dữ liệu được biểu diễn dưới dạng đồ thị. *97*

Chương 1

TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU

1.1. Giới thiệu

Cơ sở dữ liệu là một bộ sưu tập dữ liệu có hệ thống, được lưu trữ bằng điện tử. Nó có thể chứa bất kỳ loại dữ liệu nào, bao gồm từ, số, hình ảnh, video và tệp. Bạn có thể sử dụng phần mềm được gọi là hệ thống quản lý cơ sở dữ liệu (Database Management System: DBMS) để lưu trữ, truy xuất và chỉnh sửa dữ liệu.

Cơ sở dữ liệu hiệu năng cao có ý nghĩa quan trọng đối với bất kỳ tổ chức nào. Cơ sở dữ liệu hỗ trợ các hoạt động nội bộ trong công ty và lưu trữ hoạt động tương tác với khách hàng cũng như nhà cung cấp. Chúng cũng lưu giữ thông tin quản trị và nhiều dữ liệu chuyên biệt hơn, chẳng hạn như các mô hình kỹ thuật hoặc kinh tế. Ví dụ bao gồm hệ thống thư viện kỹ thuật số, hệ thống đặt chỗ du lịch và hệ thống kiểm kê. Sau đây là một số lý do cho thấy mức độ quan trọng của cơ sở dữ liệu.

- Điều chỉnh quy mô hiệu quả: Các ứng dụng cơ sở dữ liệu có thể quản lý một lượng lớn dữ liệu, giúp điều chỉnh quy mô thành hàng triệu, hàng tỷ và hơn thế nữa. Nếu không có cơ sở dữ liệu thì không thể lưu trữ lượng dữ liệu kỹ thuật số này.
- Tính toàn vẹn của dữ liệu: Cơ sở dữ liệu thường có những quy tắc và điều kiện tích hợp để duy trì tính nhất quán của dữ liệu.
- Bảo mật dữ liệu: Cơ sở dữ liệu hỗ trợ những yêu cầu về quyền riêng tư và khả năng tuân thủ liên quan đến bất kỳ dữ liệu nào.

Ví dụ: để có quyền truy cập cơ sở dữ liệu, người dùng phải đăng nhập. Người dùng khác nhau cũng có thể được truy cập ở những cấp độ khác nhau, chẳng hạn như chỉ đọc.

- Phân tích dữ liệu: Hệ thống phần mềm hiện đại sử dụng cơ sở dữ liệu để phân tích dữ liệu. Những hệ thống này có thể xác định các xu hướng và mẫu hoặc đưa ra dự đoán. Hoạt động phân tích dữ liệu giúp tổ chức tự tin đưa ra quyết định kinh doanh.

1.2. Quá trình phát triển

Lịch sử của cơ sở dữ liệu được xem như bắt đầu vào những năm 60 của thế kỷ trước, khi máy tính bắt đầu được sử dụng tương đối rộng rãi. Cơ sở dữ liệu có quy mô lớn đầu tiên được công ty (International Business Machines: IBM) thực hiện cho Cục Quản trị Hàng không và Không gian Quốc gia (National Aeronautics and

Space Administration: NASA) của Mỹ để hỗ trợ cho việc quản lý dự án Apollo dùng để thám hiểm mặt trăng. Sau đó hệ cơ sở dữ liệu này được thương mại hóa với tên (Information Management System: IMS). Hệ này sử dụng mô hình dữ liệu phân cấp (Hierarchical).

Cũng trong thời gian đó, Charles Bachman cũng phát triển (Integrated Data Store: IDS) sử dụng mô hình dạng mạng (Network). Mô hình này sau đó được tiêu chuẩn hóa bởi (Conference of Data System Language: CODASYL).

Năm 1970, Edgar Codd, một nhà khoa học làm việc cho IBM, đề xuất một số khái niệm về hệ cơ sở dữ liệu quan hệ (Relational) đánh dấu một bước chuyển biến quan trọng trong lĩnh vực này. Tuy nhiên dường như IBM không mặn mà lắm với những ý tưởng của Codd, mặc dù họ cũng tạo ra một dự án nhỏ, System R, để nghiên cứu các đề xuất của Codd. Tuy vậy họ cũng tạo ra được ngôn ngữ SEQUEL (Structured English Query Language), tiền thân của SQL mà hiện chúng ta thường sử dụng.

Năm 1976, Peter Chen đã hoàn thiện thêm hệ cơ sở dữ liệu này bằng mô hình thực thể - liên kết (Entity-Relationship hay E-R).

Năm 1979, (Relational Software Inc: RSI), sau đổi tên là Oracle, là công ty đầu tiên sử dụng mô hình quan hệ để thương mại hóa và dần dần chiếm lĩnh thị trường quản lý dữ liệu. Cũng trong thời gian này IBM cũng bắt đầu thương mại hóa SQL. Chẳng bao lâu SQL trở thành tiêu chuẩn để sử dụng với cơ sở dữ liệu quan hệ.

Cùng với sự phát triển của máy tính, bên cạnh các tập tin văn bản, dữ liệu được thể hiện dưới dạng những ký tự, ta còn có những loại tập tin khác, các tập tin hình ảnh, các bản vẽ được tạo ra từ những phần mềm (Computer Aided Design: CAD), dữ liệu được thể hiện bằng những phương thức khác. Vì thế từ thập niên 1980, hệ quản trị cơ sở dữ liệu hướng đối tượng (Object-Oriented DBMS) bắt đầu được nghiên cứu và đến những năm 1990, chúng bắt đầu được đưa ra thị trường.

Cũng bắt đầu từ thập niên 1990, cùng với sự bùng nổ của Internet và các dịch vụ trực tuyến, các hệ quản trị cơ sở dữ phục vụ cho thị trường ngày phát triển mạnh mẽ. Bên cạnh các sản phẩm của các đại gia như Oracle Database của Oracle, DB2 Universal Database của IBM, SQL Server của Microsoft, Sybase Adaptive Server

của Sybase, cũng xuất hiện các phần mềm miễn phí, mã nguồn mở, nổi bật hơn cả là MySQL, PostgreSQL, SQLite.

Bên cạnh đó, một xu hướng khác cũng xuất hiện từ đầu thế kỷ 21 là các hệ quản trị cơ sở dữ theo hướng NoSQL (được hiểu là "not only SQL"), như MongoDB, có khả năng làm việc với các dữ liệu có cấu trúc ít bị ràng buộc chặt chẽ hơn, có thể hoạt động trong điều kiện có cấu hình phần cứng hạn chế như trên điện thoại di động.

1.3. Một số đặc tính của cơ sở dữ liệu

Một cơ sở dữ liệu, được định nghĩa là một tập hợp các tài liệu, số, tên, có đặc trưng được điều chỉnh bởi cùng một thứ tự và ngữ cảnh, được lưu trữ một cách có hệ thống, miễn là bản ghi thuộc bất kỳ loại hoặc ký tự nào được tạo ra, được làm trống thành các cột điển hình, với cách sắp xếp theo chỉ mục, được xử lý dễ dàng và đơn giản sẽ được gọi giống một cơ sở dữ liệu.

Hiện tại, và nhờ sự phát triển công nghệ tuyệt vời trong lĩnh vực máy tính và điện tử, các cơ sở dữ liệu được tìm thấy ở dạng kỹ thuật số, chúng cung cấp một loạt các công cụ khi chuẩn bị cơ sở dữ liệu, tạo thành trụ cột cơ bản cho những người làm việc trong lĩnh vực tổ chức của một công ty hoặc đơn giản là kiểm soát hành chính.

Cơ sở dữ liệu chủ yếu bao gồm một hoặc nhiều hàng và cột, và mỗi hàng trong số này có liên quan đến nhau và lưu trữ một phần tử thông tin như một bản ghi dữ liệu danh nghĩa.

Các đặc điểm chính của cơ sở dữ liệu:

- Cơ sở dữ liệu cho phép truy cập thông qua các ngôn ngữ lập trình tiêu chuẩn.
- Cơ sở dữ liệu giải thích bằng lệnh các truy vấn phức tạp được tối ưu hóa.
- Cung cấp quyền truy cập và bảo mật kiểm toán.
- Tùy chọn nhiều người dùng, mật khẩu và mật khẩu thấp
- Lưu trữ, bảo vệ và truy xuất dữ liệu mà họ đã cung cấp.
- Độc lập dữ liệu logic và vật lý.
- Với chỉ số dự phòng thấp.
- Giới thiệu các định dạng để gỡ lỗi và chọn dữ liệu mà chúng tôi quan tâm trong thời gian thực.

Các yếu tố của cơ sở dữ liệu:

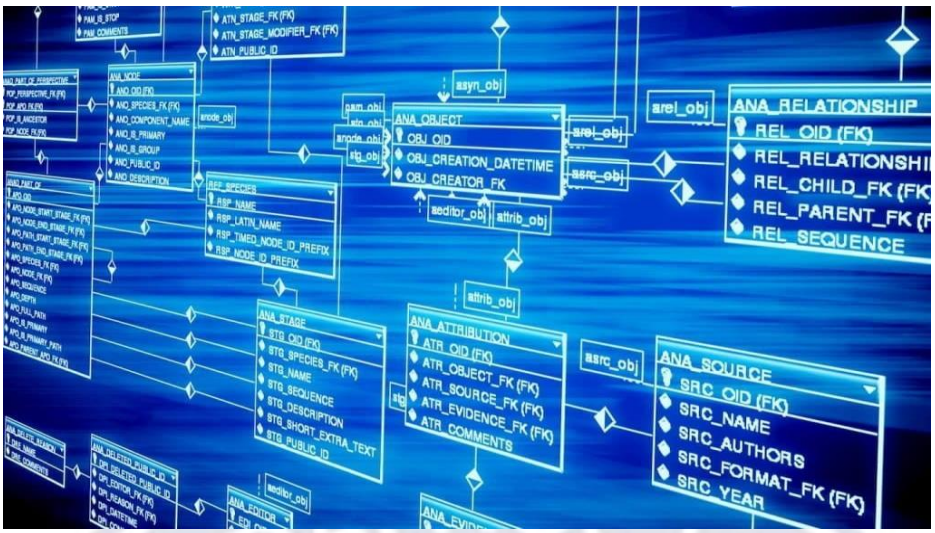
- Về cơ bản, chúng được tạo thành từ 4 yếu tố:
- Bảng: tương ứng với việc tổ chức và sắp xếp dữ liệu theo hàng và cột để dễ đọc.
- Các biểu mẫu: là các đầu vào hoặc giao diện cho phép thực hiện các lệnh để làm việc dữ liệu.
- Báo cáo: là kết quả công việc với các bảng; chúng thường được gọi là báo cáo.
- Tham vấn: họ chịu trách nhiệm về công việc trong cơ sở dữ liệu; Chúng phát ra các đặc điểm cụ thể của quá trình được yêu cầu với dữ liệu trong bảng.

Các loại cơ sở dữ liệu:

- Có nhiều loại cơ sở dữ liệu tồn tại, chúng tôi sẽ đặt tên như sau:
- Truy Cập: Nó là một cơ sở dữ liệu, được phát triển đặc biệt bởi Microsoft, là cơ sở dữ liệu được tạo trong chương trình Access, chương trình này tạo tệp, lưu trữ và bảo vệ dữ liệu một cách có tổ chức.
- PostgreSql và Oracle: Một trong những chương trình cho cơ sở dữ liệu Được sử dụng trong hệ thống này, nó có đặc điểm là quản lý một lượng lớn dữ liệu, nó có thể được quan sát trong mạng nội bộ và các hệ thống có quy mô lớn.
- MySql: Theo giấy phép (General Public License: GPL) làm máy chủ, hệ thống này được phát triển cơ sở dữ liệu được đặc trưng bởi tốc độ của nó, nó không hữu ích hoặc không có chức năng đối với khối lượng lớn dữ liệu.
- Microsoft SQL Server: Sau Microsoft Access, hệ thống này được sử dụng, được đặc trưng bởi việc xử lý một lượng dữ liệu thực, vì nó mạnh hơn Access về khả năng lưu trữ và kiểm soát.

Cơ sở dữ liệu quan hệ:

- Khi **cơ sở dữ liệu** Relational, là loại cơ sở đáp ứng tất cả các đặc điểm của cơ sở trước đó, những cơ sở này được quy hoạch, tổ chức và nâng cao thành tốt nhất trong hệ thống lưu trữ dữ liệu.
- Hệ thống này duy trì dữ liệu này trong hệ thống quản lý cơ sở dữ liệu quan hệ và do đó sử dụng cấu trúc. Ngôn ngữ truy vấn có cấu trúc, cấu trúc này đảm bảo và duy trì chứng nhận của loại hệ thống này.



Hình 1.1. Mô hình cơ sở dữ liệu quan hệ

- Một cơ sở dữ liệu Quan hệ được định hướng trong hai phần, cá thể hoặc dữ liệu và lược đồ:

1.4. Người sử dụng cơ sở dữ liệu

Có nhiều người tham gia vào sự hoạt động của cơ sở dữ liệu với các vai trò khác nhau, với những mục đích khác nhau, thực hiện những nhiệm vụ, những công việc khác nhau. Ta có thể chia các đối tượng này làm hai nhóm.

Nhóm thứ nhất là những người tương tác thường xuyên với cơ sở dữ liệu (actors on the scene). Phần lớn công việc của nhóm này liên quan đến bản thân cơ sở dữ liệu. Thuộc nhóm này ta có những đối tượng chính sau:

- Quản trị cơ sở dữ liệu (Database Administrator: DBA) có nhiệm vụ cấp quyền truy cập cơ sở dữ liệu cho những người có liên quan và giám sát hoạt động của những người này, quản lý các phần cứng và phần mềm tương ứng, chịu trách nhiệm về tình trạng vận hành, về hỏng hóc, về an toàn của hệ cơ sở dữ liệu.
- Thiết kế cơ sở dữ liệu (Database Designer: DBD) có nhiệm vụ xác định những dữ liệu nào cần được lưu trữ, cấu trúc của những dữ liệu ấy, phương pháp thể hiện và lưu trữ các dữ liệu này. Những nhân viên này cũng cần nắm bắt yêu cầu của những đối tượng khác (chủ yếu thuộc nhóm thứ nhất) để thiết kế cơ sở dữ liệu đáp ứng được một cách tốt nhất những yêu cầu ấy, ví dụ tạo ra những khung hiển thị dễ sử dụng, phù hợp, đáp ứng nhanh nhu cầu của người dùng, tạo ra các chương trình xử lý dữ liệu (tính toán, vẽ biểu đồ). Vai trò của những người thiết kế dữ liệu đặc biệt quan trọng trong giai đoạn đầu,

khi thiết kế và bắt đầu triển khai. Khi cơ sở dữ liệu đã hoạt động ổn định, họ có thể chuyển sang các bộ phận khác.

- Người dùng cuối (End user) là những người cần tìm kiếm thông tin và cập nhật thông tin. Đây là đối tượng phục vụ chủ yếu của cơ sở dữ liệu. Họ có thể là một người bên ngoài tổ chức như khách hàng, hay bên trong tổ chức như nhân viên phòng kế toán cần dữ liệu để lập bảng lương, hay nhân viên kho cần cập nhật nguyên liệu hay hàng hóa tồn kho. Họ cũng có thể là những chuyên gia, sử dụng dữ liệu để phân tích tình hình kinh doanh hay năng lực tài chính chẳng hạn.

Nhóm thứ hai thường được gọi là những nhân vật hậu trường (worker behind scene), làm việc chủ yếu với hệ quản trị cơ sở dữ liệu và các chương trình ứng dụng. Nhóm này gồm có:

- Những người thiết kế và xây dựng các thành phần chính của hệ quản trị cơ sở dữ liệu như các mođun để thực hiện các chức năng như truy vấn, tạo từ điển, truy xuất dữ liệu, thực hiện các giao tiếp với hệ điều hành, với các trình biên dịch của các ngôn ngữ, xây dựng các chương trình ứng dụng.
- Những người tạo nên các công cụ, các tiện ích bổ sung nhằm mở rộng, hay nâng cao các tính năng của hệ cơ sở dữ liệu. Các công cụ, tiện ích này có thể là những thành phần bổ sung được cung cấp kèm theo phần mềm, hoặc ở dạng tùy chọn và được mua và/hoặc cài đặt riêng biệt.
- Những người cung cấp các dịch vụ có liên quan như bảo trì phần cứng, sửa chữa các hỏng hóc.

1.5. Khái niệm và tính năng của hệ quản trị cơ sở dữ liêu

Hệ quản trị cơ sở dữ liệu (Database Management System: DBMS) có thể hiểu là một hệ thống được thiết kế để quản lý một lượng lớn thông tin. Cũng có thể là quản lý dữ liệu một cách khoa học có trật tự hệ thống và tự động. Các thao tác quản lý bao gồm thêm, lưu trữ thông tin, sửa, xóa và tìm kiếm trong một nhóm dữ liệu nhất định.

Cụ thể, chức năng của hệ quản trị cơ sở dữ liệu gồm có cung cấp môi trường tạo lập cơ sở dữ liệu, cung cấp môi trường cập nhật và khai thác dữ liệu, và cung cấp công cụ điều khiển truy cập vào cơ sở dữ liệu.

Cung cấp môi trường tạo lập cơ sở dữ liệu:

Người dùng sẽ được cấp một ngôn ngữ định nghĩa dữ liệu để mô tả, khai báo kiểu dữ liệu và các cấu trúc dữ liệu.

Cung cấp môi trường cập nhật và khai thác dữ liệu:

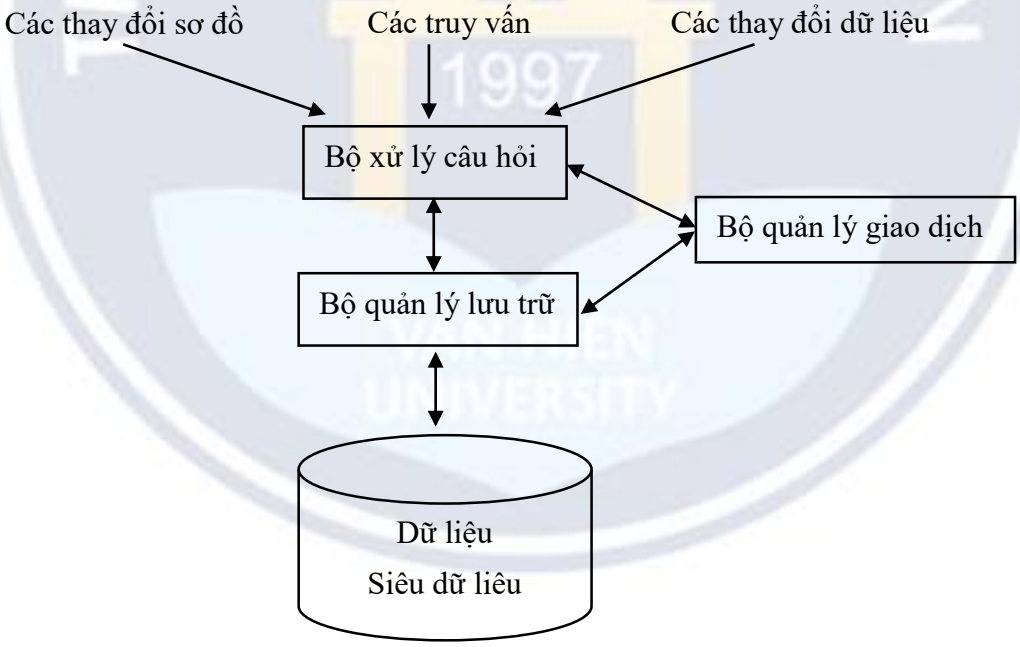
Một hệ quản trị cơ sở dữ liệu cung cấp cho người dùng ngôn ngữ thao tác dữ liệu để diễn tả các yêu cầu. Thao tác dữ liệu gồm: cập nhật (thêm, sửa, xóa dữ liệu) và khai thác (tìm kiếm, truy xuất dữ liệu).

Cung cấp công cụ điều khiển truy cập vào cơ sở dữ liêu:

Nhằm đảm bảo thực hiện một số yêu cầu cơ bản của hệ cơ sở dữ liệu. Bao gồm các công việc: đảm bảo an ninh, phát hiện và ngăn chặn hành động truy cập bất hợp pháp. Đồng thời duy trì tính nhất quán của dữ liệu, điều khiển và tổ chức các hoạt động truy cập. Đôi khi sẽ khôi phục cơ sở dữ liệu khi có sự cố liên quan đến phần cứng hay phần mềm và quản lý chi tiết các mô tả dữ liệu.

1.6. Kiến trúc của hệ quản trị cơ sở dữ liêu

Chúng ta sẽ khái quát kiến trúc và cách thức của một hệ quản trị cơ sở dữ liệu điển hình như mô hình 1.2.



Hình 1.2. Các thành phần chính của hệ quản trị cơ sở dữ liệu

- **Dữ liệu, siêu dữ liệu:** Đáy kiết trúc là thiết bị nhớ ngoài lưu trữ dữ liệu và siêu dữ liệu. Trong phần này không chỉ chứa dữ liệu được trữ trong cơ sở dữ liêu mà chứa cả các siêu dữ liệu, tức là thông tin cấu trúc của cơ sở dữ liêu.

Ví dụ: Trong hệ quản trị cơ sở dữ liệu quan hệ, các siêu dữ liệu bao gồm các tên của các quan hệ, tên các thuộc tính của các quan hệ, và các kiểu dữ liệu đối với các thuộc tính này.

- **Bộ quản lý lưu trữ:** Nhiệm vụ của bộ quản lý lưu trữ là lấy ra các thông tin được yêu cầu từ những thiết bị lưu trữ dữ liệu và thay đổi những thông tin này khi được yêu cầu bởi các mức trên nó của hệ thống.

- **Bộ xử lý câu hỏi:** Bộ xử lý câu hỏi điều khiển không chỉ các câu hỏi mà cả các yêu cầu thay đổi dữ liệu hay siêu dữ liệu. Nhiệm vụ của nó là tìm ra cách tốt nhất một thao tác được yêu cầu và phát ra lệnh đối với bộ quản lý lưu trữ và thực thi thao tác đó.

- **Bộ quản trị giao dịch:** Bộ quản trị giao dịch có trách nhiệm đảm bảo tính toàn vẹn của hệ thống. Nó phải đảm bảo rằng một số thao tác thực hiện đồng thời không cản trở mỗi thao tác khác và hệ thống không mất dữ liệu thậm chí cả khi lỗi hệ thống xảy ra.

  + Nó tương tác với bộ xử lý câu hỏi, do vậy nó phải biết dữ liệu nào được thao tác bởi các thao tác hiện thời để tránh sự đụng độ giữa các thao tác và cần thiết nó có thể làm trễ một số truy vấn nhất định hay một số thao tác cập nhật để đụng độ không thể xảy ra.

  + Nó tương tác với bộ quản lý lưu trữ bởi vì các sơ đồ đối với việc bảo vệ dữ liệu thường kéo theo việc lưu trữ một nhật ký các thay đổi đối với dữ liệu. Hơn nữa, việc sắp thứ tự các thao tác một cách thực sự được nhật ký này sẽ chứa trong một bản ghi đối với mỗi thay đổi khi gặp lỗi hệ thống, các thay đổi chưa được ghi vào đĩa có thể được thực hiện lại.

- **Các kiểu thao tác đối với hệ quản trị cơ sở dữ liêu:** Tại đỉnh kiến trúc, ta thấy có 3 kiểu thao tác:

  + *Các truy vấn:* Đây là các thao tác hỏi đáp về dữ liệu được lưu trữ trong cơ sở dữ liệu. Chúng được sinh ra như hai cách sau:

Thông qua giao diện truy vấn chung:

Ví dụ:

Hệ quản trị cơ sở dữ liêu quan hệ cho phép người sử dụng nhập các câu lệnh truy vấn SQL mà nó được chuyển qua bộ xử lý câu hỏi và được trả lời. ***Thông qua các giao diện chương trình ứng dụng:***

Một hệ quản trị cơ sở dữ liệu điển hình cho phép người lập trình viết các chương trình ứng dụng gọi đến hệ quản trị sở dữ liệu này và truy vấn sở dữ liệu.

+ *Các cập nhật dữ liệu:* Đây là các thao tác thay đổi dữ liệu như xóa, sửa dữ liệu trong cơ sở dữ liêu. Giống như các truy vấn, chúng có thể được phát ra thông qua giao diện chung hoặc thông qua giao diện của chương trình.

+ *Các thay đổi sơ đồ:* Các lệnh này thường được phát bởi một người sử dụng được cấp phép, thường là những người quản trị cơ sở dữ liêu mới được phép thay đổi sơ đồ của cơ sở dữ liêu hay tạo lập một cơ sở dữ liêu mới.

1.7. Một số ứng dụng cơ sở dữ liệu

Do sự phát triển của hệ thống quản lý cơ sở dữ liệu, các công ty có thể lưu trữ và quản lý dữ liệu một cách thuận tiện. Ngoài ra, hệ thống còn giúp doanh nghiệp tìm kiếm thông tin và hồ sơ về bất kỳ người hoặc sản phẩm nào một cách nhanh chóng, trực tiếp làm tăng hiệu quả công việc.

- **Hệ thống đặt trước vé tàu xe:**

Cơ sở dữ liệu được lập trình để lưu hồ sơ đặt vé, tình trạng khởi hành và đến nơi của tàu một cách trực tuyến. Nếu kết hợp với hệ thống vé tàu điện tử, người mua hoàn toàn có thể không phải mua vé trực tiếp. Ở trường hợp này, người mua vé điện tử cũng có thể nhanh chóng nhận được tin tức từ hệ thống nếu lịch trình có thay đổi.

- **Hệ thống quản lý thư viện:**

Có hàng ngàn cuốn sách trong thư viện nên rất khó để theo dõi và quản lý tất cả các cuốn sách trong một bảng tính hoặc qua giấy tờ. Vì vậy, ứng dụng của cơ sở dữ liệu được sử dụng để duy trì tất cả các thông tin liên quan đến ngày phát hành sách, tên của cuốn sách, tác giả và trạng thái của cuốn sách.

- **Ngân hàng:**

Với ứng dụng quản lý tất cả các giao dịch trực tuyến thông qua hệ thống cơ sở dữ liệu, khách hàng ở khắp mọi nơi có thể thực hiện hàng ngàn giao dịch qua ngân hàng hằng ngày mà không cần phải đến ngân hàng. Ngoài ra, các thông tin người dùng để sở hữu thẻ tín dụng và thực hiện các giao dịch của chủ thẻ, được theo dõi và tính toán tự động trên hệ thống cơ sở dữ liệu.

- **Hệ thống giáo dục cao cấp:**

Mọi thông tin về quá trình học tập của sinh viên như thông tin đăng ký của sinh viên, kết quả học tập, các khóa học đã tham gia được lưu trữ trực tiếp trên hệ

thống cơ sở dữ liệu của nhà trường. Ngày nay, nhiều trường đại học, cao đẳng còn bắt đầu áp dụng hình thức thi trực tuyến, tự động chấm điểm và đẩy kết quả lên hệ thống cơ sở dữ liệu này.

- **Mạng xã hội:**

Hàng triệu người dùng đã đăng ký các tài khoản mạng xã hội như Facebook, Twitter, Pinterest và Zalo. Và tất cả thông tin của người dùng, trạng thái chia sẻ, hoạt động kết nối, trao đổi giữa các tài khoản sẽ được lưu trữ trên hệ thống cơ sở dữ liệu lớn của các mạng xã hội này.

- **Viễn thông:**

Bất kỳ công ty viễn thông nào cũng không thể duy trì việc kinh doanh của mình mà không ứng dụng cơ sở dữ liệu. Hệ thống cơ sở dữ liệu sẽ giúp các công ty này lưu trữ các chi tiết cuộc gọi và hóa đơn thanh toán hàng tháng.

- **Tài chính:**

Trước khi cơ sở dữ liệu được ứng dụng rộng rãi, thông tin liên quan đến dòng tiền được lưu trữ trong sổ đăng ký và tập tin. Ngày nay thời gian đã hoàn toàn thay đổi bởi vì cơ sở dữ liệu ngành tài chính đóng vai trò người hỗ trợ đắc lực cho việc lưu trữ bán hàng, nắm giữ thông tin và quản lý báo cáo tài chính, v.v.

- **Quân đội:**

Quân đội lưu giữ hồ sơ của hàng triệu binh sĩ và nó có hàng triệu hồ sơ cần được bảo mật và an toàn. Hệ thống quản trị cơ sở dữ liệu cung cấp một sự đảm bảo an ninh lớn cho thông tin quân sự nên nó được sử dụng rộng rãi trong quân đội các nước. Nhà quản lý có thể dễ dàng tìm kiếm tất cả thông tin về bất cứ ai trong vòng vài giây với sự trợ giúp của hệ thống quản trị cơ sở dữ liệu.

- **Mua sắm trực tuyến:**

Mua sắm trực tuyến đã trở thành một xu hướng mới và ngày một phát triển trong kỉ nguyên số bởi sự nhanh chóng, tiện lợi của nó. Nhiều người không biết rằng, tất cả các sàn thương mại điện tử đều không thể quản lý, thêm và bán các sản phẩm nếu không có sự trợ giúp của hệ thống quản trị cơ sở dữ liệu. Thông tin mua hàng, hóa đơn và thanh toán, tất cả đều được lưu trữ trên cơ sở dữ liêu.

- **Quản trị nhân sự:**

Các công ty lớn có số lượng nhân công các bộ phận, chi nhánh lên tới hàng nghìn nhân công, không thể quản lý giản đơn bằng bảng tính hoặc giấy tờ. Vì vậy,

bộ phận quản lý nhân sự cần phải lưu giữ hồ sơ của từng nhân viên lương, thuế và công việc thông qua Hệ thống quản trị cơ sở dữ liệu.

- **Chế tạo:**

Cơ sở dữ liệu được ứng dụng tại các công ty sản xuất để lưu giữ hồ sơ của tất cả các chi tiết về các sản phẩm như số lượng, hóa đơn, mua hàng, quản lý chuỗi cung ứng,… Nó đóng vai trò nền tảng để ứng dụng hệ thống hoạch định nguồn lực doanh nghiệp bằng phần mềm ERP (Enterprise Resource Planning).

Với những ứng dụng của cơ sở dữ liệu trong thực tế như đã nói tới trên đây, có thể nhận định rằng, đây sẽ là xương sống của nền kinh tế trong kỉ nguyên số. Vì vậy, những doanh nghiệp, tổ chức nên sớm có phương án chuẩn bị về hạ tầng công nghệ thông tin để kịp thời ứng dụng vào điều kiện thực tế.

BÀI TẬP CHƯƠNG 1

1. Định nghĩa các thuật ngữ : cơ sở dữ liệu, hệ quản trị cơ sở dữ liệu, hệ cơ sở dữ liệu, từ điển cơ sở dữ liệu, mô hình cơ sở dữ liệu.

2. Nêu các tính chất của một cơ sở dữ liệu

3. Nêu các chức năng của một hệ quản trị cơ sở dữ liệu

4. Giải thích các đặc trưng của giải pháp cơ sở dữ liệu

5. Định nghĩa mô hình cơ sở dữ liệu và phân loại

6. Liệt kê các người có liên quan đến hệ cơ sở dữ liệu.

Chương 2
MÔ HÌNH THỰC THỂ KẾT HỢP

2.1. Quá trình thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu là quá trình mô hình hóa nhằm chuyển đổi các đối tượng từ thế giới thực (Real-world system) sang các bảng trong hệ thống cơ sở dữ liệu (Database system) đáp ứng các yêu cầu lưu trữ và khai thác dữ liệu.

People (con người): Những người tham gia vào hệ thống, bạn cần làm việc với những người này để xác định các dữ liệu cần lưu trữ, cần khai thác.

Documents (tài liệu): Bạn cần khảo sát các tài liệu trong hệ thống để xác định dữ liệu.

Facilities (cơ sở vật chất): Bạn cần quan tâm những cơ sở vật chất nào cần quản lý.

Other systems (hệ thống khác, hệ thống tương tự): Bạn cần tìm hiểu nghiên cứu các hệ thống tương tự để thu thập thêm dữ liệu.

Để thực hiện việc thiết kế cơ sở dữ liệu chúng ta cần thực hiện các Nguyên tắc sau đây:

- **Nguyên tắc một:**

Xác định thông tin trùng lặp (hay còn gọi là dữ liệu dự phòng) là có hại vì thông tin này gây lãng phí về dung lượng và làm tăng khả năng lỗi cũng như sự không nhất quán.

- **Nguyên tắc hai:**

Tính chính xác và sự hoàn chỉnh của thông tin rất quan trọng. Nếu cơ sở dữ liệu của bạn có chứa thông tin không chính xác thì mọi báo cáo sử dụng thông tin từ cơ sở dữ liệu đó cũng sẽ chứa thông tin không chính xác. Kết quả là mọi quyết định bạn đưa ra dựa trên các báo cáo đó sẽ trở thành thông tin sai lệch.

- **Do đó, thiết kế cơ sở dữ liệu tốt là một thiết kế:**
  - Phân chia thông tin của bạn vào các bảng theo đối tượng để giảm thông tin thừa.
  - Cung cấp cho truy cập thông tin cần thiết để kết hợp thông tin trong các bảng với nhau, khi cần.
  - Giúp hỗ trợ và đảm bảo tính chính xác cũng như tính toàn vẹn cho thông tin của bạn.

+ Điều chỉnh nhu cầu báo cáo và xử lý dữ liệu của bạn.

- **Quy trình thiết kế bao gồm những bước sau:**

Bước 1: Xác định mục đích của cơ sở dữ liệu

Việc này giúp bạn chuẩn bị cho các bước tiếp theo.

Bước 2: Tìm và sắp xếp thông tin cần thiết

Thu thập mọi loại thông tin bạn có thể muốn ghi vào cơ sở dữ liệu, như mả sản phẩm, tên sản phẩm, số lượng và số đơn hàng…

Bước 3: Phân chia thông tin vào các bảng

Phân chia mục thông tin vào các thực thể hoặc đối tượng chính, như Sản phẩm hoặc Đơn hàng. Mỗi đối tượng sau đó sẽ trở thành một bảng.

Bước 4: Biến mục thông tin thành các cột

Quyết định loại thông tin bạn muốn lưu trữ trong từng bảng. Mỗi mục sẽ trở thành một trường và được hiển thị dưới dạng cột trong bảng.

Ví dụ: Một bảng Nhân viên có thể chứa các trường như Mã nhân viên, Họ tên, và Ngày Tuyển dụng…

Bước 5: Chỉ định khóa chính

Chọn khóa chính cho mỗi bảng. Khóa chính là cột, dùng để xác định từng hàng riêng.

Ví dụ: Có thể là ID Sản phẩm hoặc ID Đơn hàng.

Bước 6: Thiết lập mối quan hệ cho bảng

Xem bảng và quyết định mối quan hệ của dữ liệu trong bảng đó với dữ liệu ở các bảng khác. Thêm các trường vào bảng hoặc tạo bảng mới để làm rõ mối quan hệ, nếu cần.

Bước 7: Tinh chỉnh thiết kế của bạn

Phân tích thiết kế để tìm lỗi. Tạo bảng và thêm một số bản ghi dữ liệu mẫu. Xem liệu bạn có thể nhận được kết quả mong muốn từ bảng không. Tiến hành điều chỉnh thiết kế, nếu cần.

Bước 8: Áp dụng các quy tắc chuẩn hóa

Áp dụng các quy tắc chuẩn hóa dữ liệu để xem liệu bảng của bạn đã được cấu trúc phù hợp chưa. Tiến hành điều chỉnh bảng, nếu cần.

- **Xác định mục đích của cơ sở dữ liệu:**

Tốt nhất, bạn nên ghi lại mục đích của cơ sở dữ liệu ra giấy, bạn dự định sử dụng cơ sở dữ liệu như thế nào và ai sẽ sử dụng cơ sở dữ liệu. Đối với cơ sở dữ liệu nhỏ dành cho doanh nghiệp gia đình.

Ví dụ: Bạn có thể ghi đơn giản như "Cơ sở dữ liệu khách hàng lưu danh sách thông tin khách hàng cho mục đích tạo quảng cáo qua thư và tạo báo cáo". Nếu cơ sở dữ liệu phức tạp hơn hoặc có nhiều người dùng, như thường xảy ra trong môi trường doanh nghiệp thì mục đích có thể đơn giản chỉ là một đoạn văn hoặc nhiều hơn và cần bao gồm thời gian, cũng như cách mỗi người sử dụng cơ sở dữ liệu. Tốt nhất, bạn nên có tuyên bố sứ mệnh được xây dựng tốt có thể được tham chiếu trong suốt quy trình thiết kế. Việc có tuyên bố này sẽ giúp bạn tập trung vào mục tiêu khi đưa ra quyết định.

- **Tìm và sắp xếp thông tin cần thiết:**

Để tìm và sắp xếp thông tin cần thiết, hãy bắt đầu bằng thông tin hiện có. Bạn có thể ghi lại đơn hàng vào sổ cái hoặc giữ lại thông tin khách hàng trên biểu mẫu giấy trong tủ hồ sơ. Thu thập những tài liệu đó và liệt kê từng loại thông tin hiển thị, như mỗi hộp bạn điền trên một biểu mẫu. Nếu bạn không có sẵn bất cứ biểu mẫu nào thì thay vào đó, hãy tưởng tượng bạn phải thiết kế một biểu mẫu để ghi lại thông tin khách hàng. Bạn muốn đưa thông tin nào vào biểu mẫu? Bạn sẽ tạo những ô nào để điền? Xác định và liệt kê từng mục trong số này.

Ví dụ: Giả sử bạn hiện đang giữ danh sách khách hàng trên thẻ chỉ mục. Việc kiểm tra các thẻ này có thể cho biết rằng mỗi thẻ lưu giữ một tên khách hàng, địa chỉ, thành phố, tiểu bang, mã bưu chính và số điện thoại. Mỗi mục trong số này đại diện cho một cột tiềm năng trong bảng.

Khi bạn chuẩn bị danh sách này, đừng lo lắng về việc phải tạo bảng hoàn hảo ngay từ đầu. Thay vào đó, hãy liệt kê từng mục bạn nghĩ ra. Nếu người khác sử dụng cơ sở dữ liệu, hãy hỏi cả ý kiến của họ. Bạn có thể tinh chỉnh danh sách sau đó.

Tiếp theo, hãy cân nhắc loại báo cáo hoặc quảng cáo qua thư bạn có thể muốn tạo ra từ cơ sở dữ liệu.

Ví dụ: Bạn có thể muốn báo cáo doanh số sản phẩm hiển thị doanh số theo khu vực hoặc báo cáo tóm tắt hàng tồn kho hiển thị mức tồn kho sản phẩm. Có thể bạn cũng muốn tạo thư biểu mẫu để gửi đến khách hàng, công bố một sự kiện bán

hàng hoặc đưa ra phí trả thêm. Thiết kế báo cáo trong đầu và tưởng tượng báo cáo sẽ trông như ra sao. Bạn muốn đưa thông tin nào vào báo cáo? Liệt kê từng mục. Thực hiện tương tự cho thư biểu mẫu và cho mọi báo cáo khác bạn dự định tạo.

Việc đưa ý tưởng vào báo cáo và thư tín bạn có thể muốn tạo sẽ giúp xác định được những mục cần thiết trong cơ sở dữ liệu.

Ví dụ: Giả sử bạn mang đến cho khách hàng cơ hội chọn tham gia (hoặc không tham gia) cập nhật Email định kỳ và bạn muốn in một danh sách những người đã chọn tham gia đó. Để ghi lại thông tin đó, bạn cần thêm một cột "Gửi Email" vào bảng khách hàng. Đối với mỗi khách hàng, bạn có thể đặt trường là Có hoặc Không.

Yêu cầu gửi thư Email cho khách hàng sẽ đề xuất một mục khác để ghi lại. Khi bạn biết có khách hàng muốn nhận thư Email, bạn cũng sẽ cần biết địa chỉ Email để gửi thư cho họ. Do đó, bạn cần ghi lại địa chỉ Email của từng khách hàng.

Việc tạo một mẫu thử nghiệm cho mỗi báo cáo hoặc danh sách đầu ra và cân nhắc những mục bạn cần để tạo báo cáo.

- **Mô hình hóa việc thiết kế cấu trúc cơ sở dữ liệu:**
  + People (con người): những người tham gia vào hệ thống, bạn cần làm việc với những người này để xác định các dữ liệu cần lưu trữ, cần khai thác.
  + Documents (Tài liệu): bạn cần khảo sát các tài liệu trong hệ thống để xác định dữ liệu.
  + Facilities (Cơ sở vật chất): bạn cần quan tâm những cơ sở vật chất nào cần quản lý.
  + Other systems (Hệ thống khác, hệ thống tương tự): bạn cần tìm hiểu nghiên cứu các hệ thống tương tự để thu thập thêm dữ liệu.

  Để thực hiện việc thiết kế cơ sở dữ liệu chúng ta cần thực hiện như sau:

  1. Xác định các thành phần dữ liệu.
  2. Chia nhỏ các thành phần dữ liệu ra thành các phần nhỏ nhất mà hệ thống sử dụng.
  3. Xác định các bảng và các cột.
  4. Xác định khóa chính, khóa ngoại và mối quan hệ.
  5. Kiểm tra cấu trúc cơ sở dữ liệu được thiết kế với qui định chuẩn hóa.

2.2. Mô hình thực thể kết hợp

2.2.1. Thực thể (Entity)

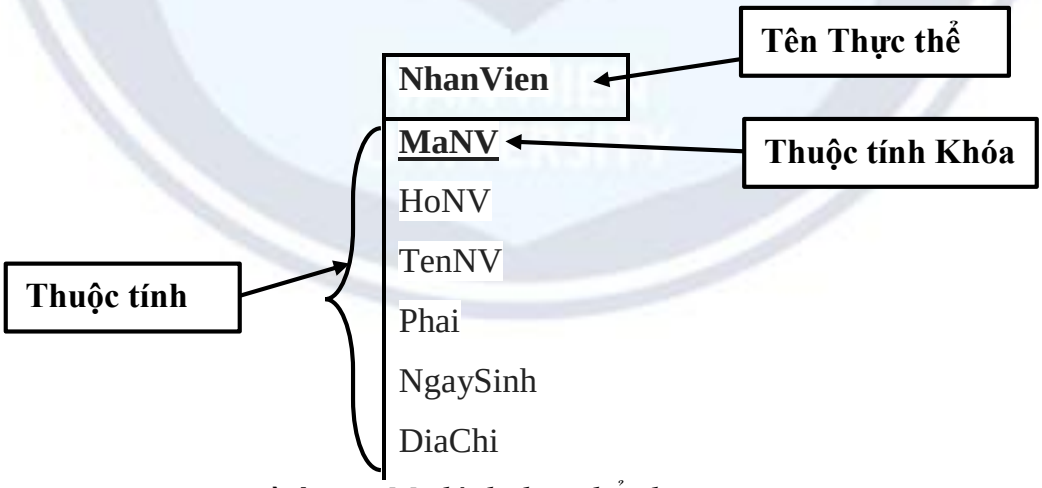
Đối tượng được trình bày trong mô hình ER (Entity-Relationship Model) là thực thể. Thực thể là một "Vật" trong thế giới thực, có sự tồn tại độc lập. Một thực thể có thể là cụ thể, tức là chúng ta có thể cảm nhận được bằng các giác quan, hoặc có thể là trừu tượng, tức là cái mà chúng ta không cảm nhận được bằng các giác quan nhưng có thể nhận biết được bằng nhận thức. Một xe ô tô, một nhân viên,… là những thực thể cụ thể. Một đơn vị công tác, một trường học… là những thực thể trừu tượng…

Mỗi một thực thể có các thuộc tính, đó là các đặc trưng cụ thể mô tả thực thể đó, một thực thể Nhânviên được mô tả bằng Mã nhân viên, Họtên, Phái, Địachỉ, … của nhân viên đó. Một thực thể cụ thể sẽ có một giá trị cho mỗi thuộc tính của nó.

Ví dụ: Nhân viên **NV1** có các giá trị cho các thuộc tính Mã nhân viên, Họ tên, Phái, Địachỉ là, "NV001", " Lê Hùng", "Nam", "Tp. HCM". Các giá trị thuộc tính mô tả mỗi thực thể sẽ trở thành một phần chính của các dữ liệu sẽ được lưu giữ trong cơ sở dữ liệu. Trong mô hình ER có mặt nhiều kiểu thuộc tính: thuộc tính đơn, thuộc tính phức hợp, thuộc tính đơn trị, thuộc tính đa trị, thuộc tính được lưu trữ, thuộc tính suy diễn được, thuộc tính có giá trị không xác định, thuộc tính phức tạp.

Thực thể là đối tượng cần quản lý, được biểu diễn thực thể bởi hình chữ nhật và có danh sách các thuộc tính.

Ví dụ: Mô hình thực thể nhân viên



Hình 2.1. Mô hình thực thể nhân viên

Các kiểu thực thể yếu: Các kiểu thực thể không có các thuộc tính khoá cho chính mình được gọi là các kiểu thực thể yếu. Ngược lại, các kiểu thực thể thông thường (nghĩa là có thuộc tính khoá) được gọi là kiểu thực thể mạnh. Các thực thể của một kiểu thực thể yếu được xác định bằng cách liên kết với các thực thể cụ thể

của một kiểu thực thể khác phối hợp với một số giá trị thuộc tính của nó. Ta gọi kiểu thực thể khác đó là kiểu thực thể xác định hoặc *kiểu thực thể chủ.* Ta gọi kiểu liên kết giữa kiểu thực thể yếu và kiểu thực thể chủ của nó là liên kết xác định của thực thể yếu. Một kiểu thực thể yếu luôn luôn có một ràng buộc tham gia toàn bộ (tồn tại phụ thuộc) vào liên kết xác định của nó bởi vì một kiểu thực thể yếu không thể được xác định mà không có kiểu thực thể chủ.

Ví dụ: Trong một công ty, con của nhân viên và nhân viên có thể hưởng chế độ bảo hiểm theo nhân viên. Như vậy, sẽ có một kiểu liên kết NHÂNVIÊN <có> <CON>. Đây là một kiểu liên kết có tỷ số lực lượng **[1:n]**. Các thuộc tính của kiểu thực thể CON là Họtên, Ngaysinh, Giớitính. Hai người con của hai nhân viên khác nhau có thể có cùng giá trị cho các thuộc tính nhưng nó là hai thực thể khác nhau. Chúng chỉ được xác định như hai thực thể khác nhau sau khi xác định một thực thể nhân viên cụ thể có liên quan đến từng người phụ thuộc. Mỗi thực thể của kiểu thực thể NHÂNVIÊN được gọi là chủ của các thực thể của kiểu thực thể CON liên kết với nó.

Thông thường, các kiểu thực thể yếu có một khoá bộ phận, đó là một tập hợp các thuộc tính có thể xác định một cách duy nhất các thực thể yếu liên kết với cùng một thực thể chủ.

Ví dụ: Nếu hai người con của một nhân viên không bao giờ có tên giống nhau thì thuộc tính Họ tên của kiểu thực thể CON là một khoá bộ phận. Trong trường hợp xấu nhất, thuộc tính phức hợp gồm tất cả các thuộc tính của thực thể yếu sẽ là một khóa bộ phận.

2.2.2. Thuộc tính (Attribute)

Thuộc tính là đặc tính vốn có của một sự vật, nhờ đó sự tồn tại và qua đó con người nhận thức được sự vật, phân biệt được sự vật này với sự vậy khác, màu sắc là một thuộc tính của mọi vật thể.

Trong toán học thuộc tính là một đặc tả định nghĩa đặc tính của một đối tượng, phần tử, hay tập tin, nó còn có thể chỉ đến giá trị cho một thực thể cụ thể.

Tuy nhiên, trong thực tế, thuật ngữ thuộc tính có thể sử dụng và xem là tương đương của đặc tính (Property) phụ thuộc vào ngữ cảnh công nghệ.

Một thuộc tính của một đối tượng thường chứa một tên và một giá trị; của một phần tử, một loại hay tên lớp, của một tập tin, một tên và phần mở rộng.

Thuộc tính bao gồm các loại như sau:

Thuộc tính đơn: là thuộc tính không thể phân chia ra được thành các thành phần nhỏ hơn.

Ví dụ: Thuộc tính MaNV(Mã nhân viên), Tuoi(Tuổi) của một nhân viên là một thuộc tính đơn.

Thuộc tính phức hợp: là thuộc tính có thể phân chia được thành các thành phần nhỏ hơn, biểu diễn các thuộc tính cơ bản hơn với các ý nghĩa độc lập.

Ví dụ: Thuộc tính Hoten(Họ tên) của thực thể nhân viên có thể phân chia thành các thuộc tính Hodem(Họ đệm) và Ten(Tên).

Giá trị của một thuộc tính là sự kết hợp kết hợp các giá trị của các thuộc tính thành phần tạo nên nó. Việc phân chia một thuộc tính phức hợp thành các thuộc tính đơn tùy thuộc vào hoàn cảnh cụ thể. Những thuộc tính có giá trị duy nhất cho một thực thể cụ thể gọi là các *thuộc tính đơn trị.*

Ví dụ: Hoten là một thuộc tính đơn trị của thực thể nhân viên, mỗi nhân viên có một họ tên duy nhất. Trong một số trường hợp, một thuộc tính có thể có một tập giá trị cho cùng một thực thể. Những thuộc tính như vậy gọi là *thuộc tính đa trị.*

Ví dụ: thuộc tính BangCap(Bằng cấp) của một người. Một người có thể không có bằng cấp nào, người khác có thể có một bằng hoặc có thể có nhiều bằng. Như vậy, các người khác nhau có thể có một số giá trị khác nhau cho thuộc tính Bằng cấp. Thuộc tính Bằng cấp là một thuộc tính đa trị.

Thuộc tính được lưu trữ: Là các thuộc tính mà giá trị của nó được nhập vào khi cài đặt cơ sở dữ liệu. Trong một số trường hợp, hai hay nhiều thuộc tính có giá trị liên quan đến nhau.

Ví dụ: Thuộc tính Tuổi và thuộc tính Ngày sinh của một người.Với một người cụ thể, ta có thể tính tuổi của anh ta bằng cách lấy năm hiện tại trừ đi năm của Ngày sinh. Thuộc tính mà giá trị của nó có thể tính được thông qua giá trị của các thuộc tính khác gọi là thuộc tính suy diễn được.

Các giá trị không xác định (null values): Trong một số trường hợp, một thực thể cụ thể có thể không có các giá trị áp dụng được cho một thuộc tính.

Ví dụ: Thuộc tính Số điện thoại của thực thể nhân viên sẽ không có giá trị đối với các nhân viên không có số điện thoại. Trong trường hợp như vậy, ta phải tạo ra

một giá trị đặc biệt gọi là giá trị không xác định (null). Giá trị không xác định được tạo ra khi một thuộc tính có giá trị không áp dụng được hoặc khi không biết.

Các thuộc tính phức tạp: Là sự kết hợp của các thuộc tính phức hợp và đa trị

Thuộc tính Khóa: Một ràng buộc quan trọng trên các thực thể của một kiểu thực thể là ***khóa***. Một kiểu thực thể thường có một thuộc tính mà các giá trị của nó là khác nhau đối với mỗi thực thể riêng biệt trong một tập thực thể. Thuộc tính như vậy gọi là ***thuộc tính khóa*** và các giá trị của nó có thể dùng để xác định từng thực thể một cách duy nhất.

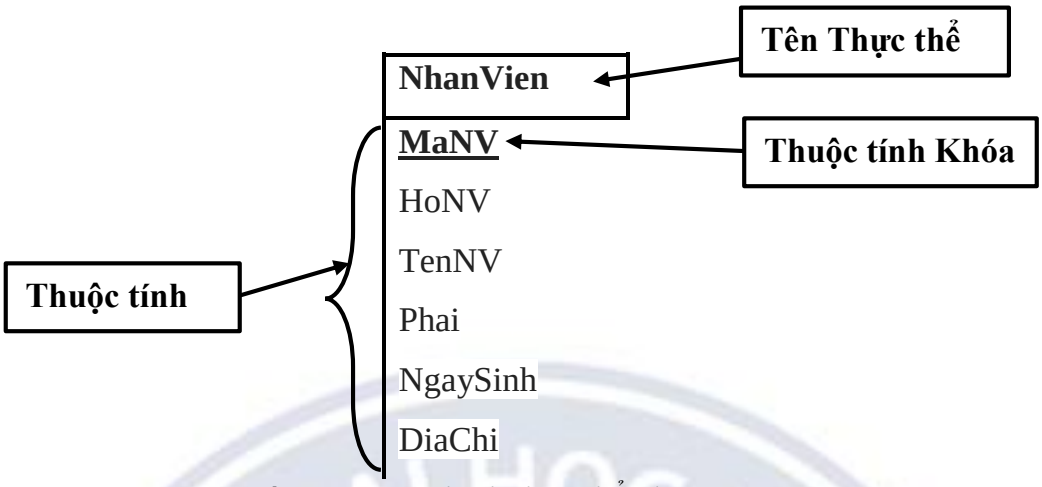
Ví dụ: Thuộc tính Tên của kiểu thực thể CÔNGTY là khóa của kiểu thực thể đó vì mỗi thực thể công ty có một tên duy nhất. Đôi khi, nhiều thuộc tính kết hợp với nhau tạo thành một khóa, nghĩa là tổ hợp các giá trị của các thuộc tính này phải khác nhau đối với mỗi thực thể. Trong trườ ng hợp như vậy ta có một thuộc tính ***khóa phức hợp.***

Chú ý: Khóa phức hợp phải tối thiểu, nghĩa là tất cả các thuộc tính thành phần phải có mặt trong thuộc tính phức hợp để thỏa mãn tính chất duy nhất. Trong biểu đồ đồ họa của mô hình ER, thuộc tính khóa được biểu diễn bằng cách gạch ngang dưới tên của nó.

Khi chỉ ra rằng một thuộc tính là khóa của một kiểu thực thể nghĩa là tính chất duy nhất nêu trên phải được thỏa mãn đối với mỗi khi mở rộng của kiểu thực thể. Như vậy, ràng buộc khóa cấm hai thực thể bất kỳ có giá trị cho thuộc tính khóa như nhau tại cùng một thời điểm. Đó là một ràng buộc trên tất cả các thể hiện của thực thể. Ràng buộc khóa cũng như các ràng buộc sẽ được giới thiệu về sau được lấy ra từ các ràng buộc của "Thế giới nhỏ" của cơ sở dữ liệu. Một kiểu thực thể có thể có nhiều hơn một thuộc tính khóa.

Ví dụ: Nếu một công ty có một mã số duy nhất và một tên duy nhất thì các thuộc tính Mã số công ty và Tên công ty đều là các thuộc tính khóa. Một kiểu thực thể cũng có thể không có khóa. Một thực thể không có khóa được gọi là ***kiểu thực thể yếu.***

Ví dụ: Mô hình thực thể Nhân viên(NhanVien)



Hình 2.2. Mô hình thực thể nhân viên

Ta có thuộc tính mã nhân viên(MaNV) làm khóa vậy tại thuộc tính này giá trị của nó là duy nhất để phân biệt nhân viên này với nhân viên khác.

Miền giá trị: Mỗi tập thực thể có một tập các tính chất đặc trưng, mỗi tính chất đặc trưng này gọi là thuộc tính của tập thực thể. Ứng với mỗi thuộc tính có một tập các giá trị cho thuộc tính đó gọi là miền giá trị.

Miền giá trị của thuộc tính gồm các kiểu giá trị như sau:

- Kiểu chuỗi (string/varchar)
- Kiểu số nguyên (integer)
- Kiểu số thực (real)
- Kiểu ngày tháng năm giờ phút giây (datetime)
- Kiểu ngày tháng năm (date)
- ...

Ví dụ: Tập thực thể SINHVIEN có các thuộc tính

- Mã Sinh viên (MaSV: string[10])
- Họ tên (Hoten: string[20])
- Ngày sinh (NS: date)
- Điểm TB (DTB: float)
- …

Ví dụ: Các loại giá trị của thuộc tính:

- Đơn trị: Các thuộc tính có giá trị duy nhất cho một thực thể.

(Ví dụ: Số CCCD, …)

- Đa trị: Các thuộc tính có một tập giá trị cho cùng một thực thể

(Ví dụ: Bằng cấp, …)

- Suy diễn được:

(Ví dụ: Năm sinh ↔ Tuổi)

Mỗi thực thể đều được phân biệt bởi thuộc tính khóa.

Ví dụ: Tập thực thể NHANVIEN gồm có các thuộc tính như sau và thuộc tính khóa là Mã nhân viên

- **<u>Mã NV</u>** (MaNV: integer)
- Họ tên (Hoten: string[50])
- Ngày sinh (NS: date)
- Địa chỉ (Diachi: string[100])
- Quê quán (Quequan: string[30])
- Hệ số lương (Hsluong: float)
- Hệ số phụ cấp (Hsphucap: float)
- Tổng lương (Togluong: float)

2.2.3. Mối kết hợp

Mối kết hợp thể hiện mối liên quan giữa hai hay nhiều thực thể. Mỗi kết hợp có một tên gọi và thường dùng động từ.

Mối kết hợp là mô hình được biểu diễn bởi những **mẫu tin (record)**, **loại mẫu tin**, **loại liên hệ giữa các mẫu tin** và **bản số của mẫu tin**.

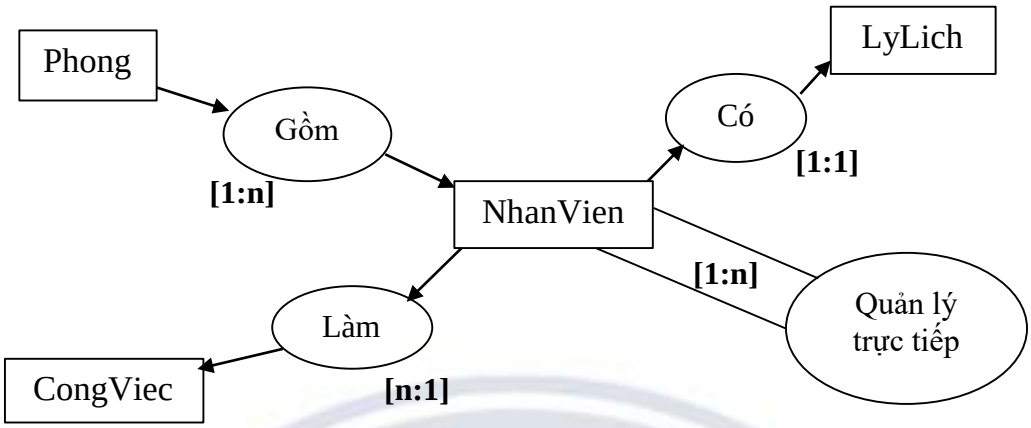
Mẫu tin: Là tin mô tả một đối tượng trong thế giới thực.

Loại liên hệ: Mô tả mối liên hệ giữa các loại mẫu tin.

Bản số mỗi nhánh của mối kết hợp: Là một cặp số tự nhiên **[Min:Max]** Chỉ ra số lượng các mẫu tin tham gia trong các mối kết hợp, chúng ta sẽ có những mối kết hợp sau đây:

Ví dụ: Mô hình kết hợp dưới đây thể hiện theo môn hình dữ liệu mạng (Network Data Model) là mô hình được biểu diễn bởi những mẫu tin (Record).

Các bạn nhớ để ý luồng đi của mũi tên nhé, nó đi theo hướng cấp cao đến cấp thấp. Đi từ PHONG đi đến NHANVIEN vì NHANVIEN là con của PHONG, từ NHANVIEN đi đến LYLICH vì mỗi lý lịch thuộc về NHANVIEN. Từ mũi tên đó ta sẽ tìm được mối quan hệ [1:n] [n:1] [1:1] (**Recursive**) một cách dễ dàng.



Hình 2.3. Sơ đồ mô hình dữ liệu mạng

Mối kết hợp [1:1] (one-to-one):

Mỗi mẫu tin của loại mẫu tin chủ kết hợp đúng với một mẫu tin của loại mẫu tin thành viên.

Mỗi nhân viên có một lý lịch (1) và mỗi lý lịch thuộc về một nhân viên (1), đây là quan hệ [1:1]

Mối kết hợp [1:n] (one-to-many): Mỗi mẫu tin của loại mẫu tin chủ kết hợp với 1 hay nhiều mẫu tin thành viên.

Mỗi phòng có nhiều nhân viên (n) và mỗi nhân viên chỉ thuộc một phòng (1), vậy đây là mối quan hệ [1:n]

Mối kết hợp [n:1] (many-to-one): Nhiều mẫu tin của loại mẫu tin chủ kết hợp với những 1 mẫu tin của loại mẫu tin thành viên.

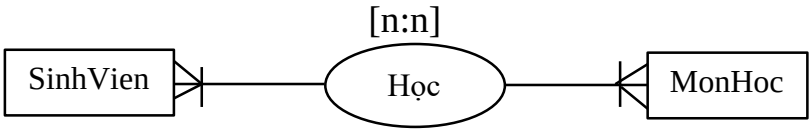
Mỗi nhân viên làm nhiều công việc (n) và mỗi công việc có một nhân viên (1) làm, đây là mối quan hệ [n:1]

Mối kết hợp đệ quy (Recursive): Một loại mẫu tin chủ cũng có thể đồng thời là loại mẫu tin thành viên với chính nó.

Môi nhân viên sẽ có một người quản lý trực tiếp (trưởng nhóm), và mỗi nhóm có thể có nhiều nhân viên, đây là mối quan hệ (Recursive) vì nó tự chỉ đến nó.

Mối kết hợp [n:n] (many-to- many): Mỗi đối tượng của thực thể thứ nhất tương ứng với nhiều đối tượng của thực thể thứ 2 và ngược lại.

Ví dụ: Mỗi sinh viên học **một hoặc nhiều môn học. Một môn học** có **một hoặc nhiều sinh viên** học.



Hình 2.4. Sơ đồ mô hình mối kết hợp [n:n]

Một số mô hình thực thể mối kết hợp mở rộng:

Chuyên biệt hóa:

Giả sử ta có hai **loại thực thể** là GIAOVIEN(Giáo viên) và HOCVIEN (Học viên) thì nếu để ý kỹ thì bạn sẽ nhận ra rằng cả hai đều là **loại thực thể** CONNGUOI(Con người). Tại sao ta lại phân ra làm hai loại như vậy mà không gộp chung lại một?

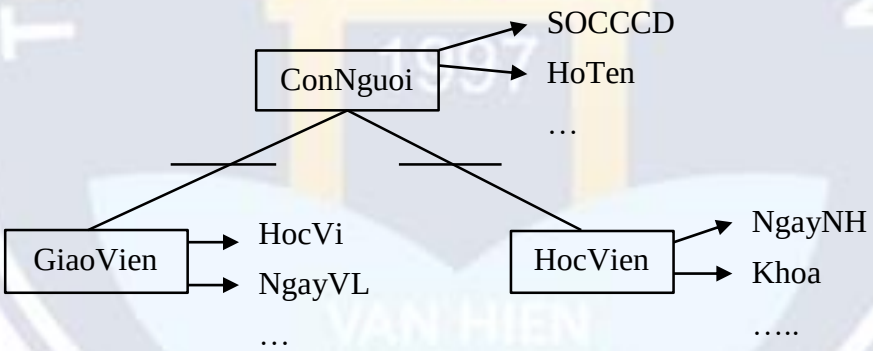
Lý do là mỗi loại thực thể sẽ có một số thuộc tính khác nhau. Nếu để ý kỹ hơn nữa thì ta thấy loại thực thể CONNGUOI sẽ chứa một số thuộc tính dùng chung cho cả hai loại thực thể GIAOVIEN và HOCVIEN.

Với GIAOVIEN ta sẽ có một số thuộc tính như (MaGiaoVien, CapBac, ChuNhiemLop, ...)

Với HOCVIEN ta sẽ có một số thuộc tính như (MaHocVien, Lop, Khoa, ...)

Với CONNGUOI ta sẽ có một số thuộc tính như (SOCCCD, GioiTinh, ...)

Ta có một sơ đồ ER mở rộng như sau:



Hình 2.5. Sơ đồ mối kết hợp Chuyên biệt hóa

Mối kết hợp đệ quy:

Mối kết hợp đệ quy là loại mối kết hợp được tạo thành từ cùng một loại thực thể, hay nói cách khác thì loại thực thể đó tự trỏ lại chính nó.

Ví dụ: Trong công ty ABC có danh sách nhân viên và trong đó có một nhân viên làm trưởng nhóm (Leader) quản lý tất cả các nhân viên còn lại. Mô hình như sau:

Hình 2.6. Sơ đồ mối kết hợp đệ quy

Mối kết hợp mở rộng có nhiều mối kết hợp với nhau:

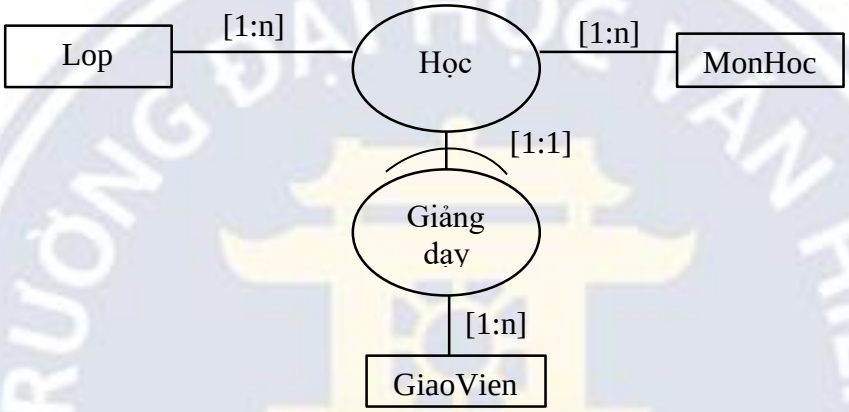
Các loại thực thể có thể có nhiều loại mối kết hợp với nhau.

Ví dụ: Ta có loại thực thể LOP, MONHOC, GIAOVIEN sẽ có các loại mối kết hợp như sau:

LOP sẽ học MONHOC

GIAOVIEN sẽ giảng dạy LOP

GIAOVIEN sẽ giảng dạy MONHOC



Hình 2.7. Sơ đồ mối kết hợp mở rộng có nhiều loại mối kết hợp với nhau

2.2.4. Lược đồ thực thể kết hợp

Mô hình lược đồ thực thể kết hợp được sử dụng để biểu diễn cơ sở dữ liệu ở mức khái niệm.

Mô hình thực thể kết hợp bao gồm có các thực thể, danh sách thuộc tính và những mối kết hợp.

Biểu diễn mô hình thực thể kết hợp dưới dạng sơ đồ thực thể liên kết (Entity Relationship Diagram: ERD).

Các thành phần gồm:

- Thực thể
- Thuộc tính
- Mối kết hợp

Các buóc xây dựng mô hình ERD:

Bước 1: Xác định các thực thể dựa vào các lớp đối tượng cần quản lý.

Căn cứ vào lớp đối tượng cần quản lý để xác định các thực thể. Khi xây dựng cần tuân theo qui tắc:

- Mỗi thực thể đều phải có khóa.

- Các thuộc tính của thực thể chỉ mô tả các đặt trưng của riêng thực thể đó không liên quan đến thực thể khác.

Ví dụ:

NhanVien
MaNV HoTenNV NgaySinh GioiTinh

DonVi
MaDV TenDV

Bước 2: Xác định các thực thể phụ thuộc (nếu có).

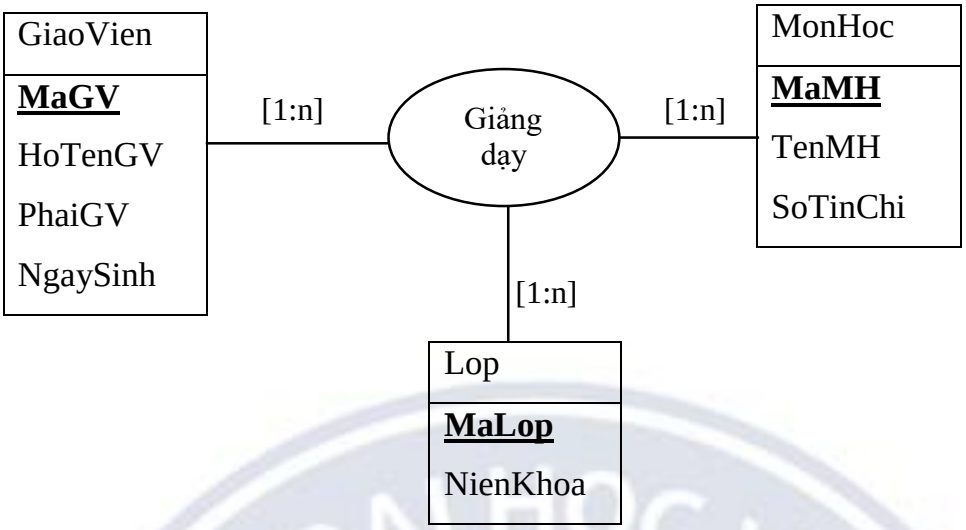
Căn cứ vào tập thuộc tính khóa xác định các thuộc tính khác của lớp đối tượng có chứa khóa của thực thể khác.

Ví dụ: Thực thể Bảng chấm công gồm các thông tin: mã số nhân viên, tháng, năm, số ngày làm việc trong tháng, số ngày nghỉ không phép…

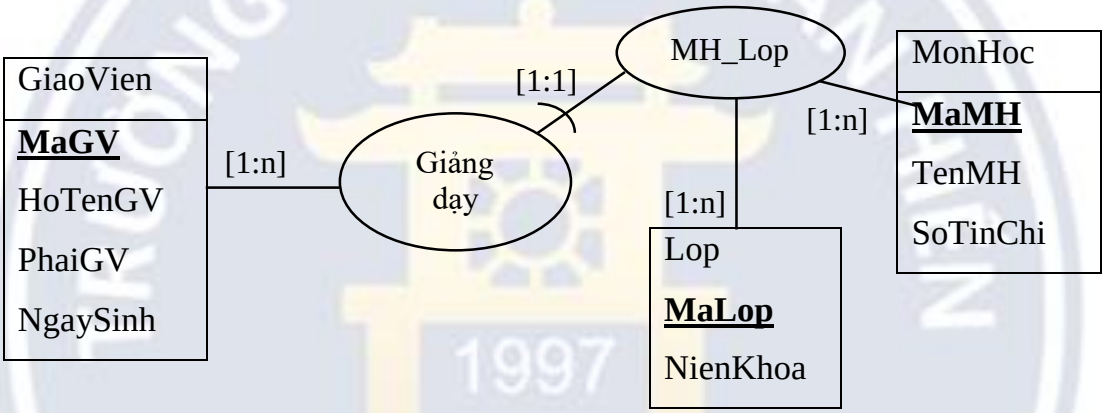
Bước 3: Xây dựng các mối kết hợp.

- Căn cứ vào mối quan hệ trong qui tắc quản lý của các đối tượng để xác định mối kết hợp, bản số và các thuộc tính của mối kết hợp.
- Căn cứ vào bản số của mối kết hợp để xác định khóa của mối kết hợp.
- Căn cứ vào khóa của mối kết hợp để kiểm tra mối kết hợp thỏa mãn qui tắc quản lý

Ví dụ 1: Mỗi môn học của một lớp có thể do nhiều giáo viên giảng dạy



Ví dụ 2: Mỗi môn học của một lớp chỉ do một giáo viên dạy.



Bước 4: Chuẩn hóa các thực thể để đạt dạng chuẩn cao nhất.

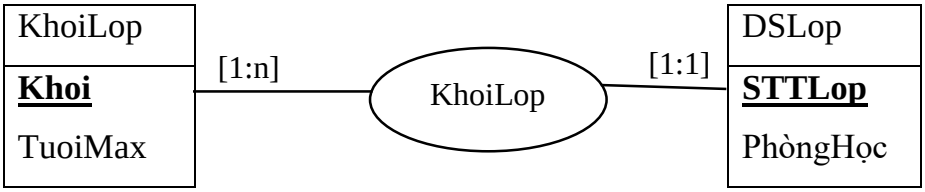
- **Dạng chuẩn 1:** Các thuộc tính phải là thuộc tính đơn.
- **Dạng chuẩn 2:** Các thuộc tính của thực thể phải phụ thuộc đầy đủ vào khóa của thực thể.

Do đó, nếu có thuộc tính phụ thuộc vào một tập con của khóa thực thể thì phải tách các thuộc tính đó vào một thực thể riêng, và thực thể đang xét trở thành thực thể phụ thuộc vào thức thể mới.

Ví dụ 1: Thực thể **danh sách lớp**

DSLop(Khối, STTLop, TuổiMax, PhòngHọc)

Thuộc tính TuổiMax dùng chỉ định tuổi tối đa của học sinh trong khối, do đó không phụ thuộc đầy đủ vào khóa.



- **Dạng chuẩn 3:** Các thuộc tính không khóa của thực thể không phụ thuộc bắc cầu vào khóa của thưc thể.

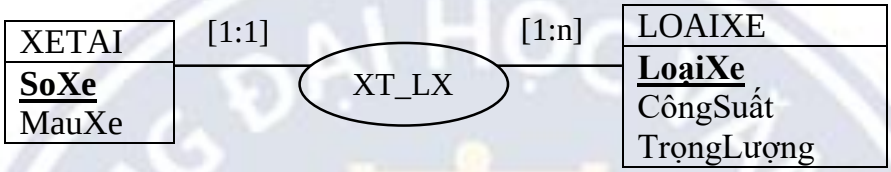
Do đó, nếu có thuộc tính phụ thuộc vào thuộc tính khác của thực thể thì phải tách các thuộc tính đó vào một thực thể riêng biệt.

Ví dụ 2: Thực thể **xe tải**

XeTai(**SốXe**, **LoạiXe**, Màu, **CôngSuất, TrọngLượng**)

Trong đó, CôngSuất và TrọngLượng phụ thuộc vào LoạiXe.

Do đó phải định nghĩa riêng.



2.2.5. Các loại lược đồ thực thể kết hợp

Mô hình dữ liệu là sự trừu tượng hóa môi trường thực của một cơ sở dữ liệu, thông qua các khái niệm mô tả dữ liệu và các phép toán xử lý dữ liệu. Thông qua mô hình dữ liệu, người thiết kế có thể mô tả toàn cảnh cơ sở dữ liệu được thiết kế, bao gồm:

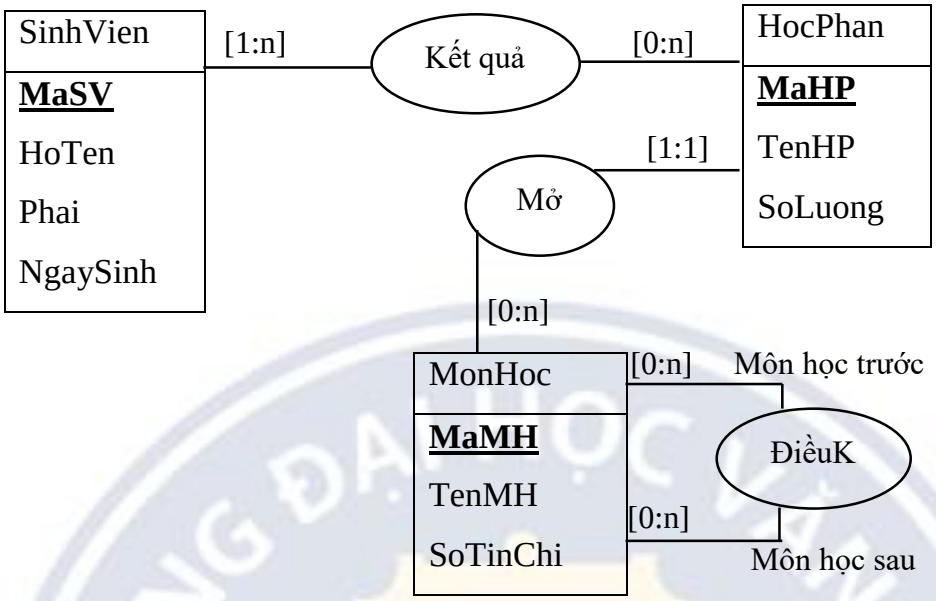
- Cấu trúc các đối tượng lưu trữ dữ liệu.
- Các mối quan hệ giữa các đối tượng.
- Các ràng buộc dữ liệu thể hiện các quy tắc quản lý ảnh hưởng đến các đối tượng.

Hiện có một số dạng mô hình dữ liệu: mô hình dữ liệu mạng, mô hình dữ liệu phân cấp, mô hình dữ liệu quan hệ, mô hình dữ liệu thực thể - kết hợp và mô hình dữ liệu hướng đối tượng.

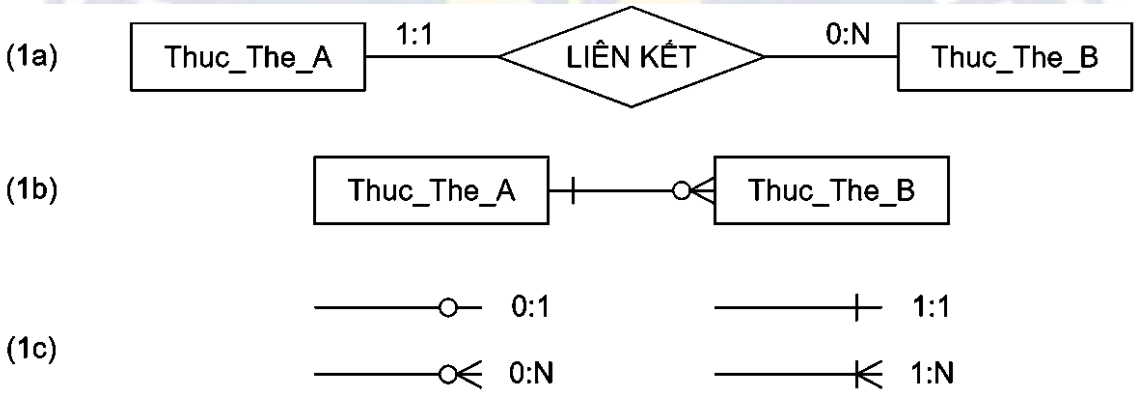
1. **Mô hình Thực thể và mối kết hợp (Entity - RelationShip Model – ER)**

- Tập Thực thể (Entity Set)
- Mối kết hợp (Relationship)

Ví dụ:



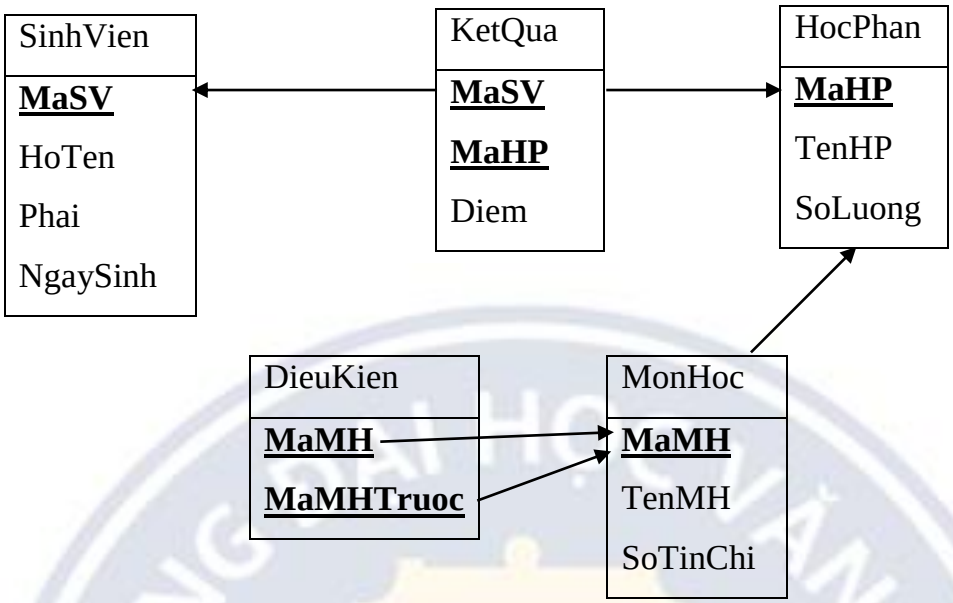
Hoặc theo dạng ký hiệu sau đây:



2. Mô hình Quan hệ (Relational Model)

- Quan hệ (Relation)

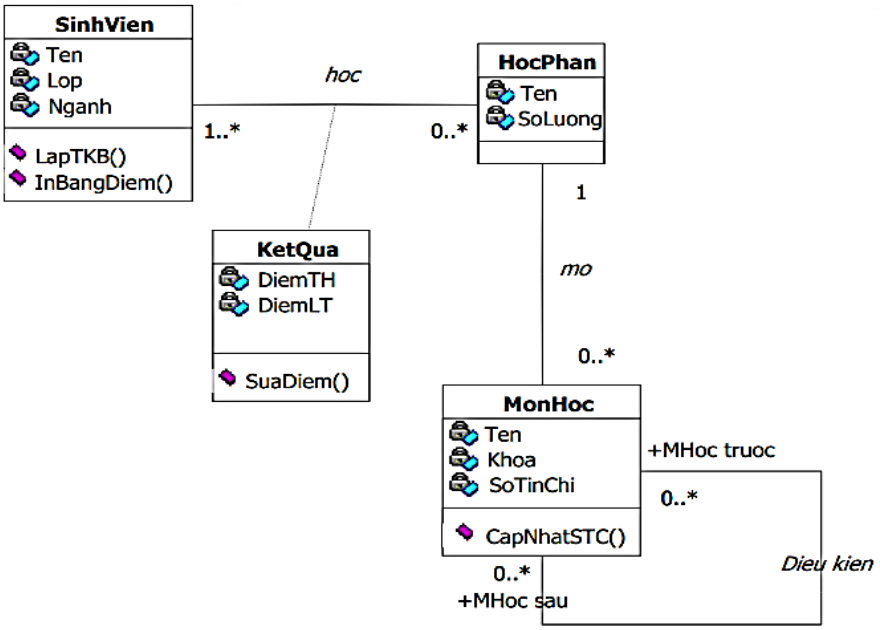
Ví dụ:



3. Mô hình hướng đối tượng (Object Oriented Model - OOM)

- Lớp (Class)
  + Thuộc tính
  + Phương thức
- Mối quan hệ
  + Kiểu kết hợp
  + Kiểu bộ phận-tổng thể
  + Kiểu kế thừa
  + Kiểu phụ thuộc

Ví dụ:



2.3. Các vấn đề về thiết kế ở mức khái niệm

1. Phương pháp thiết kế

Sau khi đã có một hiểu biết chi tiết về yêu cầu của cơ sở dữ liệu, bước đầu tiên là tìm cách mô tả lại một cách khái quát các dữ liệu ấy là gì, chúng móc nối với nhau ra sao. Để có thể nhận biết dễ dàng hơn, người ta sử dụng sơ đồ để trình bày điều này. Đối với cơ sở dữ liệu hoạt động theo mô hình quan hệ, lược đồ thực thể - liên kết được sử dụng.

Trong trường hợp đơn giản, số lượng thực thể và liên kết không nhiều, ta có thể vẽ lược đồ này bằng tay. Tuy nhiên, đối với những hệ thống phức tạp, ta cần đến sự hỗ trợ của những công cụ chuyên dùng. Để vẽ lược đồ này, hiện nay có khoảng mười phương pháp thể hiện các chi tiết. Ta nên lưu ý đến phương pháp thể hiện của phần mềm để chọn lựa công cụ vẽ thích hợp.

Ví dụ:

Ta sẽ thiết kế cơ sở dữ liệu cho một công ty bánh kẹo với một số nghiệp vụ đơn giản. Công ty mua nguyên liệu từ một số nhà cung ứng để sản xuất ra sản phẩm; từ đó bán ra thị trường thông qua các đại lý, siêu thị, cửa hàng. Ta cũng chỉ xem xét lĩnh vực sản xuất và phân phối.

Trong giai đoạn thiết kế ở mức khái niệm, ta cần xây dựng mô hình thực thể - liên kết cho cơ sở dữ liệu. Muốn vậy ta phải xác định các thực thể, các liên kết và xây dựng lược đồ thực thể - liên kết.

2. Xác định các thực thể của cơ sở dữ liệu:

Sau khi xem xét sơ bộ tình hình của công ty và yêu cầu của cơ sở dữ liệu, ta có thể chọn 4 thực thể sau:

Thực thể NGUYEN_LIEU dùng để lưu trữ các dữ liệu liên quan đến các loại nguyên liệu, bao bì và một số vật tư có liên quan dùng trong sản xuất. Ta cũng gộp trong nhóm này các tiện ích dùng trong sản xuất như điện, nước.

Thực thể NHA_CUNG_UNG dùng để lưu trữ các dữ liệu về các nhà cung ứng các loại nguyên liệu, vật tư, tiện ích vừa kể trên.

Thực thể SAN_PHAM dùng để lưu trữ các dữ liệu liên quan đến các sản phẩm chỉnh, các phụ phẩm do công ty sản xuất.

Thực thể KHACH_HANG dùng để lưu trữ các dữ liệu về những người, những công ty, những cơ quan mua hàng của công ty hay phân phối hàng cho công ty.

3. **Những liên kết giữa các thực thể:**

Với những thực thể đã chọn bên trên, ta thấy có những liên kết sau giữa chúng:

Giữa hai thực thể NHA_CUNG_UNG và NGUYEN_LIEU, ta thấy có mối liên kết CUNG CẤP. Vì mỗi nhà cung ứng có thể cung cấp nhiều loại nguyên liệu và mỗi loại nguyên liệu có thể được cung ứng bởi một số nhà cung ứng khác nhau nên liên kiết giữa NHA_CUNG_UNG và NGUYEN_LIEU là liên kết nhiều-nhiều. Mặt khác mỗi nguyên liệu phải có ít nhất một nhà cung ứng nên bản số phía NHA_CUNG_UNG là [1:n]. Có thể có nhà cung ứng không cung cấp nguyên liệu nào nên bản số ở phía NGUYEN_LIEU là [0:n].

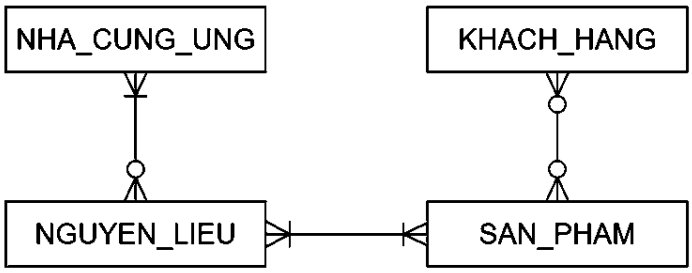
Giữa hai thực thể NGUYEN_LIEU và SAN_PHAM có liên kết SẢN XUẤT. Vì mỗi loại nguyên liệu có thể dùng để sản xuất nhiều loại sản phẩm và mỗi loại sản phẩm cần dùng một số loại nguyên liệu nên liên kết giữa NGUYEN_LIEU và SAN_PHAM là liên kết [n:n]. Ngoài ra mỗi loại nguyên liệu đều phải được dùng để sản xuất ít nhất là một sản phẩm, và mỗi sản phẩm cần sử dụng ít nhất là một loại nguyên liệu. Vì thế bản số ở cả hai phía đều là [1:n].

Giữa hai thực thể KHACH_HANG và SAN_PHAM có liên kết MUA. Vì mỗi khách hàng có thể mua nhiều loại sản phẩm và mỗi sản phẩm có thể được mua bởi nhiều khách hàng nên liên kết giữa KHACH_HANG và SAN_PHAM là liên kết [n:n]. Do có những khách hàng không mua sản phẩm nào, và cũng có những sản phẩm không được khách hàng nào đặt mua nên bản số ở cả hai phía đều là [0:n].

4. **Lược đồ của mô hình thực thể - liên kết:**

Từ những luận điểm được trình bày bên trên, ta có thể xây dựng được lược đồ của cơ sở dữ liệu theo mô hình thực thể - liên kết ở dạng rút gọn như sau.

NHA_CUNG_UNGNGUYEN_LIEUKHACH_HANGSAN_PHAM



BÁI TẬP CHƯƠNG 2

1. Hãy nói về vai trò của mô hình dữ liệu bậc cao trong quá trình thiết kế cơ sở dữ liệu.

2. Liệt kê các trường hợp cần phải sử dụng giá trị null.
3. Định nghĩa các thuật ngữ sau: thực thể, thuộc tính, giá trị thuộc tính, thể hiện liên kết, thuộc tính phức hợp, thuộc tính đa trị, thuộc tính suy diễn được, thuộc tính phức tạp, thuộc tính khoá, miền giá trị.
4. Kiểu thực thể là gì? Tập thực thể là gì? Giải thích sự khác nhau giữa một thực thể, một kiểu thực thể và một tập thực thể.
5. Giải thích sự khác nhau giữa một thuộc tính và một tập giá trị.
6. Kiểu liên kết là gì? Giải thích sự khác nhau giữa một thể hiện liên kết, một tập liên kết và một kiểu liên kết.
7. Vai trò tham gia là gì? Khi nào cần phải sử dụng các tên vai trò trong mô tả các kiểu liên kết.
8. Mô tả cách chỉ ra các ràng buộc cấu trúc trên các kiểu liên kết.
9. Với điều kiện nào một thuộc tính của một kiểu liên kết cấp 2 có thể chuyển thành một thuộc tính của một trong các kiểu thực thể tham gia vào kiểu liên kết.
10. Khi chúng ta nghĩ đến các liên kết như là các thuộc tính, các tập giá trị của các thuộc tính đó là gì?
11. Kiểu liên kết đệ quy là gì? Cho một số ví dụ về các kiểu liên kết đệ quy.
12. Khi nào khái niệm kiểu thực thể yếu được dùng trong mô hìn hoá cơ sở dữ liệu? Định nghĩa các thuật ngữ: kiểu thực thể chủ, kiểu thực thể yếu, khoá bộ phận, kiểu liên kết xác định.
13. Trình bày các khái niệm lớp, lớp con, chuyên biệt hoá, tổng quát hoá. Trong hoàn cảnh nào ta cần tách một lớp thành các lớp con.
14. Trình bày cách biểu diễn đồ hoạ của các mô hình ER và EER.
15. Xây dựng mô hình ER cho cơ sở dữ liệu TRƯỜNG Hãy xây dựng lược đồ ER cho CSDL "TRƯỜNG", dựa trên các ghi chép sau:
    a. Trường được chia thành các trường con: Trường KHTN, Trường KHXH, Trường Công nghệ, Trường VanHien, …. Mỗi trường có một hiệu trưởng quản lý. Mỗi hiệu trưởng quản lý một trường.

b. Mỗi trường có nhiều khoa. Chẳng hạn, trường KHTN có các khoa Toán, Lý, Hóa,… Mỗi một khoa chỉ thuộc về một trường. Thông tin về Khoa gồm Mã khoa, tên khoa, địa chỉ, số điện thoại, tên trường.

c. Mỗi Khoa cung cấp nhiều môn học. Mỗi môn học gồm có Tên môn học, mã số, số đơn vị học trình, trình độ, tên Khoa.

d. Mỗi môn học có thể có nhiều học phần.Mỗi học phần được lưu giữ bằng các thông tin: Mã học phần, Tên môn học, Tên giáo viên dạy, học kỳ.

e. Mỗi khoa có nhiều giáo viên làm việc, nhưng mỗi giáo viên chỉ làm việc cho một khoa. Mỗi một khoa có một chủ nhiệm khoa, đó là một giáo viên.

f. Mỗi giáo viên có thể dạy nhiều nhất là 4 học phần và cũng có thể không dạy học phần nào.

g. Mỗi sinh viên phải học nhiều học phần.

h. Mỗi một khoa có nhiều sinh viên, mỗi sinh viên chỉ thuộc về một khoa.

i. Thông tin về mỗi sinh viên gồm: Mã sinh viên, Họ tên, địa chỉ, ngày sinh, giới tính, Lớp, Tên Khoa và chế độ đào tạo.

j. Mỗi sinh viên có một người giám sát (giáo viên chủ nhiệm), người đó là một giáo viên.

k. Sau mỗi học kỳ sẽ có một danh sách điểm để phân loại. Nó gồm các thông tin: Mã sinh viên, mã học phần, điểm bằng chữ, điểm bằng số.

16. Xây dựng mô hình ER cho cơ sở dữ liệu THƯ VIỆN.Hãy xây dựng lược đồ ER cho CSDL "THƯ VIỆN", dựa trên các ghi chép sau:

a. Thư viện được chia ra thành các nhánh. Thông tin về mỗi nhánh gồm có Mã nhánh, Tên nhánh và Địa chỉ.

b. Mỗi cuốn sách trong thư viện có các thông tin về Mã sách, Tên sách Nhà xuất bản và Tác giả…

c. Một tác giả có thể viết nhiều cuốn sách. Một cuốn sách có thể có nhiều tác giả viết.

d. Một nhà xuất bản xuất bản nhiều cuốn sách. Một cuốn sách do một nhà xuất bản xuất bản. Thông tin về Nhà xuất bản gồm có Tên, Địachỉ và Số điện thoại.

e. Một cuốn sách có thể có nhiều bản sao được lưu trữ tại các nhánh. Thông tin về bản sao sách gồm Mã sách, số các bản sao.

f. Thư viện có những người mượn sách. Thông tin về những người mượn sách gồm có Số thẻ, Họ tên, Địa chỉ và Số điện thoại.

g. Sách được cho các người mượn mượn tại các nhánh. Thông tin về một lần mượn gồm có Ngày mượn và ngày trả.

TRƯỜNG ĐẠI HỌC VĂN HIẾN
1997
VAN HIEN UNIVERSITY

Chương 3
MÔ HÌNH DỮ LIỆU QUAN HỆ

3.1. Giới thiệu

Mô hình Dữ liệu Quan hệ (Relational Data Model – RDM) lần đầu tiên được Ted Codd của IBM phát triển vào những năm 1970. Sau đó khoảng 10 năm, RDM chính thức được đưa vào triển khai thương mại nhằm mục đích lưu trữ và xử lý dữ liệu trong cơ sở dữ liệu. Sở dĩ RDM trở nên phổ biến như vậy chính bởi tính đơn giản trong sử dụng cơ sở dữ liệu, cũng như nền tảng hỗ trợ tốt cho các nhà phát triển.

Mô hình dữ liệu quan hệ biểu diễn cơ sở dữ liệu dưới dạng một tập hợp các quan hệ (bảng giá trị). Mỗi bảng giá trị có các cột và hàng được gọi lần lượt là thuộc tính (attributes) và bộ giá trị (tuples). Mỗi bộ giá trị (tuple) kí hiệu một thực thể hoặc mối quan hệ trong thế giới thực. Tên của quan hệ và tên của các thuộc tính sẽ góp phần giải thích ý nghĩa của từng bộ.

- Một hệ thống các ký hiệu để mô tả dữ liệu dưới dạng dòng và cột như quan hệ, bộ, thuộc tính, khóa chính, khoá ngoại, ...
- Một tập hợp các phép toán thao tác trên dữ liệu như phép toán tập hợp, phép toán quan hệ.
- Các ràng buộc toàn vẹn quan hệ.

3.2. Các khái niệm của mô hình quan hệ

3.2.1. Quan hệ (Relation)

Quan hệ (Relation): Là một bảng giá trị gồm hai thành phần: Lược đồ quan hệ (relation schema) và thể hiện của quan hệ (relation instance).

Lược đồ quan hệ (relation schema): Xác định tên của quan hệ, tên và kiểu dữ liệu của thuộc tính của quan hệ.

Ký hiệu: **R(A_1, A_2, …, A_n)**.

Trong đó:

A_1, A_2, …, A_n: Danh sách các thuộc tính.

R: Tên của quan hệ

Ví dụ:

NhanVien(**<u>MaNV</u>**, HoTenNV, Phai, NamSinh, DiaChi)

Thể hiện của quan hệ (relation instance): Là một bảng giá trị gồm các dòng và các cột.

Ví dụ: Thực thể **NhanVien**

Các thuộc tính của thực thể NhanVien

MaNV	HoTenNV	Phai	NamSinh	DiaChi
001	Trần Quang Bình	Nam	2005	33 Lê Lai, Q. Bình Thạnh
002	Lê Thị Nga	Nữ	2005	12 Lê Lợi, Q. 1
003	Lương Thái Hiền	Nam	2005	624 Âu Cơ. Q. Tân Phú

Các giá trị khóa

Các dòng (Record)

Hình 3.1. Lược đồ quan hệ

3.2.2. Schema

Vì các lược đồ tạo thành một tính năng cấu trúc cơ bản của cơ sở dữ liệu, hầu hết các môi trường cơ sở dữ liệu đều áp dụng quyền truy cập cho các đối tượng ở mức lược đồ.

Ví dụ: Cơ sở dữ liệu của công ty có thể chứa một loạt người dùng. Mỗi người dùng phải chịu một lược đồ, nhưng quyền truy cập vào các lược đồ khác nhau được cấp riêng và với mức độ chi tiết của các quyền đối với người dùng bên ngoài lược đồ máy chủ.

Hầu hết các công cụ quản lý cơ sở dữ liệu không liệt kê các lược đồ; thay vào đó, họ liệt kê cơ sở dữ liệu và người dùng.

Cách tạo Database Schema:

Một lược đồ được định nghĩa chính thức bằng cách sử dụng hệ quản trị cơ sở dữ liệu Oracle, SQL, MySQL,… bạn tạo một lược đồ bằng cách tạo tài khoản người dùng sở hữu nó.

Những người dùng khác được cấp quyền truy cập vào các lược đồ mới nhờ tên người dùng hoặc bởi một hoặc nhiều vai trò mà tài khoản người dùng đã được thêm vào.

Phân loại Database Schema:

- Lược đồ cơ sở dữ liệu logic: Lược đồ này xác định tất cả các ràng buộc logic cần được áp dụng trên thông tin được lưu trữ. Nó định nghĩa các bảng, khung nhìn và các ràng buộc toàn vẹn.

- Lược đồ cơ sở dữ liệu vật lý: Lược đồ này liên quan đến việc lưu trữ thông tin cụ thể và đó là loại lưu trữ như tệp, chỉ mục, v.v. Nó xác định cách thông tin sẽ được lưu trữ trong một bộ lưu trữ phụ trợ quá mức.
- Lược đồ dự kiến: Lược đồ dự kiến được phác thảo do kiểu của cơ sở dữ liệu ở mức xem thường mô tả tương tác của người dùng với các hệ thống cơ sở dữ liệu.

3.2.3. Thuộc tính

Thuộc tính là các đặc điểm riêng của một đối tượng (đối tượng được hiểu như là một loại thực thể ở mô hình thực thể kết hợp), mỗi thuộc tính có một tên gọi và phải thuộc về một kiểu dữ liệu nhất định (số, chuỗi, ngày tháng, logic, hình ảnh,…). Kiểu dữ liệu ở đây là kiểu đơn.

Trong cùng một đối tượng không được có hai thuộc tính cùng tên.

3.2.4. Lược đồ

- Lược đồ quan hệ bao gồm:
  + Tên quan hệ.
  + Tên các thuộc tính cùng với tên domain tương ứng.
  + Các ràng buộc bảo toàn (integrity constraint): là các hạn chế trên các quan hệ của lược đồ này.
- Domain : là một tập hợp các giá trị cho phép của một hay nhiều thuộc tính, thường là một tập con của kiểu dữ liệu.

Ký hiệu: dom(A) → miền trị của thuộc tính A

Ví dụ:

- Thuộc tính Address của quan hệ STUDENT có domain là tập hợp các chuỗi.
- Thuộc tính điểm có domain là các số thực trong phạm vi từ 0 đến 10

3.2.5. Bộ

Bộ dữ liệu: là các dòng của quan hệ không kể dòng tên của các thuộc tính, mỗi bộ chứa các giá trị cụ thể của các thuộc tính.

Ví dụ: Thực thể **NhanVien**

Các bộ dữ liệu

MaNV	HoTenNV	Phai	NamSinh	DiaChi
001	Trần Quang Bình	Nam	2005	33 Lê Lai, Q. Bình Thạnh
002	Lê Thị Nga	Nữ	2005	12 Lê Lợi, Q. 1
003	Lương Thái Hiền	Nam	2005	624 Âu Cơ. Q. Tân Phú

Hình 3.2. Các bộ dữ liệu trong mô hình quan hệ

3.2.6. Miền giá trị

Miền giá trị là tập hợp các giá trị nguyên tử gắn với thuộc tính, có tên, kiểu dữ liệu, khuôn dạng và mô tả.

Ví dụ:

- Tên: SDT_Nhanvien.
- Kiểu dữ liệu: chuỗi ký tự.
- Khuôn dạng: xxx-xxxxxxx.
- Mô tả: miền giá trị của số điện thoại của nhân viên.

3.3. Các ràng buộc của mô hình quan hệ

Ràng buộc là những quy tắc được áp đặt lên trên dữ liệu đảm bảo tính tin cậy và độ chính xác của dữ liệu. Các luật toàn vẹn được thiết kế để giữ cho dữ liệu phù hợp và đúng đắn. Các kiểu ràng buộc chính:

Ràng buộc miền giá trị (Domain Constraints):

Bảo đảm tính hợp lệ của dữ liệu trongmột thuộc tính. Việc xác định miền giá trị của các thuộc tính bao gồm một số các yêu cầu sau: Tên thuộc tính, Kiểu dữ liệu, Độ dài dữ liệu, khuôn dạng của dữ liệu, các giá trị giới hạn cho phép, ý nghĩa, có duy nhất hay không, có cho phép giá trị rỗng hay không. Tất cả giá trị xuất hiện trong một thuộc tính của một quan hệ phải cùng một miền trị.

Ví dụ: Trong quan hệ KQUATHI, DiemThi là một số nguyên nằm trong khoảng từ 0 đến 10.

Ràng buộc khoá (Key Constraints):

- Siêu khoá (superkey) là một thuộc tính hoặc một tập các thuộc tính dùng để xácđịnh duy nhất một bộ của quan hệ. Một siêu khoá có thể chứa thêm các thuộc tính không cần thiết để xác định duy nhất một bộ. Siêu khoá mặc định là tập tấtcả các thuộc tính của quan hệ.
- Khoá dự tuyển (candidate key): là một siêu khoá K mà không có một tập

con thực sự bất kỳ K’ ⊂ K lại là một siêu khoá.

- Khóa dự tuyển cần thỏa mãn hai tính chất sau:
  + Xác định duy nhất: các giá trị của K trong các bộ của r là duy nhất.
  + Không dư thừa: Khi xóa đi bất kỳ một thuộc tính nào của khóa đều pháhủy tính xác định duy nhất của khóa.
- Khóa chính (primary key): là một khóa dự tuyển được chọn để xác định duy nhất một bộ của quan hệ. Khóa chính có thể chỉ có một thuộc tính hay khóa phức hợp.

Định nghĩa khóa theo quan hệ toán học:

Khóa (key) của R(U) với U={A1, A2,. . ,Am} là một tập con K= {Aj1, Aj2,…, Ajn} với j1, j2,..jn là các số nguyên phân biệt nằm trong khoảng từ 1 đến m, phải thỏa mãn đồng thời hai điều kiện sau:

- $\forall r(R), \forall t_1, t_2 \in r$, nếu $t_1 \neq t_2$ thì $t[K_1] \neq t[K_2]$ (1)
- Không tồn tại K’ ⊂ K sao cho K’ thỏa mãn điều kiện (1)

Ràng buộc thực thể (Entity Integrity Constraints):

Bảo toàn thực thể dùng để bảo đảm tính duy nhất của mỗi hàng trong bảng. Để bảo toànthực thể thì mỗi quan hệ có 1 khóa chính và giá trị của khóa chính phải luôn hợp lệ. Trong một quan hệ cơ sở, mọi thuộc tính khóa chính không được có giá trị rỗng (null).

Ràng buộc toàn vẹn tham chiếu (Referential Integrity Constraints):

Toàn vẹn tham chiếu dùng để xác định mối quan hệ giữa các bảng khi thêm, sửa hay xóa các hàng trong bảng. Ràng buộc toàn vẹn tham chiếu nhằm duy trì tính nhất quán (consistency) giữa các bộ của 2 quan hệ.

Một bộ trong quan hệ R tham chiếu đến một bộ trong quan hệ S thì bộ trong S phải tồntại trước. Ràng buộc toàn vẹn tham chiếu xác định giữa 2 quan hệ.

PhongBan	**MaPB**	TenPB
	05	Nghiên cứu

NhanVien	**MaNV**	Ho	Ten	NgaySinh	GioiTinh	MaPB
	001	Trần Văn	Hùng	01/02/2005	Nam	05

Hình 3.3. Mô hình ràng buộc toàn vẹn tham chiếu

3.4. Các đặc trưng của mô hình quan hệ

- Thứ tự các bộ trong quan hệ là không quan trọng.
- Thứ tự giữa các giá trị trong một bộ là quan trọng.
- Mỗi giá trị trong một bộ: hoặc là một giá trị nguyên tố, hoặc là một giá trị rỗng (null).
- Không có bộ nào trùng nhau.

3.5. Chuyển lược đồ cơ sở dữ liệu theo mô hình thực thể kết hợp sang mô hình quan hệ

3.1.1. Qui tắc chung

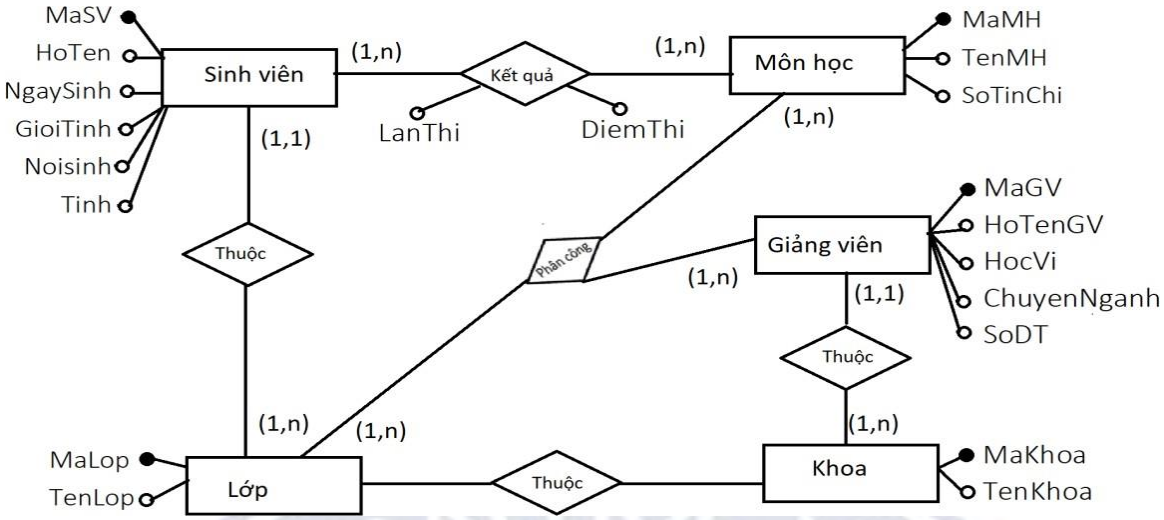
Khi biến đổi mô hình ER thành các mô hình quan hệ ta áp dụng các qui tắc sau:

- Mỗi tập thực thể trong mô hình ER được chuyển thành một lược đồ quan hệ.
- Mỗi thuộc tính trong mô hình ER được chuyển thành thuộc tính trong lược đồ quan hệ tương ứng
- Mỗi thuộc tính nhận diện trong mô hình ER được chuyển thành khóa chính trong lượcđồ quan hệ tương ứng.
- Mỗi mối quan hệ trong ER được chuyển thành khóa ngoại theo qui tắc sau

3.1.2. Qui tắc chuyển mối quan hệ thành khóa ngoại

- **Mối quan hệ một-một:** chuyển khóa chính từ quan hệ 1 sang quan hệ 2 hay ngược lại
- **Mối quan hệ một-nhiều**: chuyển khóa chính từ bên một sang bên nhiều.
- **Mối quan hệ nhiều-nhiều:** tạo một quan hệ mới có khóa chính là sự kết hợp các khóachính của hai quan hệ có tính kết nối nhiều nhiều.
- **Thực thể yếu**: chuyển thành một quan hệ có cùng tên với thực thể yếu và thêm vàothuộc tính khóa của quan hệ liên quan

Ví dụ:



Hình 3.4. Mô hình ERD quản lý Sinh viên

Được chuyển sang mô hình quan hệ gồm các quan hệ sau:

Sinhvien(**MASV,** HOTENSV, GIOITINH, NGAYSINH, NOISINH, TINH, MALOP)

Lop(**MALOP**, TENLOP, MAKHOA)

Khoa(**MAKHOA**, TENKHOA)

Monhoc(**MAMH**, TENMH, SOTINCHI)

Giangvien(**MAGV**, HOTENGV, HOCVI, HUYENNGANH, SODT, MAKHOA,)

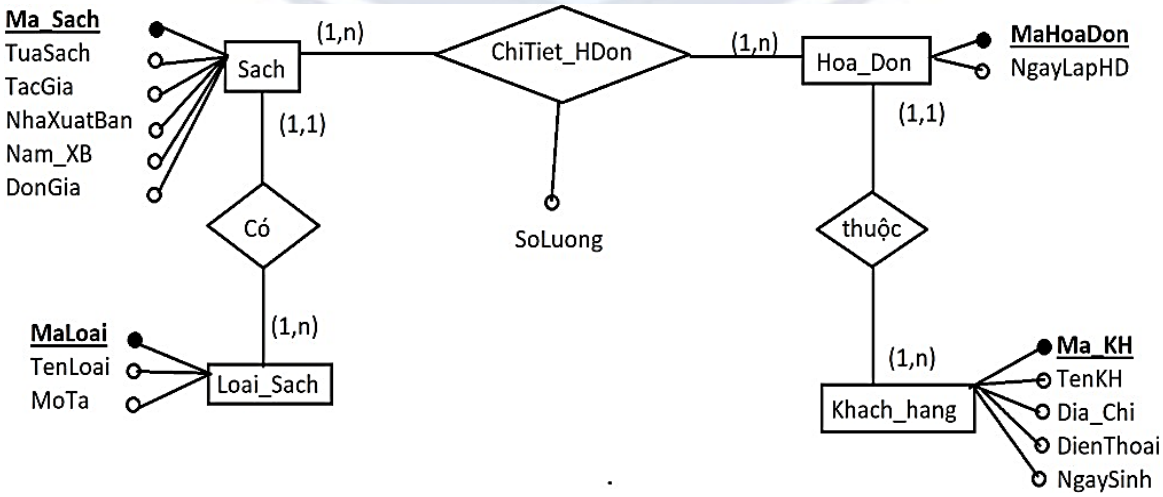
Ketqua(**MASV, MAMH**, LANTHI, DIEMTHI)

PhanCong(**MALOP, MAMH, MAGV**)

BÀI TẬP CHƯƠNG 3

1. Hãy chuyển đổi các lược đồ ERD ở chương hai sang mô hình quan hệ.

2. Chuyển đổi lược đồ ERD sau sang mô hình quan hệ



Chương 4

ĐẠI SỐ QUAN HỆ VÀ PHÉP TÍNH QUAN HỆ

3.1. Giới thiệu

- Đại số quan hệ là **tập hợp các phép toán cơ sở** của mô hình dữ liệu quan hệ.
- Biểu thức đại số quan hệ là một chuỗi các phép toán.
- Kết quả của một biểu thức là một thể hiện quan hệ.
- Ý nghĩa: Đại số quan hệ là cơ sở hình thức cho các phép toán của mô hình quan hệ; cơ sở để cài đặt và tối ưu hóa các truy vấn trong các HQT CSDL quan hệ và được áp dụng trong SQL.
- Trong đại số quan hệ, toán hạng là các thể hiện quan hệ, các tập hợp.
- Toán tử là các phép toán:
  + Phép toán tập hợp: hội, giao, hiệu, tích Cartesian.
  + Phép toán quan hệ: phép chọn, chiếu, kết, chia, đổi tên và một số phép toán khác.

3.2. Phép toán một ngôi

Các phép toán chỉ tác động lên một quan hệ: phép chọn (Select), phép chiếu (Project), Phép đổi tên (Rename).

4.2.1. Phép chọn

Phép chọn được sử dụng để rút trích các bộ dữ liệu thỏa điều kiện chọn từ một quan hệ.

Cú pháp: $\sigma_{<ĐK>}(R)$

Trong đó <ĐK> là biểu thức điều kiện.

Ví dụ:

R	A	B	C	D
	α	α	1	7
	α	β	5	7
	β	β	12	3
	β	β	23	10

$$\xrightarrow{\sigma_{A=B \wedge D>5}(R)}$$

A	B	C	D
α	α	1	7
β	β	23	10

- Biểu thức điều kiện:
  + Chứa các mệnh đề có dạng
    - <thuộc tính> <toán tử so sánh> <hằng số>.

- <thuộc tính> <toán tử so sánh> <thuộc tính>.

+ Toán tử so sánh: =, <, ≤, >, ≥, ≠.

+ Các mệnh đề được nối bởi toán tử logic: ∧, ∨, ¬.

- Đặc trưng của phép chọn:

+ Phép chọn có tính giao hoán.

- $\sigma_{<ĐK1>}(\sigma_{<ĐK2>}(R)) = \sigma_{<ĐK2>}(\sigma_{<ĐK1>}(R))$.

+ Kết quả của phép chọn là một quan hệ có cùng bậc với R và có số bộ ít hơn hoặc bằng số bộ của R.

4.2.2. *Phép chiếu*

- Để rút trích các cột ứng với các thuộc tính nào đó của một quan hệ.

R	A	B	C	D
	α	α	1	7
	α	β	5	7
	β	β	12	3
	β	β	23	10

$\xrightarrow{\pi_{A,D}(R)}$

A	D
α	7
α	7
β	3
β	10

- **Cú pháp:**

  - π<DSTT>(R).
  - <DSTT> là danh sách các thuộc tính của R.

- Đặc trưng của phép chiếu:

+ Phép chiếu không có tính giao hoán.

+ $\pi_{<DSTT1>}(\pi_{<DSTT2>}(R)) \neq \pi_{<DSTT2>}(\pi_{<DSTT1>}(R))$.

+ Phép chiếu loại bỏ các bộ trùng nhau.

+ Kết quả là một quan hệ

+ Có bậc bằng số thuộc tính của danh sách thuộc tính.

+ Có bậc nhỏ hơn hoặc bằng bậc của R.

+ Có số bộ ít hơn hoặc bằng số bộ của R.

- Mở rộng phép chiếu

+ Cho phép sử dụng các phép toán số học trong danh sách thuộc tính.

+ $\pi_{A,2*C}(R)$.

4.2.3. *Phép đổi tên*

- Để đổi tên quan hệ và các thuộc tính.

- **Cú pháp:** Cho quan hệ $R(A_1, ..., A_n)$

+ Đổi tên quan hệ R thành S: $\rho_S(R)$
+ Đổi tên quan hệ R thành S và các thuộc tính A_i thành B_i: $\rho_S(B_1, B_2, ..., B_n)(R)$
+ Đổi tên các thuộc tính A_i thành B_i: $\rho(B_1, B_2, ..., B_n)(R)$
+ Đổi tên quan hệ R thành S và thuộc tính A_1 thành B_1: $\rho_S(B_1, A_2, A_3, ..., A_n)(R)$
+ Đổi tên thuộc tính A_1 thành B_1: $\rho(B_1, A_2, A_3, ..., A_n)(R)$

3.3. Phép toán hai ngôi

- Là các phép toán tác động lên hai quan hệ và chỉ được sử dụng khi hai quan hệ được tác động là *khả hợp*.
- Hai quan hệ $R(A_1, ..., A_n)$ và $S(B_1, ..., B_n)$ gọi là khả hợp nếu: bậc R = bậc S và miền xác định A_i ≡ Miền xác định B_i, với i = 1, ..., n.
- Các phép toán hai ngôi gồm 2 loại: *phép toán tập hợp* gồm phép hội (Union), phép giao(Intersection), phép hiệu (Mimus) và phép tích Cartesian; *phép toán phi tập hợp* gồm phép kết (Join), phép chia (Division).

4.3.1. Phép toán tập hợp

a. Phép hội

Cho 2 quan hệ R và S khả hợp, phép hội của R và S ký hiệu là $R \cup S$, kết quả của phép hộilà một quan hệ gồm các bộ thuộc R hoặc thuộc S, hoặc cả hai (các bộ trùng lắp sẽ bị bỏ).

$$R \cup S = \{ t / t \in R \vee t \in S \}$$

Ví dụ:

R	A	B
	α	1
	α	2
	β	1

S	A	B
	α	2
	β	3

R ∪ S	A	B
	α	1
	α	2
	β	1
	α	2
	β	3

b. Phép giao

Cho 2 quan hệ R và S khả hợp, phép giao của R và S được ký hiệu là $R \cap S$, kết quả củaphép giao là một quan hệ gồm các bộ thuộc R đồng thời thuộc S.

$$R \cap S = \{ t / t \in R \wedge t \in S \}$$

Ví dụ:

R	A	B
	α	1
	α	2
	β	1

S	A	B
	α	2
	β	3

R ∩ S	A	B
	α	2

c. Phép trừ

Cho 2 quan hệ R và S khả hợp, phép giao của R và S được ký hiệu là R – S, kết quả củaphép trừ là một quan hệ gồm các bộ thuộc R và không thuộc S.

$$R - S = \{ t / t \in R \wedge t \notin S \}$$

Ví dụ:

R	A	B
	α	1
	α	2
	β	1

S	A	B
	α	2
	β	3

R – S	A	B
	α	1
	β	1

d. Các tính chất khác

- **Tính giao hoán**

  $R \cup S = S \cup R R \cap S = S \cap R$

- **Tính kết hợp**

  $R \cup (S \cup T) = (R \cup S) \cup T$

  $R \cap (S \cap T) = (R \cap S) \cap T$

e. Phép tích Descartes

Phép tích **Descartes (hay tích Đề Các)** được dùng để kết hợp các bộ của các quan hệ lại với nhau được ký hiệu là $R \times S$; có kết quả trả về là một quan hệ Q và mỗi bộ của Q là tổ hợp giữa 1 bộ trong R và 1 bộ trong S; nếu R có u bộ và S có v bộ thì Q sẽ có u × v bộ, nếu R có n thuộc tính và Q có m thuộc tính thì Q sẽ có n + m thuộc tính ($R^+ \cap Q^+ \neq \varnothing$).

Ví dụ:

R	A	B
	α	1
	β	2

S	X	C	D
	α	10	+
	β	10	+
	β	20	-
	γ	10	-

$R \times S$

A	B	X	C	D
α	1	α	10	+
α	1	β	10	+
α	1	β	20	-
α	1	γ	10	-
β	2	α	10	+
β	2	β	10	+
β	2	β	20	-
β	2	γ	10	-

$\rho_{(X,C,D)}(S)$

4.3.2. Phép kết

Phép kết được dùng để tổ hợp hai bộ có liên quan từ hai quan hệ thành một bộ.

Ký hiệu: $R \bowtie S$

Với $R(A_1, A_2, \ldots, A_n)$ và $S(B_1, B_2, \ldots, B_m)$.

Kết quả của phép kết là một quan hệ Q Có n + m thuộc tính.

$Q(A_1, A_2, \ldots, A_n, B_1, B_2, \ldots, B_m)$ và mỗi bộ của Q là tổ hợp của 2 bộ trong R và S, thỏa mãn một số điều kiện kết nào đó và có dạng $A_i\ \theta\ B_j$; trong đó, A_i là thuộc tính của R và B_j là thuộc tính của S; A_i và B_j có cùng miền giá trị và θ là phép so sánh ≠, =, <, >, ≤, ≥.

Phép kết gồm có 3 loại:

- Kết Theta (Theta join) là phép kết có điều kiện, ký hiệu $R \bowtie_c S$ trong đó **c** là điều kiện kết trên thuộc tính.
- Kết bằng (Equi join) khi **c** là điều kiện so sánh bằng.
- Kết tự nhiên (Natural join).

  Ký hiệu: $R \bowtie S$ hay $R * S$

  với $R^+ \cap Q^+ \neq \varnothing$**;** kết quả của phép kết bằng bỏ bớt đi một cột giống nhau.

Ví dụ:

- **Phép kết Theta**

$R \bowtie_{B<D} S$

R	A	B	C
	1	2	3
	4	5	6
	7	8	9

S	D	E
	3	1
	6	2

A	B	C	D	E
1	2	3	3	1
1	2	3	6	2
4	5	6	6	2

$$R \bowtie_C S = \sigma_C(R \times S)$$

- **Phép kết bằng**

$R \bowtie_{C=D} S$

R	A	B	C
	1	2	3
	4	5	6
	7	8	9

S	D	E
	3	1
	6	2

A	B	C	D	E
1	2	3	3	1
4	5	6	6	2

$R \bowtie_{C=S.C} S$

R	A	B	C
	1	2	3
	4	5	6
	7	8	9

S	S.C	D
	3	1
	6	2

A	B	C	S.C	D
1	2	3	3	1
4	5	6	6	2

$\rho_{(S.C,D)}\ S$

- **Phép kết tự nhiên**

$R \bowtie S$

R	A	B	C
	1	2	3
	4	5	6
	7	8	9

S	C	D
	3	1
	6	2

A	B	C	~~S.C~~	D
1	2	3	~~3~~	1
4	5	6	~~6~~	2

A	B	C	D
1	2	3	1
4	5	6	2

4.3.3. Phép chia

Phép chia được dùng để lấy ra một số bộ trong quan hệ R sao cho thỏa với

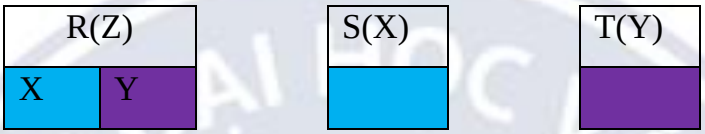
tất cả các bộ trong quan hệ S.

Ký hiệu: R ÷ S.

Cho R(Z) và S(X) trong đó Z là tập thuộc tính của R, X là tập thuộc tính của S; $X \subseteq Z$

Kết quả của phép chia R ÷ S là một quan hệ T(Y) với Y = Z - X và có t là một bộ của T nếu với mọi bộ $t_S \in S$, tồn tại bộ $t_R \in R$ thỏa 2 điều kiện:

$t_R(Y) = t$ và $t_R(X) = t_S(X)$



Ví dụ:

R	A	B	C	D	E
	α	a	α	a	1
	α	a	γ	a	1
	α	a	γ	b	1
	β	a	γ	a	1
	β	a	γ	b	3
	γ	a	γ	a	1
	γ	a	γ	b	1
	γ	a	β	b	1

S	D	E
	a	1
	b	1

R ÷ S

A	B	C
α	a	γ
γ	a	γ

3.4. Phép toán khác

3.4.1. Hàm kết hợp

Hàm kết hợp cho phép nhận vào một tập hợp các giá trị và trả về một giá trị hơn. Các hàm kết hợp gồm: AVG, MIN, MAX, SUM, COUNT.

3.4.2. Gom nhóm

Phép gom nhóm được dùng để phân chia quan hệ thành nhiều nhóm dựa trên điều kiện gom nhóm nào đó.

Ký hiệu: $^{G_1, G_2, \ldots, G_n}\mathbf{I}_{F1(A1), F2(A2), \ldots, Fn(An)}{}^{(E)}$

Trong đó:

- E là biểu thức ĐSQH
- $G_1, G_2, \ldots, G_n$ là các thuộc tính gom nhóm
- $F_1, F_2, \ldots, F_n$ là các hàm
- $A_1, A_2, \ldots, A_n$ là các thuộc tính tính toán trong hàm F

Ví dụ:

R	A	B	C
	α	2	7
	α	4	7
	β	2	3
	γ	2	10

$\mathcal{J}_{SUM(C)}(R)$

SUM_C
27

$_{A}\mathcal{J}_{SUM(C)}(R)$

SUM_C
14
3
10

3.4.3. Kết ngoài

Sử dụng phép kết mở rộng để tránh mất mát thông tin: thực hiện phép kết để lấy thêm các bộ không thỏa điều kiện kết.

Có 3 hình thức:

- Mở rộng bên trái: giữ lại tất cả các bộ bên trái
- Mở rộng bên phải: giữ lại tất cả các bộ bên phải
- Mở rộng 2 bên: giữ lại tất cả các bộ ở hai bên trái, phải

Ví dụ:

R	A	B	C
	α	α	1
	α	β	5
	β	β	12
	β	β	23

S	D	E
	1	7
	2	7
	12	3
	23	10

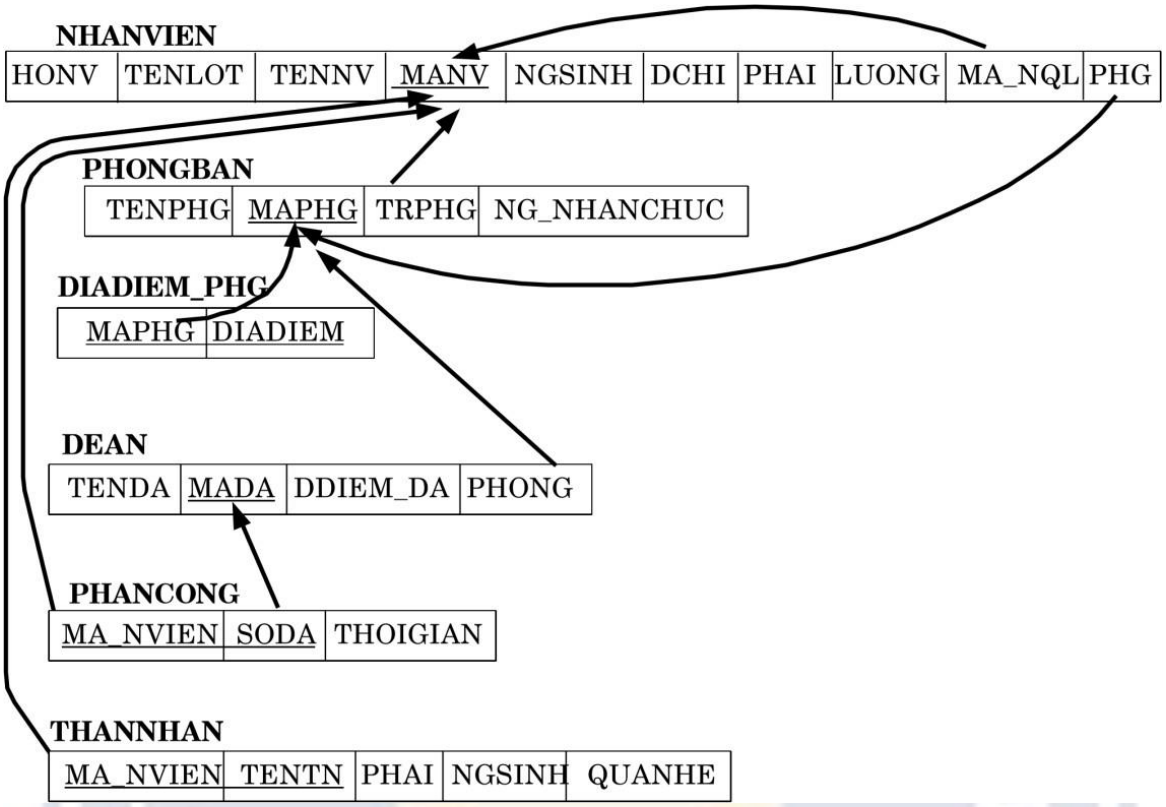
$R ⟗_{C<D} S$ →

A	B	C	D	E
α	α	1	2	7
α	α	1	12	3
α	α	1	23	10
α	β	5	12	3
α	β	5	23	10
β	β	12	23	10
β	β	23	null	null

3.5. Phép tính quan hệ biến bộ

Sử dụng lược đồ quan hệ Quản lý đề án.

Ví dụ:



4.5.1. Biến bộ

Biến bộ là biến biến thiên trên một quan hệ R xác định, { t | P(t) }: t là biến bộ, P(t) là vị từ hoặc là công thức.

- Biến tự do (Free variable)

  { t | t ∈ NHANVIEN ∧ t.LUONG > 30000 }

  t là biến tự do

- Biến kết buộc (Bound variable)

  { t | t ∈ NHANVIEN ∧ ∃s ∈ PHONGBAN (s.MAPHG = t.PHG) }

  Biến tự do Biến kết buộc

4.5.2. Biểu thức và phép tính quan hệ

Biểu thức phép tính quan hệ trên bộ có dạng **{ t.A | P(t) }** trong đó:

- t là biến bộ, là biến nhận giá trị là một bộ của quan hệ trong CSDL.
- t.A là giá trị của bộ t tại thuộc tính A.

- P là công thức có liên quan đến t, P(t) có giá trị **Đúng** hoặc **Sai** phụ thuộc vào t.
- Kết quả trả về là tập các bộ t sao cho P(t) **Đúng**.

Ví dụ: Tìm các nhân viên có lương trên 30000

$$\{ t \mid \underbrace{t \in NHANVIEN}_{P(t)} \wedge \underbrace{t.LUONG > 30000}_{P(t)} \}$$

Một công thức truy vấn tổng quát có dạng:

{ $t_1.A_i$, $t_2.A_j$, …$t_n.A_k$ | $P(t_1, t_2, \ldots, t_n)$ }

Trong đó:

- $t_1, t_2, \ldots, t_n$ là các biến bộ.
- $A_i, A_j, \ldots, A_k$ là các thuộc tính trong các bộ t tương ứng.
- P là công thức, được hình thành từ những công thức nguyên tố. Công thức nguyên tố có chân trị **Đúng** hoặc **Sai**, có các dạng sau:
- $t \in R$
  + t là biến bộ
  + R là quan hệ
- $t.A\ \theta\ s.B$
  + A là thuộc tính của biến bộ t
  + B là thuộc tính của biến bộ s
  + θ là các phép so sánh < , > , ≤ , ≥ , ≠ , =
- $t.A\ \theta\ c$
  + c là hằng số
  + A là thuộc tính của biến bộ t
  + θ là các phép so sánh < , > , ≤ , ≥ , ≠ , =

Ví dụ: Cho quan hệ R như sau

R	A	B	C
	α	10	1
	α	20	1

t_1 = <α, 10, 1>, $t_1 \in R$ có chân trị **Đúng**

t_2 = <α, 20, 2>, $t_2 \in R$ có chân trị **Sai**

Qui tắc:

- Mọi công thức nguyên tố là công thức

- Nếu P là công thức thì:
  + $\neg$P là công thức
  + (P) là công thức
- Nếu P1 và P2 là các công thức thì:
  + P1 $\vee$ P2 là công thức
  + P1 $\wedge$ P2 là công thức
  + P1 $\Rightarrow$ P2 là công thức
- Nếu P(t) là công thức thì
  + $\forall$t $\in$ R (P(t)) là công thức
    - Chân trị **Đúng** khi P(t) ĐÚNG với mọi bộ t trong R
    - Chân trị **Sai** khi có ít nhất 1 bộ làm cho P(t) SAI
  + $\exists$t $\in$ R (P(t)) là công thức
    - Chân trị **Đúng** khi có ít nhất 1 bộ làm cho P(t) ĐÚNG
    - Chân trị **Sai** khi P(t) SAI với mọi bộ t trong R
- Nếu P là công thức nguyên tố thì các biến bộ t trong P là biến tự do.
- Công thức P=P1$\wedge$P2 , P=P1$\vee$P2 , P=P1$\Rightarrow$P2 Sự xuất hiện của biến t trong P là tự do hay kết buộc phụ thuộc vào việc nó là tự do hay kết buộc trong P1, P2

3.6. Phép tính quan hệ biến miền

Biểu thức phép tính quan hệ trên miền có dạng

$\{ x_1, x_2, \ldots, x_n \mid P(x_1, x_2, \ldots, x_n) \}$.

Trong đó: $x_1, x_2, \ldots, x_n$ là các biến miền, nhận giá trị là một miền giá trị của một thuộc tính; P là công thức theo $x_1, x_2, \ldots, x_n$ và P được hình thành từ những công thức nguyên tố.

Kết quả trả về là tập các giá trị $x_1, x_2, \ldots, x_n$ sao cho khi các giá trị được thay thế cho các x_i thì P đúng.

Ví dụ:

- Cho biết mã và tên nhân viên có lương trên 30000

{ r, s | $\exists$x (<p, q, r, s, t, u, v, x, y, z> $\in$ NHANVIEN $\wedge$ x > 30000) }

Ví dụ:

- Cho biết các nhân viên (MANV) làm việc ở phòng ‘Nghien cuu’

{ s | $\exists$z (<p, q, r, s, t, u, v, x, y, z> $\in$ NHANVIEN $\wedge$ $\exists$a, b (<a, b, c, d> $\in$

PHONGBAN ∧ a = ‘Nghien cuu’ ∧ b = z))}

BÀI TẬP CHƯƠNG 4

Cho lược đồ CSDL quan hệ ‘Quản lý máy bay’ như sau:

PLANE_TYPE (model, capacity, weight) Lưu trữ thông tin loại máy bay gồm model củamáy bay (vd: boeing 777, Boeing 40A, Airbus A300, ATR666...), sức chứa của loại máy bay, trọng lượng của loại máy bay)

AIRPLANE (reg, model) Lưu trữ thông tin của máy bay bao gồm biển số, loại máy bay (model).

EMPLOYEE (ssn, name, address, birthday, phone, salary, level, startdate) Lưu trữ thôngtin của nhân viên (Mã số, tên, địa chỉ, ngày sinh, điện thoại, lương, cấp bậc, ngày bắt đầu làm việc)

WORKS_ON (ssn, model) Lưu trữ nhân viên nào làm việc được trên loại máy bay nào.

Viết các biểu thức đại số quan hệ biểu diễn cho các câu truy vấn sau:

1. Liệt kê các mã nhân viên (ssn) làm việc được trên máy bay có loại máy bay (model) làBoeing 777.

2. Liệt kê những nhân viên (ssn, name, phone) sinh sau năm 1977 và làm việc được trênloại máy bay ATR666.

3. Với mỗi loại máy bay (model) cho biết lương trung bình của tất cả nhân viên làm việc được trên loại máy bay đó.

4. Liệt kê các nhân viên (ssn, name) không làm việc được trên loại máy bay Airbus A300.

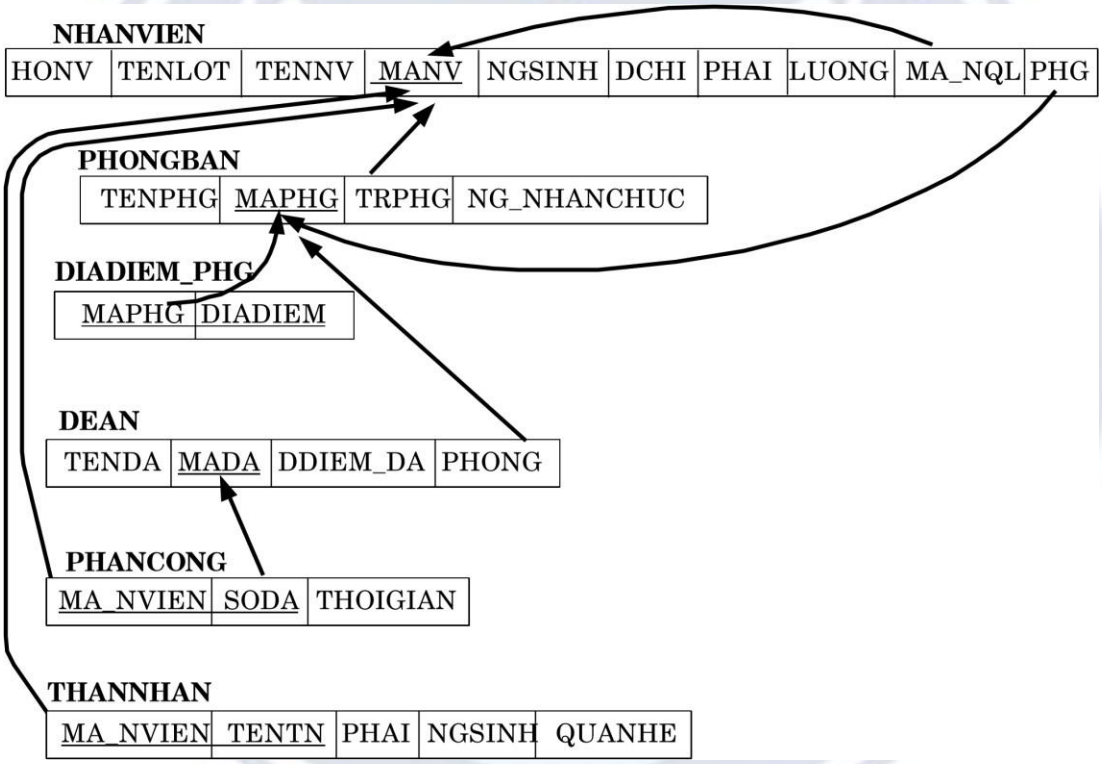
5. Liệt kê các nhân viên (ssn, name) làm việc được trên tất cả các loại máy bay. 6. Không dùng phép toán trừ, hãy liệt kê các loại máy bay (model) mà không một nhân viên nào làm việc được trên đó.

Chương 5
NGÔN NGỮ TRUY VẤN SQL

5.1. Giới thiệu

SQL(Structured Query Language) là ngôn ngữ truy vấn dựa trên đại số quan hệ. Câu lệnh của SQL dùng để rút trích dữ liệu của một một hay nhiều quan hệ. Kết quả của một câu lệnh SQL (truy vấn) là một quanhệ. Để đơn giản trong cách trình bày, ta xem quan hệ mà câu truy vấn sử dụng để tạo ra quan hệ khác gọi là quan hệ nguồn, quan hệ kết quả của truy vấn là quan hệ đích.

Sử dụng lược đồ cơ sở dữ liệu Quản lý Đề án cho các ví dụ minh họa trong phạm vi chương này.



5.2. Schema là gì?

Schema là một khái niệm mới được đưa vào SQL Server từ phiên bản 2005, nó là một **namespace** dùng để gom nhóm các table có chung một đặc điểm nào đó để dễ dàng quản lý. Nếu bạn không sử dụng Schema trong cơ sở dữ liệu thì nó sẽ lấy Schema mặc định là **dbo**.

Ví dụ: Trong lược đồ cơ sở dữ liệu của bạn có hai loại Table chính như sau:

Các Table về tin tức -> mình sẽ tạo Schema tên là news gồm những Table liên quan đến tin tức, Các table hệ thống -> mình sẽ tạo schema tên là sys gồm những table liên quan đến hệ thống.

Trong một database thì tên của schema là duy nhất, luôn được chỉ định.

Cú pháp:

server.database.schema.object.

Ưu điểm: Khi bạn phân nhóm các table lại thì sẽ rất dễ dàng quản lý, và bạn có thể phân quyền quản lý từng schema cho từng user khác nhau, đây chính là điểm mạnh của schema.

5.2.1. Tạo Schema với lệnh Create Schema

SQL Server cung cấp đầy đủ công cụ giao diện đồ họa đến công cụ dòng lệnh nên bạn có hai cách để tạo schema:

- **Cách 1:** Sử dụng SSMS(SQL Server Management Studio) thì bạn click chuột phải vào database và chọn Create schema.
- **Cách 2:** Tạo bằng dòng lệnh **CREATE SCHEMA**.

Cú pháp:

CREATE SCHEMA <Name_ SCHEMA **>;**

Ví dụ: Tạo hai schema tên là news và sys.

CREATE SCHEMA news;

CREATE SCHEMA sys;

5.2.2. Cách xóa Schema với lệnh DROP SCHEMA

Sau khi tạo schema xong nếu bạn không dùng tới thì hãy sử dụng lệnh DROP SCHEMA xóa schema.

Cú pháp:

DROP SCHEMA < **Name_** SCHEMA >;

Ví dụ:

DROP SCHEMA news;

DROP SCHEMA sys;

5.2.3. Phân quyền cho Schema

Giả sử bạn muốn user thehalfheart có quyền quản lý cho schema news thì chỉ cần sử dụng lệnh **GRANT**, còn xóa quyền thì dùng lệnh **DENY**.

- **Cấp quyền:**

GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA :: news TO thehalfheart;

- **Xóa quyền:**

DENY SELECT, INSERT, UPDATE, DELETE ON SCHEMA :: news TO thehalfheart;

Như vậy việc phân nhóm như thế này giúp lược đồ cơ sở dữ liệu của bạn trông chuyên nghiệp hơn, dễ dàng phân quyền và bảo mật hơn.

5.3. Định nghĩa dữ liệu

5.3.1. Kiểu dữ liệu

- Số (numeric): INTEGER, SMALLINT, NUMERIC, NUMERIC(p), NUMERIC(p,s), DECIMAL, DECIMAL(p), DECIMAL(p,s), REAL, DOUBLE PRECISION, FLOAT, FLOAT(p)
- Chuỗi ký tự (character string): CHARACTER, CHARACTER(n), CHARACTER VARYING(x), Chuỗi bit (bit string), BIT, BIT(x), BIT VARYING(x)
- Ngày giờ (datetime)
  + DATE gồm ngày, tháng và năm
  + TIME gồm giờ, phút và giây
  + TIMESTAMP gồm ngày và giờ

5.3.2. Các lệnh định nghĩa dữ liệu

a. ***Tạo cơ sở dữ liệu***

Cú pháp:

CREATE DATABASE <Tên_CSDL>

Ví dụ:

CREATE DATABASE QL_NhanVien

b. ***Xóa cơ sở dữ liệu***

Cú pháp:

DROP DATABASE [RESTRICT | CASCADE]

Ví dụ:

DROP DATABASE QL_NhanVien

c. ***Tạo bảng***

Để định nghĩa một bảng cần có các thông tin: Tên bảng, tên thuộc tính và kiểu dữ liệu, các ràng buộc toàn vẹn(RBTV) trên thuộc tính.

Cú pháp:

CREATE TABLE <Tên_bang> (

<Tên_cột> <Kiểu_dữ_liệu> [<RBTV>],

<Tên_cột> <Kiểu_dữ_liệu> [<RBTV>],

…

[<RBTV>]

)

- Các RBTV có thể:
  + NOT NULL: Không được để trống (rỗng)
  + NULL: cho phép rỗng
  + UNIQUE: duy nhất
  + DEFAULT: giá trị mặc định khi nhập thêm mẫu tin mới
  + PRIMARY KEY: khóa chính
  + FOREIGN KEY / REFERENCES: Khóa ngoại
  + CHECK: kiểm tra miền giá trị
- Có thể đặt tên cho RBTV khóa chính thông qua phát biểu:

  CONSTRAINT <Ten_RBTV> <RBTV>

Ví dụ:

```
CREATE TABLE NHANVIEN (
    HONV VARCHAR(10) CONSTRAINT NV_HONV_NN NOT NULL,
    TENLOT VARCHAR(20) NOT NULL,
    TENNV VARCHAR(10) NOT NULL,
    MANV CHAR(9) CONSTRAINT NV_MANV_PK PRIMARY KEY,
    NGSINH    DATETIME,
    DCHI VARCHAR(50),
    PHAI CHAR(3) CONSTRAINT NV_PHAI_CHK
    CHECK (PHAI IN ('Nam', 'Nu')),
    LUONG INT CONSTRAINT NV_LUONG_DF DEFAULT (10000),
    MA_NQL CHAR(9),
    PHG INT
)
```

d. ***Hiệu chỉnh bảng***

Muốn thay đổi cấu trúc bảng hay RBTV có thể sử dụng các phát biểu:

+ Thêm cột

ALTER TABLE <Tên_bảng> **ADD COLUMN** <Tên_cột> <Kiểu_dữ_liệu> [<RBTV>]

+ Xóa cột

ALTER TABLE <Tên_bảng> **DROP COLUMN** <Tên_cột>

+ Hiệu chỉnh kiểu dữ liệu cột

ALTER TABLE <Tên_bảng> **ALTER COLUMN** <Tên_cột> <Kiểu_dữ_liệu_mới>

e. ***Thêm ràng buộc toàn vẹn bằng lệnh UNIQUE trong SQL Server***

- Ràng buộc quan trọng trong SQL dùng để xử lý vấn đề bảo toàn ràng buộc toàn vẹn dữ liệu đó là UNIQUE.
- UNIQUE là ràng buộc trên giá trị duy nhất trên column, có nghĩa là các dòng dữ liệu không được có giá trị trùng nhau ở column đó. Ràng buộc này giống như khóa chính vậy, vì bản chất nó cũng là một khóa.

Ví dụ:

Trong bảng SINHVIEN sẽ có MASINHVIEN là khóa chính, và cột CCCD sẽ là UNIQUE. Thực ra bạn cũng có thể lấy CCCD là khoa chính nhưng như vậy sẽ không hay lắm.

Chúng ta có hai cách tạo UNIQUE như sau:

Thiết lập ngay cuối column:

Cách này bạn sẽ đặt từ khóa UNIQUE ở cuối column muốn nó có giá trị không trùng lặp.

```
CREATE TABLE persons(
  person_id INT IDENTITY PRIMARY KEY,
  first_name VARCHAR(255) NOT NULL,
  last_name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE
);
```

Column email là **UNIQUE**.

Thiết lập ở cuối các column:

Trường hợp này bạn đặt UNIQUE (column_list) ở cuối lệnh Create Table, nếu có nhiều column thì cách nhau bởi dấu phẩy.

```
CREATE TABLE persons(
  person_id INT IDENTITY PRIMARY KEY,
  first_name VARCHAR(255) NOT NULL,
  last_name VARCHAR(255) NOT NULL,
  email VARCHAR(255),
```

```
    UNIQUE(email)
);
```

SQL Server sẽ tự động tạo ra một **INDEX UNIQUE** và chạy nó mỗi khi bạn thực hiện thao tác INSERT hoặc UPDATE, nếu bạn cập nhật dữ liệu bị trùng thì nó sẽ thông báo lỗi.

Hãy thử chạy hai câu lệnh INSERT dưới đây để xem có đúng không nhé.

Ví dụ: Lệnh 1

```
INSERT INTO persons(first_name, last_name)
VALUES('John', 'Smith');
```

Lệnh 1 này hoạt động bình thường.

Ví dụ: Lệnh 2

```
INSERT INTO persons(first_name, last_name, email)
VALUES('Jane', 'Doe', 'j.doe@bike.stores');
```

Lệnh 2 sẽ bị báo lỗi vì cột email bị trùng:

Violation of UNIQUE KEY constraint 'UQ__persons__AB6E616417240E4E'.
Cannot insert duplicate key in object 'hr.persons'. The duplicate key value is (j.doe@bike.stores).

Đặt tên cho UNIQUE

Cái tên UQ__persons__AB6E616417240E4E là do SQL Server tự sinh ra vì bạn không đặt tên cho UNIQUE, nếu bạn muốn đặt tên thì hãy sử dụng từ khóa CONSTRAINT.

```
CREATE TABLE persons (
    person_id INT IDENTITY PRIMARY KEY,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    CONSTRAINT unique_email UNIQUE(email)
);
```

Tại sao chúng ta cần đặt tên? Như bạn thấy khi nhận được thông báo lỗi thì ta sẽ dễ dàng biết được do đâu, thứ hai nữa bạn dễ dàng thay đổi khi sử dụng kết hợp với lệnh ALTER TABLE.

- **Phân biệt Primary Key với UNIQUE**
  + Primary Key là khóa chính của bảng, đây là index đặc biệt nên bạn không chấp nhận giá trị NULL
  + UNIQUE cũng là một khóa nhưng vì là một column bình thường nên có thể nhận giá trị NULL.

NULL cũng là một giá trị bình thường nên khi có 2 record đều NULL thì vẫn bị lỗi duplicate value nhé.

- **Tạo UNIQUE cho nhiều column**

Để tạo UNIQUE cho nhiều column thì bạn phải sử dụng cách thứ hai đó là **thiết lập ở cuối các column**,

Cú pháp:

```
CREATE TABLE table_name (
    key_column data_type PRIMARY KEY,
    column1 data_type,
    column2 data_type,
    column3 data_type,
    ...,
    UNIQUE (column1,column2)
);
```

Ví dụ: Dưới đây tạo UNIQUE cho hai column person_id và skill_id.

```
CREATE TABLE person_skills (
    id INT IDENTITY PRIMARY KEY,
    person_id int,
    skill_id int,
    updated_at DATETIME,
    UNIQUE (person_id, skill_id)
);
```

- **Thêm UNIQUE bằng lệnh ALTER TABLE**

Khi bạn đã tạo xong table rồi nhưng sau đó muốn bổ sung UNIQUE cho một field nào đó thì hãy sử dụng lệnh ALTER TABLE.

Cú pháp:

```
ALTER TABLE table_name
```

```
ADD CONSTRAINT constraint_name
UNIQUE(column1, column2,...);
```

Ví dụ: Giả sử ta có table như sau.

```
CREATE TABLE hr.persons (
    person_id INT IDENTITY PRIMARY KEY,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    phone VARCHAR(20),
);
```

Bây giờ mình sẽ bổ sung UNIQUE cho hai column email và phone.

```
ALTER TABLE persons
ADD CONSTRAINT unique_email UNIQUE(email);
ALTER TABLE persons
ADD CONSTRAINT unique_phone UNIQUE(phone);
```

- **Xóa UNIQUE**

UNIQUE cũng là một **constraint** nên bạn hoàn toàn có thể sử dụng lệnh DROP CONSTRAINT để xóa.

Cú pháp:

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

Ví dụ: Xóa unique có tên là unique_phone ra khỏi bảng persons.

```
ALTER TABLE hr.persons
DROP CONSTRAINT unique_phone;
```

f. *Thêm ràng buộc toàn vẹn bằng lệnh CHECK trong SQL Server*

- **CHECK trong SQL Server**

CHECK là ràng buộc miền giá trị, nghĩa là nó chỉ giá trị của bạn nhập vào chỉ nằm trong một phạm vi nhất định.

Ví dụ:

column DIEMTHI thì phải nằm trong khoảng [0, 10], giới tính thì phải là {'Nam', 'Nữ'}.

Hãy xem ví dụ tạo ràng buộc giá trị của sản phẩm phải lớn hơn 0 như sau.

```
CREATE SCHEMA test;
GO
  CREATE TABLE test.products(
    product_id INT IDENTITY PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    unit_price DEC(10,2) CHECK(unit_price > 0)
);
```

Nếu bạn muốn để sau này dễ quản lý nên đặt tên cho CHECK.

Ví dụ:

```
CREATE TABLE test.products(
    product_id INT IDENTITY PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    unit_price DEC(10,2) CONSTRAINT positive_price CHECK(unit_price >
0) );
```

Với cách này thì bạn dễ dàng thay đổi CHECK bằng lệnh ALTER TABLE. Nếu bạn cố tình nhập sai dữ liệu như ví dụ dưới đây thì sẽ bị báo lỗi.

```
INSERT INTO test.products(product_name, unit_price)
VALUES ('Awesome Free Bike', 0);
```

Lỗi xuất hiện như sau:

The INSERT statement conflicted with the CHECK constraint positive_price". The conflict occurred in database "BikeStores", table "test.products", column 'unit_price'.

Lưu ý: Trong ví dụ này price vẫn chấp nhận giá trị NULL vì trong SQL Server NULL và 0 là hai giá trị khác nhau.

- **Tạo nhiều điều kiện ở CHECK**

Nếu bạn muốn tạo nhiều điều kiện ở CHECK thì hãy sử dụng toán tử AND để nối các điều kiện với nhau.

```
CREATE TABLE test.products(
    product_id INT IDENTITY PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    unit_price DEC(10, 2),
    discounted_price DEC(10, 2),
```

```
    CHECK(unit_price > 0),
    CHECK(discounted_price > 0 AND discounted_price > unit_price
);
```

- **Thêm CHECK ở lệnh ALTER TABLE**

Nếu bạn tạo Table rồi nhưng quên tạo CHECK thì có thể sử dụng lệnh ALTER TABLE.

+ **Không đăt tên**

```
ALTER TABLE test.products
ADD discounted_price DEC(10, 2)
CHECK(discounted_price > 0);
```

+ **Có đặt tên**

```
ALTER TABLE test.products
ADD CONSTRAINT valid_price
CHECK(unit_price > discounted_price);
```

- **Xóa ràng buộc CHECK**

Nếu bạn muốn xóa CHECK thì hãy sử dụng lệnh ALTER TABLE kết hợp DROP CONSTRAINT nhé.

Cú Pháp:

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

Nếu bạn không muốn xóa mà tạm thời vô hiệu hóa nó thì hãy sử dụng lệnh sau:

Cú Pháp:

```
ALTER TABLE test.products
NO CHECK CONSTRAINT valid_price
```

5.4. Cập nhật dữ liệu

5.4.1. Thêm dữ liệu

- Có thể thêm 1 hay nhiều dòng vào bảng
- Để thêm dữ liệu cần các thông tin: tên quan hệ, danh sách các thuộc tính cần thêm dữ liệu và danh sách các giá trị tương ứng
- **Thêm một dòng**

Cú pháp:

```
INSERT INTO <table_name> (<column_list>)
```

VALUES (<value_list>);

Trong đó:

+ table_name là tên của table muốn thêm dữ liệu
+ column_list là danh sách các column
+ value_list là danh sách các giá trị tương ứng với column

Các cặp giá trị của column và value tương ứng với nhau. value đứng thứ 3 sẽ thuộc về column đứng thứ 3.

Ví dụ: Thêm một sinh viên mới.

```
INSERT INTO SINHVIEN (ID, NAME, AGE)
VALUES (1, "Nguyen Van Cuong", 29);
```

Giá trị của các column tương ứng như sau:

ID	NAME	AGE
1	Nguyen Van Cuong	29

- **Các ví dụ với lệnh INSERT**

Sau đây là các ví dụ minh họa cho các trường hợp thường gặp khi thêm dữ liệu vào table với lệnh INSERT.

+ **Tạo một bảng như sau**

```
CREATE TABLE sales.promotions (
    promotion_id INT PRIMARY KEY IDENTITY (1, 1),
    promotion_name VARCHAR (255) NOT NULL,
    discount NUMERIC (3, 2) DEFAULT 0,
    start_date DATE NOT NULL,
    expired_date DATE NOT NULL
);
```

Hãy để ý đến các ràng buộc nhé, nó rất quan trọng trong việc xử lý dữ liệu trước khi chạy lệnh SQL.

+ **INSERT không trả về giá trị**

Trường hợp này thì chạy bình thường.

```
INSERT INTO sales.promotions (
    promotion_name,
    discount,
```

```
    start_date,
    expired_date
)
VALUES
    (
        '2018 Summer Promotion',
        0.15,
        '20180601',
        '20180901'
    );
```

Các bạn thấy mình không truyền dữ liệu cho khóa chính **promotion_id**, lý do là cột này được thiết lập tăng tự động nên giá trị sẽ tự tăng lên 1 cho mỗi record. Chạy câu SQL này bạn sẽ nhận được kết thành công quả như sau:

Lỗi xuất hiện như sau:

1 (1 row affected)

Để xem dữ liệu đã được thêm chưa thì ta chạy lệnh.

```
SELECT   *
FROM   sales.promotions;
```

Kết quả:

promotion_id	promotion_name	discount	start_date	expired_date
1	2018 Summer Promotion	0.15	2018-06-01	2018-09-01

+ **INSERT lấy thông tin trả về**

Tất cả dữ liệu vừa được insert sẽ được lưu trong biến inserted, và để lấy dữ liệu của inserted thì ta dùng lệnh OUTPUT.

Ví dụ: Dưới đây lấy ID vừa insert.

```
INSERT INTO sales.promotions (
        promotion_name,
        discount,
        start_date,
        expired_date
) OUTPUT inserted.promotion_id
VALUES
```

```
(
        '2018 Fall Promotion',
        0.15,
        '20181001',
        '20181101'
);
```

Nếu muốn lấy nhiều **column** hơn thì hãy tách chúng bởi dấy phẩy.

```
INSERT INTO sales.promotions (
        promotion_name,
        discount,
        start_date,
        expired_date
) OUTPUT inserted.promotion_id,
        inserted.promotion_name,
        inserted.discount,
        inserted.start_date,
        inserted.expired_date
VALUES
(
        '2018 Winter Promotion',
        0.2,
        '20181201',
        '20190101'
);
```

+ **Gán dữ liệu cho khóa chính tăng tự động**

Khác với MySQL, SQL Server không cho phép bạn gán dữ liệu cho khóa chính tăng tự động (identity), vì vậy khi bạn cố tình gán thì sẽ bị và không thêm được.

Ví dụ: Hãy xem câu truy vấn dưới đây:

```
INSERT INTO sales.promotions (
        promotion_id,
        promotion_name,
        discount,
```

```
        start_date,
        expired_date
    ) OUTPUT inserted.promotion_id
    VALUES
    (
        4,
        '2019 Spring Promotion',
        0.25,
         '20190201',
         '20190301'
     );
```

Mình đã cố gắng gán giá trị cho ID là 4 nên khi chạy câu SQL này sẽ bị lỗi như sau:

Lỗi xuất hiện như sau:

Cannot insert explicit value for identity column in table 'promotions' when IDENTITY_INSERT is set to OFF.

Ý nó muốn bạn hãy tắt **Identity** trước rồi mới gán được data cho ID.

Để tắt / mở Identity cho thao tác Insert thì bạn chạy câu SQL sau:

- **Lệnh ON**

  SET IDENTITY_INSERT table_name ON;

- **Lệnh OFF**

  SET IDENTITY_INSERT table_name OFF;

Như vậy tổng câu lệnh SQL cho phần này như sau:

```
SET IDENTITY_INSERT sales.promotions ON;
INSERT INTO sales.promotions (
    promotion_id,
    promotion_name,
    discount,
    start_date,
    expired_date
)
VALUES
```

```
(
        4,
        '2019 Spring Promotion',
        0.25,
        '20190201',
        '20190301'
);
SET IDENTITY_INSERT sales.promotions OFF;
```

- **Thêm nhiều dòng**

Cú pháp:

INSERT INTO <tên bảng>(<danh sách các thuộc tính>) <câu truy vấn con>

Ví dụ:

Bảng thống kê được tạo có cấu trúc như sau:

```
CREATE TABLE THONGKE_PB (
    TENPHG VARCHAR(20),
    SL_NV INT,
    LUONG_TC INT
);
```

Câu lệnh chèn nhiều dòng:

```
INSERT INTO THONGKE_PB(TENPHG, SL_NV, LUONG_TC)
  SELECT TENPHG, COUNT(MANV), SUM(LUONG)
  FROM NHANVIEN, PHONGBAN
  WHERE PHG=MAPHG
  GROUP BY TENPHG
```

5.4.2. Xóa dữ liệu

- Dùng phát biểu **Delete** để xóa dữ liệu

Cú pháp:

DELETE FROM <tên bảng> [**WHERE** <điều kiện>]

Lưu ý:

+ Số lượng số dòng bị xóa phụ thuộc vào điều kiện ở mệnh đề WHERE
+ Nếu không chỉ định điều kiện ở mệnh đề WHERE, tất cả các dòng trong bảng sẽ bị xóa

+ Lệnh DELETE có thể gây ra vi phạm RB tham chiếu: không cho xóa, xóa luôn những dòng có giá trị đang tham chiếu đến CASCADE
+ Đặt NULL cho những giá trị tham chiếu

- **Delete** kết hợp **Limit** trong SQL Server

Một điều khá đặc biệt mà ít người chú ý đến đó là bạn có thể kết hợp lệnh Order By và Limit để tạo ra khoảng giới hạn cần xóa.

Cú pháp:

```
DELETE FROM table
LIMIT row_count;
```

Để sắp xếp thì ta sẽ thêm câu lệnh Order By.

Cú pháp:

```
DELETE FROM table_name
ORDER BY c1, c2, ...
LIMIT row_count;
```

Ví dụ:

Xóa 10 nhân viên đầu tiên và danh sách được sắp xếp theo tên nhân viên.

```
DELETE FROM customers
ORDER BY customerName
LIMIT 10;
```

Hoặc xóa 20 nhân viên đầu tiên sắp xếp theo employeeNumber.

```
DELETE FROM customers
ORDER BY employeeNumber
LIMIT 20;
```

5.4.3. Sửa dữ liệu

Lệnh **Update** giúp bạn cập nhật dữ liệu có sẵn trong table, thường sẽ update theo một điều kiện khóa chính.

Cú pháp:

```
UPDATE <tên bảng>
SET <tên thuộc tính 1>=<giá trị mới 1>,
    <tên thuộc tính 2>=<giá trị mới 2>,
        ...
```

[**WHERE** <điều kiện>]

Trong đó các cặp <tên thuộc tính 1> = <giá trị mới 1>, <tên thuộc tính 2> = <giá trị mới 2>, chính là các cặp tên_field = value mà bạn muốn update. Không nhất thiết phải nhập tất cả các field trong table mà chỉ nhập vào những field mà bạn muốn update mà thôi.

Lưu ý:

Bạn vô tình nhập sai thông tin của sinh viên mã sô 'SV001' thì bạn sẽ xử lý lại bằng cách update lại thông tin theo mã số sinh viên đó. Nếu bạn không nhập điều kiện thì nó sẽ update toàn bộ các record trong bảng.

Ví dụ:

- Thay đổi email của nhân viên có mã số 1056.

```
UPDATE employees
SET
    email = 'mary.patterson@classicmodelcars.com'
WHERE employeeNumber = 1056;
```

Ví dụ: Update nhiều field

Để update nhiều field thì ta chỉ đặt chúng cách nhau bởi dấu phẩy trong mục SET.

```
UPDATE employees
SET
  lastname = 'Hill',
  email = 'mary.hill@classicmodelcars.com'
WHERE
  employeeNumber = 1056;
```

- **Kết hợp lệnh Replace**

Giả sử bạn muốn thay đổi **phần đuôi email** của một nhân viên nào đó mà bạn chỉ nhớ tên email, tên job và mã văn phòng mà thôi. Lúc này bạn sẽ sử dụng nhiều lệnh ở Where và lệnh Replace.

Ví dụ:

```
UPDATE employees
SET
```

email =
REPLACE(email,'@classicmodelcars.com','@mysqltutorial.org')
WHERE
jobTitle = 'Sales Rep' AND officeCode = 6;

- **Một số điều kiện khác**

Lệnh Update trong SQL Server thực ra nó là câu lệnh riêng biệt, vì vậy bạn có thể kết hợp với Where với nhiều điều kiện khác nhau.

- Kết hợp với **IS NULL**

```
SELECT  customername,  salesRepEmployeeNumber
FROM  customers
WHERE salesRepEmployeeNumber IS NULL;
```

- Kết hợp với **IS NOT NULL**

```
SELECT  salesRepEmployeeNumber
FROM   customers
WHERE   salesRepEmployeeNumber IS NOT NULL;
```

5.5. Truy vấn dữ liệu

Khi có nhu cầu thể hiện các dòng dữ liệu của một quan hệ hay của nhiều quan hệ dưới dạng một quan hệ có số cột và số dòng theo ý muốn như bảng điểm của sinh viên, danh sách sinh viên thì ta sử dụng truy vấn chọn.

5.5.1. Truy vấn cơ bản

Gồm ba thành phần cơ bản trong một phát biểu truy vấn

Cú pháp:

SELECT <danh sách các cột>
FROM <danh sách các bảng>
WHERE <điều kiện>

Trong đó:

- <danh sách các cột>
  + Tên các cột cần được hiển thị trong kết quả truy vấn
- <danh sách các bảng>
  + Tên các bảng liên quan đến câu truy vấn
- <điều kiện>
  + Biểu thức boolean xác định dòng nào sẽ được rút trích

+ Nối các biểu thức: AND, OR, và NOT

+ Phép toán: <, >, ≤, ≥, ≠, =, LIKE và BETWEEN

- **Mệnh đề SELECT**

Chỉ định tên các thuộc tính cần hiển thị trong kết quả truy vấn bằng cách chỉ định theo cấutrúc <ten bảng>.<tên thuộc tính>; nếu tên thuộc tính trong các bảng truy vấn không trùng nhau, có thể sử dụng <tên thuộc tính> Dùng dấu * để lấy tất cả các thuộc tính có trong các bảng ở mệnh đề FROM

- **Mệnh đề FROM**

Mệnh đề From chỉ định các bảng cung cấp dữ liệu khi thực hiện truy vấn.

- **Mệnh đề WHERE**

Mệnh đề Where để chỉ điều kiện cần truy vấn.

Ví dụ:

In ra danh sách các nhân viên (MANV, HONV, TENLOT, TENNV) làm việc ở phòng có mã là 5 và có Phái là Nam.

```
SELECT MANV, HONV, TENLOT, TENNV
FROM NHANVIEN
WHERE PHG=5 AND PHAI='Nam'
```

Ví dụ:

In ra tất cả thông tin của các nhân viên có trong danh sách

```
SELECT *
FROM NHANVIEN
```

- **Sắp xếp kết quả truy vấn**

Dùng để hiển thị kết quả câu truy vấn theo một thứ tự nào đó (tăng dần hoặc giảm dần)

Cú pháp:

```
SELECT <danh sách các cột>
FROM <danh sách các bảng>
WHERE <điều kiện>
ORDER BY <danh sách các cột>
```

- **ASC**: sắp xếp tăng dần (mặc định)
- **DESC**: sắp xếp giảm dần.

Ví dụ:

In ra danh sách các nhân viên (MA-NVIEN, SODA) theo thứ tự giảm dần của mã nhânviên, nếu một nhân viên tham gia nhiều đề án thì sắp xếp SODA tăng dần.

```
SELECT MA_NVIEN, SODA
FROM PHANCONG
ORDER BY MA_NVIEN DESC, SODA;
```

5.5.2. Tập hợp, so sánh tập hợp và truy vấn lồng

a. ***Phép toán tập hợp***

- SQL cho phép cài đặt các phép toán tập hợp: Hội (UNION), Giao (INTERSECT), Trừ (EXCEPT).
- Kết quả trả về là tập hợp sẽ loại bỏ các bộ trùng nhau.
- Để giữ lại các bộ trùng nhau dùng các phép toán: UNION ALL, INTERSECT ALL, EXCEPT ALL.

Cú pháp:

SELECT <ds cột>
FROM <ds bảng>
WHERE <điều kiện>
UNION [ALL]/ INTERSECT [ALL]/ EXCEPT [ALL]
SELECT <ds cột>
FROM <ds bảng>
WHERE <điều kiện>

Ví dụ: Cho biết các mã đề án có nhân viên với họ là 'Nguyen' tham gia hoặc, trưởng phòng chủ trì đề án đó với họ là 'Nguyen'.

```
SELECT SODA
FROM NHANVIEN, PHANCONG
WHERE MANV=MA_NVIEN AND HONV='Nguyen'
UNION
SELECT MADA
FROM NHANVIEN, PHONGBAN, DEAN
WHERE MANV=TRPHG AND MAPHG=PHONG AND HONV='Nguyen'
```

b. ***So sánh tập hợp***

- So sánh tập hợp thường đi cùng với một số toán tử

+ IN, NOT IN: thuộc/ không thuộc tập hợp
+ ALL: tất cả
+ ANY hoặc SOME: bất kỳ hoặc một số phần tử

- Kiểm tra sự tồn tại: dùng EXISTS: tồn tại ; NOT EXISTS: không tồn tại

c. *Truy vấn lồng (truy vấn con)*

Cú pháp:

```
SELECT <danh sách các cột>
FROM <danh sách các bảng>
WHERE <so sánh tập hợp> (SELECT <danh sách các cột>
                         FROM <danh sách các bảng>
                         WHERE <điều kiện>);
```

- Các câu lệnh SELECT có thể lồng nhau ở nhiều mức. Các câu truy vấn con trong cùngmột mệnh đề WHERE được kết hợp bằng phép nối logic. Câu truy vấn con thường trảvề một tập các giá trị.
- Mệnh đề WHERE của câu truy vấn cha có dạng: <biểu thức> <so sánh tập hợp> <truyvấn con>
- **Có 2 loại truy vấn lồng:**
  + Lồng phân cấp: mệnh đề WHERE của truy vấn con không tham chiếu đến thuộc tính của các quan hệ trong mệnh đề FROM ở truy vấn cha. Khi thực hiện, câu truy vấn con sẽ được thực hiện trước.
  + Lồng tương quan: mệnh đề WHERE của truy vấn con tham chiếu ít nhất một thuộctính của các quan hệ trong mệnh đề FROM ở truy vấn cha. Khi thực hiện, câu truy vấn con sẽ được thực hiện nhiều lần, mỗi lần tương ứng với một bộ của truy vấn cha.

Ví dụ:

```
SELECT *
FROM NHANVIEN
WHERE MANV NOT IN ( SELECT MA_NVIEN
                    FROM THANNHAN );
```

Ví dụ: Tìm tất cả những nhân viên không có người thân

```
SELECT DISTINCT TENDA
FROM DEAN
```

WHERE MADA **IN** (SELECT SODA

FROM NHANVIEN, PHANCONG

WHERE MANV=MA_NVIEN **AND**

HONV='Nguyen') **OR** MADA **IN** (

SELECT *MADA*

FROM *NHANVIEN, PHONGBAN, DEAN*

WHERE *MANV=TRPHG AND*

MAPHG=PHONGAND HONV='Nguyen');

5.5.3. Hàm kết hợp và gom nhóm

a. *Hàm kết hợp*

Các hàm kết hợp được đặt ở mệnh đề SELECT, các hàm kết hợp thường được sử dụng:

- COUNT(*) đếm số dòng
- COUNT(<tên thuộc tính>) đếm số giá trị khác NULL của thuộc tính
- COUNT(DISTINCT <tên thuộc tính>) đếm số giá trị khác nhau và khác NULL củathuộc tính
- MIN (giá trị nhỏ nhất), MAX (giá trị lớn nhất), SUM (tính tổng), AVG (tính giá trịtrung bình cộng)

Ví dụ:

Tính tổng lương, lương lớn nhất, lương nhỏ nhất, lương trung bình của tất cả cácnhân viên.

```
SELECT SUM(LUONG), MAX(LUONG), MIN(LUONG), AVG(LUONG)
FROM NHANVIEN;
```

b. *Gom nhóm*

- ***Mệnh đề Gom nhóm GROUP BY***

+ Sử dụng hàm gom nhóm trên các bộ trong quan hệ.
+ Mỗi nhóm bộ bao gồm tập hợp các bộ có cùng giá trị trên các thuộc tính gom nhóm
+ Hàm gom nhóm áp dụng trên mỗi bộ độc lập nhau.
+ SQL có mệnh đề GROUP BY để chỉ ra các thuộc tính gom nhóm, các thuộc tính này phải xuất hiện trong mệnh đề SELECT
+ Thông thường các hàm kết hợp thường hay được sử dụng đi kèm với gom

nhóm dữ liệu.

Cú pháp:

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE <điều kiện>

GROUP BY <danh sách các cột gom nhóm>

Ví dụ: Tính lương trung bình của các nhân viên theo từng phòng ban.

SELECT MaPhong, avg(LUONG) as LUONGTB

FROM NhanVien

GROUP BY MaPhong;

- ***Mệnh đề điều kiện gom nhóm HAVING***

+ Lọc kết quả theo điều kiện, sau khi đã gom nhóm
+ Điều kiện ở HAVING được thực hiện sau khi gom nhóm, các điều kiện có liên quan đến thuộc tính Group By

Cú pháp:

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE <điều kiện>

GROUP BY <danh sách các cột gom nhóm>

HAVING <điều kiện sau khi gom nhóm>

Ví dụ: Tìm phòng có số lượng nhân viên 'Nữ' trên 5 người

SELECT MaPhong

FROM NhanVien

WHERE Phai = 'Nữ'

GROUP BY MaPhong

HAVING count(MaNV) > 5

5.5.4. Một số dạng truy vấn khác

a. *Truy vấn con ở mệnh đề FROM*

Kết quả trả về của một câu truy vấn phụ là một bảng, bảng này là trung gian trong quá trìnhtruy vấn và không có lưu trữ thật sự

Cú pháp:

SELECT <danh sách các cột>

FROM R1, R2, **(**<truy vấn con>**) AS** tên_bảng

WHERE <điều kiện>

Ví dụ: Cho biết mã phòng, tên phòng và lương trung bình của các phòng có mức lương trung bình trên 20000.

```
SELECT PHG, TENPHG, AVG(LUONG) AS LUONG_TB
FROM NHANVIEN, PHONGBAN
WHERE PHG=MAPHG
GROUP BY PHG, TENPHG
HAVING AVG(LUONG) > 20000
```

b. ***Lệnh Inner Join trong SQL Server***

Không chỉ SQL Server mà ở hầu hết các hệ quản trị CSDL quan hệ hiện nay thì Inner join được sử dụng nhiều nhất trong các loại Join, nó giúp ta liên kết nhiều table với nhau thông qua ràng buộc khóa ngoại.

Cú pháp:

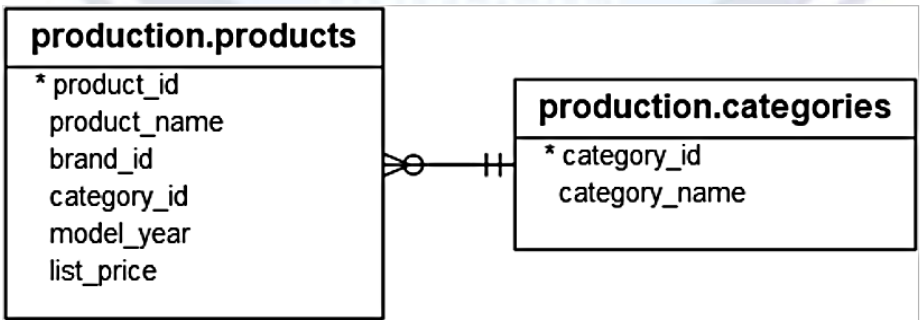
SELECT select_list

FROM R1 **INNER JOIN** R2 **ON** join_predicate;

Trong đó:

- select_list: là danh sách các field bạn muốn chọn ở hai bảng.
- R1 và R2 là hai table cần JOIN với nhau.
- INNER JOIN là toán tử JOIN
- ON join_predicate là điều kiện JOIN.

Ví dụ: Ta có hai Table như sau



<u>Cách 1:</u>

SELECT product_name, category_name, list_price

FROM production.products p **INNER JOIN** production.categories c

ON c.category_id = p.category_id

ORDER BY product_name **DESC**;

Cách 2:

SELECT product_name, category_name, list_price

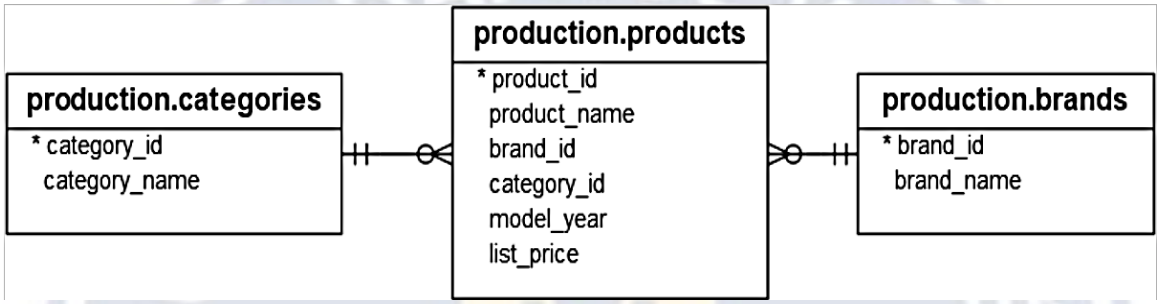
FROM production.products p, production.categories c

WHERE c.category_id = p.category_id

ORDER BY product_name **DESC**;

Hãy chú ý: **c.category_id = p.category_id** chính là điều kiện JOIN giữa hai bảng. Nó sẽ so sánh record giữa hai bảng nếu có giá trị category_id trùng nhau thì sẽ chọn.

Ví dụ: Ta có ba Table như sau



Bảng **products** sẽ có hai mối liên hệ. Mỗi sản phẩm sẽ thuộc một **category** và thuộc một **brand**.

Bây giờ có bài toán yêu cầu như sau: Hãy lấy danh sách sản phẩm kèm thông tin về chuyên mục và thương hiệu mà nó thuộc về.

Để làm bài này thì ta thực hiện phép JOIN giữa ba bảng với nhau, và bảng **products** là bảng chính bởi nó có các khóa ngoại liên kết để hai bảng còn lại.

Cách 1:

SELECT product_name, category_name, brand_name, list_price

FROM production.products p

INNER JOIN production.categories c **ON** c.category_id = p.category_id

INNER JOIN production.brands b **ON** b.brand_id = p.brand_id

ORDER BY product_name **DESC**;

Cách 2:

SELECT product_name, category_name, brand_name, list_price

FROM production.products p, production.categories c,

production.brands b

WHERE c.category_id = p.category_id **AND** b.brand_id = p.brand_id

ORDER BY product_name **DESC**;

c. ***Lệnh Left/Right Join trong SQL Server***

Left Join sẽ hợp hai bảng lại và lấy những cặp record thỏa mãn ở điều kiện ON. Những dòng dữ liệu ở bảng bên trái có nhưng bảng bên phải không có và ngược lại với right.

Cú pháp:

SELECT select_list

FROM R1 **LEFT| RIGHT JOIN** T2 **ON** join_predicate**;**

Trong đó:

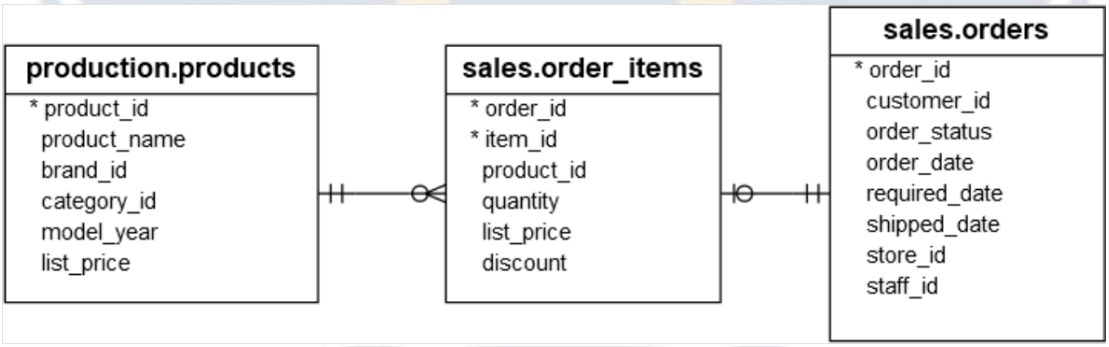
Select_list: Là những field mà bạn muốn câu lệnh này tra về. Bạn có thể chọn các field bất kì, miễn là nó thuộc một trong hai bảng R1 và R2.

join_predicate: Chính là điều kiện kiểm tra ràng buộc giữa hai bảng. Nếu dòng record nào thỏa điều kiện này thì ta gọi là thỏa điều kiện ON.

Left| Right Join: Sẽ trả về những dòng thỏa điện kiện ở ON với nhau và những dòng mà bảng R1 có dữ liệu nhưng bảng R2 không có.

Ví dụ:

Giả sử ta có ba table gồm: products, order_items và orders có mối liên hệ như lược đồ diagram sau:



Mỗi đơn hàng sẽ có nhiều sản phẩm, vì vậy ta sẽ có bảng trung gian nằm giữa để chia mối quan hệ [n – n] này thành hai mối quan hệ mới:

Gữa **products** và **order_items** là [1 – n]

Giữa **orders** và **order_items** là [1 – n]

Bây giờ cần lấy danh sách tên sản phẩm và mã số **order_id** của các đơn hàng, nhưng với điều kiện là lấy luôn những sản phẩm không có đơn nào, vì vậy ta sẽ đặt bảng **products** nằm bên trái và **order_items** nằm bên phải.

SELECT product_name, order_id

FROM production.products p **LEFT JOIN** sales.order_items o **ON** o.product_id = p.product_id

ORDER BY order_id;

Ví dụ: join 3 bảng với nhau.

SELECT p.product_name, o.order_id, i.item_id, o.order_date

FROM production.products p

LEFT JOIN sales.order_items i **ON** i.product_id = p.product_id

LEFT JOIN sales.orders o **ON** o.order_id = i.order_id

ORDER BY order_id;

Với trường hợp này thì SQL Server sẽ thực hiện 2 lần Join như sau:

Lần thứ nhất join bảng **products** với **order_items** => tạo ra danh sách các record và nói là một bảng tạm. Lúc này bảng **products** là bảng bên trái.

Lần thứ hai join bảng tạm đó với bảng **orders**, lúc này bảng tạm chính là bảng bên trái.

d. ***Thêm lệnh Where trong Left|Right Join***

Nếu bạn muốn lọc thêm điều kiện thì hãy đặt nó trong lệnh WHERE nhé.

Giả sử mình cần lấy thông tin tên các sản phẩm mã đơn hàng của đơn hàng có **order_id = 100**.

SELECT product_name, order_id

FROM production.products p

LEFT JOIN sales.order_items o **ON** o.product_id = p.product_id

WHERE order_id = 100

ORDER BY order_id;

e. ***Cấu trúc CASE***

Cho phép kiểm tra điều kiện và xuất thông tin theo từng trường hợp.

Cú pháp:

CASE <tên cột>

WHEN <giá trị> **THEN** <biểu thức>

WHEN <giá trị> **THEN** <biểu thức>

...

[**ELSE** <biểu thức>]

END

Ví dụ:

Cho biết họ tên các nhân viên đã đến tuổi về hưu (nam 60 tuổi, nữ 55 tuổi)

```
SELECT HONV, TENNV
FROM NHANVIEN
WHERE YEAR(GETDATE()) – YEAR(NGSINH) >= ( CASE PHAI
                    WHEN 'Nam' THEN 60
                    WHEN 'Nu' THEN 55
                    END );
```

BÀI TẬP CHƯƠNG 5

Cho lược đồ cơ sở dữ liệu Quản lý Sách như sau:

LOAISACH(MaLoai, TenLoai)

DAUSACH(Masach, TuaSach, MaLoai, TacGia, NgayXuatBan, SoLuong, GiaBan)

Thể hiện của lược đồ:

LOAISACH	
MaLoai	TenLoai
1	Van hoc
2	Toan hoc
3	Tin hoc
4	Anh van

DAUSACH						
Ma Sach	TuaSach	Ma Loai	TacGia	Ngay XuatBan	So Luong	Gia Ban
1	Nguoi me	1	Nguyen Ngoc Thach	08-04-2009	120	30.000
2	Lap trinh Java	3	Le Minh Khiet	11-04-2008	240	45.000
3	100 Essays	4	Hoang Ngoc Minh	03-02-2007	250	23.000
4	Delta Key	4	Nancy Galla	10-02-2008	110	110.000
5	Ben them hoa nang	1	Truong Ngoc Hung	09-02-2008	90	80.000

DAUSACH						
Ma Sach	TuaSach	Ma Loai	TacGia	Ngay XuatBan	So Luong	Gia Ban
6	Giai tich	2	Dang Ngoc Thanh	02-01-2006	50	35.000
7	Thuong mai dien tu	3	Hoang Tran Nghia	03-07-2009	70	50.000
8	Symbian	3	Luong Ngoc Han	01-02-2009	90	29.000
9	Toan cao cap	2	Hoang Minh Luan	12-03-2001	30	40.000
10	Toan roi rac	2	Tran Khanh Tuyen	03-01-2006	30	60.000

Hãy viết lệnh SQL để thực hiện các yêu cầu sau:

1. Tạo bảng, tạo khóa chính, khóa ngoại
2. Thêm dữ liệu vào các bảng
3. Cho biết danh mục các loại Sách hiện có.
4. Cho biết danh mục các sách có tác giả là Le Minh Khiet.
5. Cho biết danh mục các Sách có tựa sách có chứa chữ Java.
6. Cho biết TuaSach, TacGia, SoLuong của các sách có giá bán trên 40000.
7. Cho biết các thông tin Tựa sách, ngày xuất bản được xuất bản từ năm 2008 đến 2015
8. Cho biết TuaSach, TacGia, SoLuong, GiaBan của các sách thuộc nhóm Văn học.
9. Cho biết số đầu sách của các loại sách, thông tin có: TenLoai, SoDauSach.
10. Cho biết các sách có tựa sách bắt đầu bằng Toan, thông tin có: TuaSach, TacGia,NamXuatBan.
11. Cho biết danh mục các sách có các tác giả bắt đầu bằng Hoang, thông tin gồm”TuaSach, TacGia, SoLuong, DonGia.

Chương 6
RÀNG BUỘC TOÀN VẸN

6.1. Giới thiệu

Ràng buộc toàn vẹn (Integrity Constraint) là kiểm tra sự vi phạm ràng buộc toàn vẹn trong quá trình phân tích, thiết kế và khai thác cơ sở dữ liệu. Nếu không quan tâm đến vấn đề này thì có thể dẫn đến hậu quả rất nghiêm trọng về tính an toàn và toàn vẹn dữ liệu.

Ràng buộc toàn vẹn là một điều kiện bất biến không được vi phạm trên cơ sở dữ liệu. Trong một cơ sở dữ liệu tồn tại rất nhiều mối liên hệ ảnh hưởng qua lại lẫn nhau giữa các thuộc tính của một quan hệ, giữa các bộ giá trị trong một quan hệ và giữa các thuộc tính của các bộ giá trị trong các quan hệ với nhau. Các mối quan hệ phụ thuộc lẫn nhau này chính là những điều kiện bất biến mà tất cả các bộ của những quan hệ có liên quan trong cơ sở dữ liệu đều phải thỏa mãn bất kỳ thời điểm nào. Ràng buộc toàn vẹn còn được gọi là quy tắc quản lý được áp đặt lên trên các đối tượng của thế giới thực.

6.2. Các yếu tố của ràng buộc toàn vẹn

6.3.1. Bối cảnh

Bối cảnh của ràng buộc toàn vẹn là tập những quan hệ mà khi thực hiện thêm, xóa, sửa có khả năng làm cho ràng buộc toàn vẹn đó bị vi phạm. Do đó bối cảnh ràng buộc toàn vẹn trên một hoặc nhiều quan hệ cần phải được kiểm tra ngay khi cập nhật dữ liệu. Ràng buộc toàn vẹn phát sinh cần được ghi nhận và xử lý một cách tường minh thường là một hàm chuẩn hoặc một đoạn chương trình.

Ví dụ: Tổng số giờ làm việc của nhân viên không vượt quá 40 giờ trên một tuần

6.3.2. Nội dung

Nội dung của ràng buộc toàn vẹn có thể được phát biểu bằng ngôn ngữ tự nhiên hoặc ngôn ngữ hình thức (Ngôn ngữ đại số quan hệ, mã giả, Phép tính quan hệ, Truy vấn SQL, …).

Ngôn ngữ tự nhiên: Dễ hiểu không chặt chẽ, logic

Ngôn ngữ hình thức: Chặt chẽ, cô đọng

Ví dụ:

Một số ràng buộc toàn vẹn trên lược đồ cơ sở dữ liệu quản lý nhân viên.

Mỗi phòng ban có một mã số duy nhất để phân biệt với các phòng ban khác trong công ty.

Mỗi nhân viên có một mã số duy nhất để phân biệt các nhân viên khác trong công ty.

6.3.3. Bảng tầm vực ảnh hưởng

Người phân tích thiết kế cơ sở dữ liệu cần phải lập bảng tầm ảnh hưởng ràng buộc toàn vẹn cho một hoặc nhiều quan hệ nhằm xác định thời điểm kiểm tra ngay khi cập nhật dữ liệu.

Những thuộc tính khoá (những thuộc tính nằm trong khóa chính của quan hệ) không được phép sửa giá trị.

6.1. Bảng tầm ảnh hưởng

Tên RBTV (R_i)	Thêm (T)	Xóa (X)	Sửa (S)
r_1	-	+	-
r_2	-	-	+
r_3	+(A)	-	+
...			
r_n	+	+	-(*)

Dấu + : Có thể gây ra vi phạm RBTV

Dấu - : Không gây ra vi phạm RBTV

Dấu -(*) : Không vi phạm RBTV do thao tác không thực hiện được

Dấu +(A) : Có thể làm vi phạm RBTV khi sửa trên thuộc tính A

6.3. Phân loại ràng buộc toàn vẹn

Trong quá trình người phân tích thiết kế cơ sở dữ liệu cần phải phát hiện tất cả các ràng buộc toàn vẹn tiềm ẩn trong cơ sở dữ liệu. Rồi phân loại các ràng buộc toàn vẹn là rất có ích, nhằm giúp người phân tích thiết kế có một định hướng, không bỏ sót những ràng buộc toàn vẹn. Các ràng buộc toàn vẹn được chia làm hai loại chính.

- Ràng buộc toàn vẹn trên phạm vi một quan hệ (Miền giá trị, Liên bộ, Liên thuộc tính)
- Ràng buộc toàn vẹn trên phạm vi nhiều quan hệ (Tham chiếu, Liên bộ liên quan hệ, Liên thuộc tính liên quan hệ, Thuộc tính tổng hợp, Chu trình)
- Để minh họa cho ràng buộc toàn vẹn ta sử dụng ví dụ cơ sở dữ liệu sau đây

Ví dụ:

HOCVIEN (MAHV, HOTEN, NGSINH, GIOITINH, NOISINH, MALOP)

Tân từ: Mỗi học viên có một mã học viên (MAHV) duy nhất, mỗi MAHV xác định một họ tên học viên (HOTEN), một ngày sinh (NGSINH), một giới tính (GIOITINH), một nơi sinh (NOISINH) và thuộc một mã lớp duy nhất của học viên theo học (MALOP).

LOP (MALOP, TENLOP, LOPTRG, SISO, MAGVCN)

Tân từ: Mỗi lớp có một mã lớp (MALOP) duy nhất, mỗi MALOP xác định một tên lớp (TENLOP), một lớp trưởng (LOPTRG), một sỉ số (SISO) và thuộc một mã giáo viên chủ nhiệm duy nhất (MAGVCN).

KHOA (MAKHOA, TENKHOA, NGTLAP, TRGKHOA)

Tân từ: Mỗi khoa có một mã khoa (MAKHOA) duy nhất, mỗi MAKHOA xác định một tên khoa (TENKHOA), một ngày thành lập khoa (NGTLAP), một trưởng khoa (TRGKHOA).

MONHOC (MAMH, TENMH, TCMH, MAKHOA)

Tân từ: Mỗi môn học có một mã môn học (MAMH) duy nhất, mỗi MAMH xác định một tên môn học (TENMH), một số tín chỉ môn học (TCMH) và thuộc một mã khoa duy nhất phụ trách (MAKHOA).

DIEUKIEN (MAMH, MAMHTRUOC)

Tân từ: Mỗi môn học (MAMH) có thể có một hoặc nhiều môn học phải buộc học trước nó (MAMHTRUOC) còn gọi là môn học tên quyết.

GIANGVIEN(MAGV, HOTEN, HOCVI, HOCHAM, GIOITINH, NGSINH, NGVL, HESO, MUCLUONG, MAKHOA)

Tân từ: Mỗi giảng viên có một mã giảng viên (MAGV) duy nhất, mỗi MAGV xác định một họ tên giảng viên (HOTEN), một học vị (HOCVI), một học hàm (HOCHAM), một giới tính (GIOITINH), một ngày sinh (NGSINH), một ngày vào làm (NGVL), một hệ số lương (HESO), một mức lương (MUCLUONG), và thuộc một mã khoa duy nhất (MAKHOA).

GIANGDAY(MALOP, MAMH, MAGV, HOCKY, NAM, TUNGAY, DENNGAY)

Tân từ: Một lớp học (MALOP) có thể được mở nhiều môn học (MAMH) được tổ chức trong các học kỳ (HOCKY) của một năm học (NAM) có thể do nhiều giảng viên chịu trách nhiệm giảng dạy (MAGV), một môn học có thể được

mở nhiều lần cho các năm, thời gian mở lớp từ ngày (TUNGAY) đến ngày (DENNGAY).

KETQUA(MAHV, MAMH, LANTHI, NGTHI, DIEM, KQUA)

Tân từ: Mỗi học viên (MAHV) đăng ký học môn học (MAMH) trong một học kỳ một môn học có thể tổ chức thi nhiều lần (LANTHI) và ngày tổ chức thi (NGTHI) sẽ có điểm đánh giá (DIEM) của học viên, từ điểm đánh giá sẽ có kết quả (KETQUA) đạt hay không đạt.

6.3.1. Ràng buộc toàn vẹn liên quan đến một quan hệ

6.3.1.1. Ràng buộc toàn vẹn miền giá trị

Ràng buộc toàn vẹn miền giá trị là tập giá trị mà một thuộc tính có thể nhận. Một số hệ quản trị cơ sở dữ liệu có thể tự động kiểm tra một số ràng buộc toàn vẹn miền giá trị.

Ví dụ:

R1: Giới tính của học viên chỉ là Nam hoặc Nữ

+ Nội dung:

$\forall$hv $\in$ HOCVIEN: hv.Gioitinh $\in$ {'Nam','Nữ'}

+ Bối cảnh: quan hệ HOCVIEN
+ Bảng tầm ảnh hưởng:

R1	Thêm	Xóa	Sửa
HOCVIEN	+	-	+(GIOITINH)

6.3.1.2. Ràng buộc toàn vẹn liên bộ

Ràng buộc toàn vẹn liên bộ là ràng buộc toàn vẹn giữa các bộ trên cùng một quan hệ. Ràng buộc toàn vẹn liên bộ còn gọi ràng buộc toàn vẹn về khóa, nó tồn tại trong mọi lược đồ quan hệ của cơ sở dữ liệu và được các hệ quản trị cơ sở dữ liệu kiểm tra tự động.

Ví dụ:

R2: Tất cả các học viên phải có mã số phân biệt với nhau

+ Nội dung:

$\forall h_1, h_2 \in$ HOCVIEN: Nếu $h_1 \neq h_2$ thì h_1.MAHV $\neq$ h_2.MAHV

+ Bối cảnh: quan hệ HOCVIEN
+ Bảng tầm ảnh hưởng:

R2	Thêm	Xóa	Sửa
HOCVIEN	+	-	-(*)

6.3.1.3. Ràng buộc toàn vẹn liên thuộc tính

Ràng buộc toàn vẹn liên thuộc tính là mối liên hệ giữa các thuộc tính trên cùng một lược đồ quan hệ.

Ví dụ:

R3: Ngày bắt đầu (TUNGAY) giảng dạy một môn học cho một lớp luôn nhỏ hơn ngày kết thúc (DENNGAY)

+ Nội dung:

  $\forall$gd $\in$ GIANGDAY: gd.TUNGAY < gd.DENNGAY

+ Bối cảnh : GIANGDAY
+ Bảng tầm ảnh hưởng:

R3	Thêm	Xóa	Sửa
GIANGDAY	+	-	+(TUNGAY, DENNGAY)

6.3.2. Ràng buộc toàn vẹn liên quan đến nhiều quan hệ

6.3.2.1. Ràng buộc toàn vẹn tham chiếu

Là ràng buộc quy định giá trị thuộc tính trong một bộ của quan hệ R (tập thuộc tính này gọi là khoá ngoại), phải phụ thuộc vào sự tồn tại của một bộ trong quan hệ S (tập thuộc tính này là khóa chính trong quan hệ S).

Ràng buộc toàn vẹn tham chiếu còn gọi là ràng buộc phụ thuộc tồn tại hay ràng buộc khóa ngoại.

Ví dụ:

R4: Học viên thi một môn học nào đó thì môn học đó phải có trong danh sách các môn học

+ Nội dung:
  - $\forall$k $\in$ KETQUATHI, $\exists$m $\in$ MONHOC: k.MAMH = m.MAMH
  - Hoặc: KETQUATHI[MAMH] $\subseteq$ MONHOC[MAMH]
+ Bối cảnh: quan hệ KETQUATHI, MONHOC

+ Bảng tầm ảnh hưởng:

R4	Thêm	Xóa	Sửa
KETQUATHI	+	-	-(*)
MONHOC	-	+	-(*)

6.3.2.2. Ràng buộc toàn vẹn liên bộ liên quan hệ

Ràng buộc toàn vẹn liên bộ liên quan hệ ràng buộc về từng nhóm các bộ của nhiều quan hệ bối cảnh khác nhau.

Ví dụ:

R5: Ngày giáo viên giảng dạy một môn học phải lớn hơn hoặc bằng ngày giáo viên đó vào làm việc.

+ Nội dung:

$\forall$gd $\in$ GIANGDAY

Nếu $\exists$gv $\in$ GIANGVIEN: gd.MAGV = gv.MAGV thì

gv.NGVL $\leq$ gd.TUNGAY

+ Bối cảnh: GIANGDAY, GIANGVIEN
+ Bảng tầm ảnh hưởng:

R5	Thêm	Xóa	Sửa
GIANGDAY	+	-	+(TUNGAY)
GIANGVIEN	-	-	+(NGVL)

6.3.2.3. Ràng buộc toàn vẹn liên thuộc tính liên quan hệ

Ràng buộc toàn vẹn liên thuộc tính liên quan hệ là ràng buộc giữa các thuộc tính, các bộ trên những quan hệ khác nhau.

Ví dụ: Xét cơ sở dữ liệu có hai quan hệ sau.

HOADON(<u>MAHD</u>, MAKH, NGAYHD, TRIGIA),

THANHTOAN(<u>MAHD, NGAYTTOAN, LANTTOAN</u>, SOTIENTTOAN)

R6: Giả sử cho phép thanh toán tiền nhiều lần và thanh toán sau khi mua hàng, khi đó ngày thanh toán tiền theo một hóa đơn mua hàng phải bằng hoặc sau ngày mua hàng.

+ Nội dung:

$\forall hd \in HOADON(\forall tt \in THANHTOAN(hd.MAHD = tt.MAHD$

$\Rightarrow hd.NGAYHD \leq tt.NGAYTTOAN))$

+ Bối cảnh:

HOADON(MAHD, MAKH, NGAYHD, TRIGIA),

THANHTOAN(MAHD, NGAYTTOAN, LANTTOAN, SOTIENTTOAN)

+ Bảng tầm ảnh hưởng:

R6	Thêm	Xóa	Sửa
HOADON	+	-	+(MAHD, NGAYHD)
THANHTOAN	+	-	+(MAHD, NGAYTTOAN)

6.3.2.4. Ràng buộc toàn vẹn thuộc tính tổng hợp

Ràng buộc toàn vẹn thuộc tính tổng hợp là ràng buộc giữa các thuộc tính, các bộ trên những quan hệ khác nhau. Thuộc tính tổng hợp là thuộc tính được tính toán từ giá trị của các thuộc tính khác, các bộ khác.

Ví dụ: Xét cơ sở dữ liệu quan hệ sau.

SANPHAM(MASP, TENSP, NUOCSX, GIA)

KHACHHANG(MAKH, HOTEN, DOANHSO)

HOADON(SOHD, NGHD, MAKH,TRIGIA)

CTHD(SOHD, MASP, SOLUONG, GIA)

Trị giá của một hóa đơn bằng tổng thành tiền của các chi tiết thuộc hóa đơn đó.

R7: Doanh số của một khách hàng bằng tổng trị giá các hóa đơn mà khách hàng đó đã mua.

+ Nội dung:

$\forall kh \in KHACHHANG,$

$kh.DOANHSO = \sum_{(hd \in HOADON:\ hd.MAKH=kh.MAKH)}(hd.TRIGIA)$

+ Bối cảnh: KHACHHANG, HOADON

+ Bảng tầm ảnh hưởng:

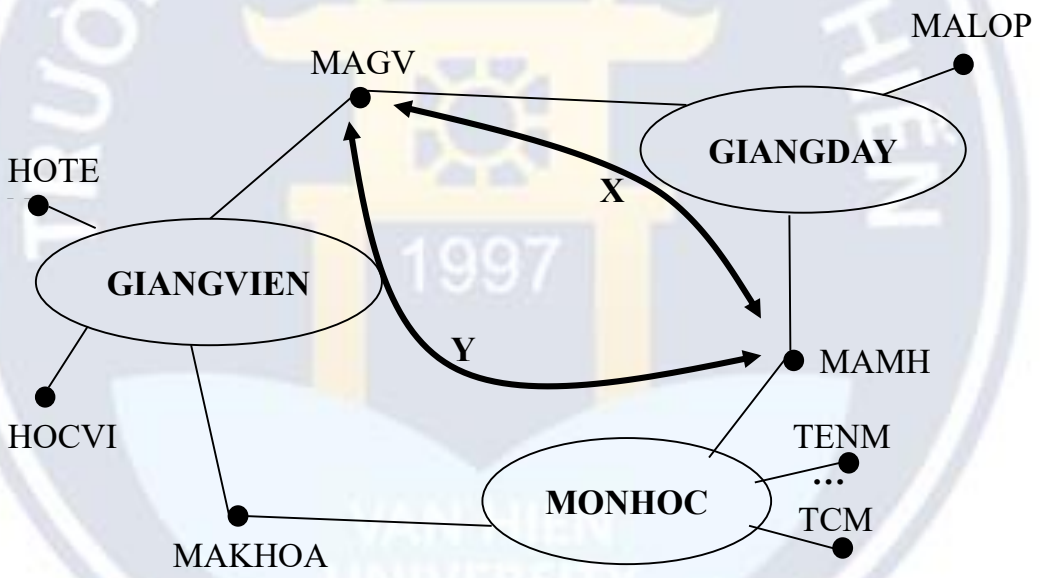
R7	Thêm	Xóa	Sửa
KHACHHANG	-	-	+(DOANHSO)

R7	Thêm	Xóa	Sửa
HOADON	+	+	+(TRIGIA)

6.3.2.5. Ràng buộc toàn vẹn chu trình

Biểu diễn lược đồ quan hệ dưới dạng đồ thị:

- Quan hệ được biểu diễn bằng nút tròn rỗng to.
- Thuộc tính được biểu diễn bằng nút tròn đen nhỏ.
- Tất cả các nút đều được chỉ rõ bằng tên của quan hệ hoặc thuộc tính. Thuộc tính thuộc một quan hệ được biểu diễn bởi một cung nối giữa nút tròn rỗng to và nút tròn đen nhỏ.
- Nếu đồ thị biểu diễn xuất hiện một đường khép kín => lược đồ cơ sở dữ liệu có sự hiện diện của chu trình.



Hình 6.1. Lược đồ cơ sở dữ liệu được biểu diễn dưới dạng đồ thị.

- X = **GIANGDAY**[MAGV, MAMH]
- Y = (GIANGVIEN $\bowtie_{\text{GIANGVIEN.MAKHOA= MONHOC.MAKHOA}}$ MONHOC) [MAGV, MAMH]
- **Ý nghĩa:**
  + X: giảng viên và những môn học đã được phân công cho giảng viên đó giảng dạy
  + Y: giảng viên và những môn học thuộc khoa giảng viên đó phụ trách
- Mối quan hệ giữa X và Y trong các ràng buộc sau:

+ **Ràng buộc 1**: giảng viên chỉ được phân công giảng dạy những môn học thuộc khoa giảng viên đó phụ trách **X⊆Y**
+ **Ràng buộc 2**: giảng viên phải được phân công giảng dạy tất cả những môn học thuộc khoa giảng viên đó phụ trách **X=Y**
+ **Ràng buộc 3**: có thể phân công giảng viên giảng dạy bất kỳ môn học nào **X ≠ Y**

Ví dụ:

R8: Giảng viên chỉ được phân công giảng dạy những môn học thuộc khoa giảng viên đó phụ trách **X⊆Y**

+ Nội dung:

$$GVMH \leftarrow MONHOC \bowtie_{MONHOC.MAKHOA=GIANGVIEN.MAKHOA} GIANGVIEN$$

$$\forall t \in GIANGDAY(\exists s \in GVMH(t.MAGV=s.MAGV \wedge t.MAMH=s.MAMH)$$

+ Bối cảnh: MONHOC, GIANGVIEN, GIANGDAY
+ Bảng tầm ảnh hưởng:

R8	Thêm	Xóa	Sửa
MONHOC	-	-	+(MAMH, MAKHOA)
GIANGVIEN	-	-	+(MAGV, MAKHOA)
GIANGDAY	+	-	+(MAGV, MAMH)

6.4. Cài đặt các ràng buộc toàn vẹn

6.4.1. Assertion

Các khẳng định (assertion) là dạng của một khẳng định. Câu lệnh CREATE ASSERTION được tạo cho một ràng buộc mới và xác định chế độ của ràng buộc, thời gian kiểm tra ràng buộc ban đầu và điều kiện tìm kiếm CHECK của nó. Cú pháp bắt buộc cho câu lệnh CREATE ASSERTION là.

CREATE ASSERTION <*Tên ràng buộc*>

CHECK (Điều kiện) *[<Thuộc tính ràng buộc>];*

CREATE ASSERTION xác định một quy tắc mới sẽ ràng buộc tập hợp các giá trị hợp lệ cho một hoặc nhiều bảng cơ sở dữ liệu.

Ví dụ:

CREATE TABLE LOP(

```
    MALOP    varchar(10),
    TENLOP   varchar(50),
    LOPTRG   varchar(50),
    SISO     number,
    MAGVCN   varchar(10)
);
```

- Sỉ số của mỗi lớp không được vượt quá 50.

```
CREATE ASSERTION Kiem_tra_si_so
CHECK (50 >= (SELECT SISO
              FROM LOP));
```

Ví dụ:

```
CREATE TABLE NHANVIEN(
    MANV          number,
    HOTEN         varchar2(50),
    TENLUONG      number,
    MAPHONGBAN    number
);
```

- Tổng lương của mỗi phòng ban không được vượt quá 100000000.

```
CREATE ASSERTION Kiểm tra tiền lương nhỏ hơn 100000000
CHECK (100000000 >= ALL (SELECT SUM(TENLUONG)
            FROM NHANVIEN
            GROUP BY MAPHONGBAN));
```

6.4.2. Trigger

Trigger là một loại thủ tục đặc biệt được thực thi một cách tự động khi có một sự kiện thay đổi cơ sở dữ liệu xảy ra như Update, Insert hay Delete. Trigger được dùng để đảm bảo ràng buộc dữ liệu, tính nhất quán, hoặc thực hiện các quy tắc dữ liệu phức tạp.

Trigger là đối tượng được tạo trên một bảng và là một phần của cơ sở dữ liệu. Trigger có thể truy vấn tới các bảng khác và có thể bao gồm các câu lệnh SQL(Structured Query Language) phức tạp. Chúng ta có thể tạo Trigger cho mỗi hành động trên bảng hoặc có thể kết hợp hai hoặc ba hành động (Insert/Update/Delete) trên một trigger.

Trigger được DBA(Database Administrator) xây dựng. Một cơ sở dữ liệu có một tập hợp các trigger được gọi là cơ sở dữ liệu hoạt động. Một trigger gồm ba phần:

- Sự kiện: Những thay đổi ảnh hưởng tới cơ sở dữ liệu khi trigger hoạt động.
- Điều kiện: Một truy vấn hoặc một kiểm tra được thực hiện khi trigger hoạt động.
- Hoạt động: Một thủ tục được thực hiện khi trigger hoạt động và các điều kiện của nó là đúng.

Trigger điều khiển một cơ sở dữ liệu, và thực hiện khi cơ sở dữ liệu có sự thay đổi ứng với sự kiện đã xác định.

Điều kiện của một trigger có thể trả về giá trị True/False hoặc một truy vấn. Một truy vấn hiểu là true nếu tập kết quả không rỗng và false nếu truy vấn không có kết quả trả về. Nếu phần điều kiện có giá trị true thì những hành động của trigger được thực hiện.

Một hoạt động có thể kiểm tra kết quả của truy vấn trong một phần điều kiện của trigger, tham chiếu tới các giá trị cũ và mới của những bộ giá trị thay đổi sau khi thực hiện trigger, thực hiện các truy vấn mới và làm những thay đổi đối với cơ sở dữ liệu. Thực tế, một hoạt động có thể thực hiện một chuỗi các câu lệnh định nghĩa dữ liệu.

Để hủy một Trigger, hãy sử dụng câu lệnh DROP TRIGGER hoặc để thay đổi một trigger hiện có.

Cú pháp câu lệnh CREATE TRIGGER là:

```
CREATE TRIGGER <Trigger name>
{BEFORE | AFTER} <trigger event> ON <Table name>
[ REFERENCING <old or new values alias list> ]
<triggered action>
  <trigger event> = INSERT | DELETE | UPDATE
        [ OF <trigger Column list> ]
  <trigger Column list> = <Column name> [ {,<Column name>} ... ]
  <old or new values alias list> = <old or new values alias>...
  <old or new values alias> =
    OLD [ ROW ] [ AS ] old values <Correlation name> |
```

NEW [ROW] [AS] new values <Correlation name> |

OLD TABLE [AS] <old values Table alias> |

NEW TABLE [AS] <new values Table alias>

<old values Table alias> = <identifier>

<new values Table alias> = <identifier>

<triggered action> =

[FOR EACH {ROW | STATEMENT}] [WHEN (search condition)]

<triggered SQL statement>

<triggered SQL statement> =

SQL statement | BEGIN ATOMIC {SQL statement;}... END

Chú thích:

- Thời gian kích hoạt : BEFORE hoặc AFTER. Cần phải chỉ định thời gian kích hoạt khi bạn xác định được tiến trình kích hoạt của nó. Sử dụng từ khóa BEFORE nếu bạn muốn xử lý hành động trước khi thực hiện thay đổi trên bảng và AFTER nếu bạn cần phải xử lý hành động sau khi thay đổi được thực hiện xong.
  - + Before Insert: trước khi nhập dữ liệu vào.
  - + Before Delete: trước khi xóa (chỉ khi hàng bị xóa).
  - + After Insert: sau khi nhập liệu
  - + After Delete: sau khi xóa (chỉ khi xóa hàng).
- Mệnh đề ON của câu lệnh CREATE TRIGGER tên bảng kích hoạt.
- Sự kiện Trigger xác định câu lệnh thay đổi dữ liệu SQL mà việc thực thi (trên Bảng Trigger) sẽ kích hoạt Trigger: nó có thể là Insert, Delete hoặc Update. Trong trường hợp chỉ Update, bạn có thể thêm điều khoản phụ tùy chọn liệt kê các cột trình kích hoạt: các cột mà trên đó Update sẽ kích hoạt trình kích hoạt (Update trên các cột không có trong danh sách sẽ không kích hoạt trình kích hoạt). Danh sách cột đặt tên cho một số hoặc tất cả các cột của bảng kích hoạt (mỗi cột chỉ có thể xuất hiện trong danh sách một lần). Nếu bạn bỏ qua điều khoản phụ tùy chọn này, hiệu quả giống như bạn đã bao gồm mệnh đề OF đặt tên cho mọi cột của bảng kích hoạt.

Cho cơ sở dữ liệu như sau:

Khohang(MaKH, MaHang, TenHang, SoLuongTon)

ĐatHang(MaDH,)

BÀI TẬP CHƯƠNG 6

Bài tập 1:

Cho cơ sở dữ liệu Quản lý bán hàng gồm các quan hệ như sau:

KHACHHANG (MAKH, HOTEN, DCHI, SODT, NGSINH, DOANHSO, NGDK)

Tân từ: Quan hệ khách hàng sẽ lưu trữ thông tin của khách hàng thành viên gồm có các thuộc tính: mã khách hàng, họ tên, địa chỉ, số điện thoại, ngày sinh, ngày đăng ký và doanh số (tổng trị giá các hóa đơn của khách hàng thành viên này).

NHANVIEN (MANV,HOTEN, NGVL, SODT)

Tân từ: Mỗi nhân viên bán hàng cần ghi nhận họ tên, ngày vào làm, điện thọai liên lạc, mỗinhân viên phân biệt với nhau bằng mã nhân viên.

SANPHAM (MASP,TENSP, DVT, NUOCSX, GIA)

Tân từ: Mỗi sản phẩm có một mã số, một tên gọi, đơn vị tính, nước sản xuất và một giá bán.

HOADON (SOHD, NGHD, MAKH, MANV, TRIGIA)

Tân từ: Khi mua hàng, mỗi khách hàng sẽ nhận một hóa đơn tính tiền, trong đó sẽ có số hóa đơn, ngày mua, nhân viên nào bán hàng, trị giá của hóa đơn là bao nhiêu và mã số của khách hàng nếu là khách hàng thành viên.

CTHD (SOHD,MASP,SL)

Tân từ: Diễn giải chi tiết trong mỗi hóa đơn gồm có những sản phẩm gì với số lượng là bao nhiêu.

Hãy cài đặt các ràng buộc toàn vẹn sau:

- Thuộc tính LOAIKH trong quan hệ KHACHHANG có thể lưu các giá trị là: “Vanglai”, “Thuong xuyen”, “Vip”, …
- Đơn vị tính của sản phẩm chỉ có thể là (“cây”,”hộp”,”cái”,”quyển”,”chục”)
- Giá bán của sản phẩm từ 500 đồng trở lên.
- Ngày gia nhập của khách hàng thành viên phải lớn hơn ngày sinh của người đó.
- Ngày mua hàng (NGHD) của một khách hàng thành viên sẽ lớn hơn hoặc bằng ngàykhách hàng đó đăng ký thành viên (NGDK).

- Ngày bán hàng (NGHD) của một nhân viên phải lớn hơn hoặc bằng ngày nhân viên đó vào làm.
- Tạo Update Trigger trên bảng Nhân Viên, khi sửa một record thì xuất ra câu thôngbáo "đã sửa thông tin một nhân viên".
- Viết Trigger cho bảng nhân viên, thực hiện sửa số điện thoại của một hoặc nhiều nhân viên thì in ra danh sách các mã nhân viên vừa được sửa thông tin.
- Viết Trigger cho phép khi thêm một chi tiết hóa đơn, trị giá hóa đơn trong bảng hóa đơn sẽ được tự động cập nhật.

Bài tập 2:

Giả sử có 2 bảng

LOP(**Malop**, TenLop),

SV(**Masv**, HoTen, Malop),

Với một ràng buộc dạng liên bộ như sau: Mỗi lớp chỉ có tối đa 10 hoc viên. Hãy viết 1 trigger để bẩy ràng buộc đó, nếu vi phạm ràng buộc thì nảy một mã lỗi và đưa ra thông báo "Lop này da du hoc vien".

Bài tập 3:

Giả sử có 2 bảng

LOP(**Malop**, TenLop, SiSoToida, SiSoThuc),

SV(**Masv**, HoTen, Malop),

Với các ràng buộc dạng liên bộ liên quan hệ như sau:

- Chỉ chèn thêm hoặc hiệu chỉnh chuyển lớp cho một sinh viên của bảng SV khi sỉ số của lớp đó vẫn còn nhỏ hơn SiSIToiDa.
- Khi có thao tác thêm, xoá, sửa thông tin trên bảng SV thì phải cập nhật lại SiSoThuc

Hãy bắt các ràng buộc trên và nảy ra mã lỗi thích hợp khi có các thao tác tương ứng xuất hiện mà vi phạm ràng buộc đặt ra.

Chương 7

PHỤ THUỘC HÀM VÀ CHUẨN HÓA CƠ SỞ DỮ LIỆU

7.1. Sự dư thừa thông tin

Trong quá trình thiết kế lược đồ quan hệ là đi làm tối thiểu không gian lưu trữ của quan hệ. Các thuộc tính được nhóm vào trong các lược đồ quan hệ có ảnh hưởng rất lớn đến không gian lưu trữ. Nếu cùng một thông tin được lưu giữ nhiều lần trong cơ sở dữ liệu ta gọi đó là dư thừa thông tin vì điều đó sẽ làm tăng không gian nhớ và khó khăn trong quá trình cập nhật dữ liệu.

Giả sử ta có bảng dữ liệu sau đây SINHVIEN_LOP của một trường đại học.

MaSV	HoTen	NgaySinh	NoiSinh	Phai	MaLop	TenLop
SV01	Trần Văn A	01/01/2000	Đồng Tháp	Nam	T01	Tin học
SV02	Lê Văn C	19/01/2001	Tp. HCM	Nữ	T01	Tin học
SV20	Hồ Trí D	18/09/2005	Nha Trang	Nam	A01	Anh văn
SV13	Trần Văn B	20/12/2002	Phú Yên	Nam	A01	Anh văn
SV34	Hồ Văn E	20/11/2004	Cần Thơ	Nữ	T01	Tin học
SV19	Nguyễn B	09/09/2003	Long An	Nam	K01	Kế toán
SV26	Lương D	07/09/2005	Tiền Giang	Nam	K01	Kế toán

Trên bảng cơ sở dữ liệu này ta thấy có sự dư thừa thông tin. Nếu một trường có nhiều sinh viên theo học thì thông tin về lớp (MaTop, TenLop) được lưu giữ nhiều lần trên một bảng. So với việc dùng hai bảng SINHVIEN và LOP riêng rẽ, việc sử dụng bảng này làm tăng không gian nhớ.

Ngoài việc làm tăng không gian nhớ nó còn dẫn đến các vấn đề dị thường nghiêm trọng trong quá trình cập nhật dữ liệu như chèn, xóa, sửa làm cho cơ sở dữ liệu mất tính đúng đắn.

- Dị thường Xóa: Gây ra việc mất thông tin khi xóa

Ví dụ: Khi ta xóa một bộ giá trị trên bàng SINHVIEN_LOP. Nếu sinh viên tương ứng với bộ giá trị đó là sinh viên cuối cùng của lớp thì phép xóa sẽ kéo theo việc làm mất thông tin về lớp.

- Dị thường Sửa: Gây ra việc sửa đổi hàng loạt khi ta sửa đổi một giá trị trên một bộ nào đó.

Ví dụ: Ta muốn thay đổi giá trị thuộc tính MaLop hoặc TenLop. Điều đó kéo theo ta phải sửa giá trị thuộc tính này trong tất cả các bộ ứng với MaLop hoặc TenLop.

- Dị thường Chèn: Gây ra khó khăn khi chèn các bộ giá trị vào bảng hoặc vi phạm ràng buộc.

Ví dụ:

Để chèn một bộ giá trị cho một sinh viên vào bảng SINHVIEN_LOP ngoài các thông tin của sinh viên ta phải đưa vào các thông tin MaLop, TenLop mà sinh viên đó theo học hoặc giá trị Null.

Khi thiết kế các lược đồ quan hệ sao cho không sinh ra các dị thường khi cập nhật dự liệu trong các quan hệ. Nếu có xuất hiện những dị thường khi cập nhật dữ liệu thì phải ghi chép lại một cách rõ ràng và phải đảm bảo các chương trình khi cập nhật dữ liệu sẽ thực hiện một cách đúng đắn.

7.2. Phụ thuộc hàm

7.2.1. Định nghĩa

Cho lược đồ quan hệ $R(A_1, A_2,\ldots, A_n)$. X, Y là hai tập con của $R^+(A_1, A_2,\ldots, A_n)$. ta nói X xác định Y hay Y phụ thuộc hàm vào X.

Ký hiệu: $X \rightarrow Y$.

r là quan hệ trên R.

t_1, t_2 là hai bộ bất kỳ thuộc r.

$X \rightarrow Y \Leftrightarrow (t_1.X = t_2.X \Rightarrow t_1.Y = t_2.Y)$.

Các giá trị của thành phần X của một bộ xác định duy nhất các giá trị của thành phần Y. Một phụ thuộc hàm từ X vào Y hoặc Y phụ thuộc hàm vào X phụ thuộc hàm được viết tắt FD (Functional depenency). Tập thuộc tính X được gọi là vế trái của FD, tập thuộc tính Y được gọi là vế phải của FD.

Ví dụ: xét lược đồ quan hệ sau.

SINHVIEN_LOP (MaSV, HoTen, NgaySinh, NoiSinh, Phai, MaLop, TenLop)

F là tập phụ thuộc hàm.

F = {MaSV $\rightarrow$ HoTen, NgaySinh, NoiSinh, Phai, MaLop, TenLop

MaLop $\rightarrow$ TenLop}

7.2.2. Các tính chất của phụ thuộc hàm

- Phụ thuộc hàm $X \rightarrow \varnothing$ đúng với mọi quan hệ r.

- Phụ thuộc hàm $\varnothing \rightarrow X$ chỉ đúng trên quan hệ r có cùng giá trịnh Y.

  Ví dụ: GIANGDAY(MaGV, MaMH, NgayDay, GioDay)

MaGV	MaMH	NgayDay	**GioDay**
GV01	MH01	18/09/2021	**13:15**
GV01	MH02	19/09/2021	**13:15**
GV03	MH04	18/09/2021	**07:15**
GV03	MH05	19/09/2021	**07:15**
GV03	MH06	20/09/2021	**07:15**

7.3. Hệ luật dẫn Armstrong

Hệ luật dẫn Armstrong là một phát biểu cho biết nếu một quan hệ r thỏa mãn vài phụ thuộc hàm thì nó phải thỏa mãn phụ thuộc hàm khác.

Chúng ta ký hiệu F là tập các phụ thuộc hàm được xác định trên một lược đồ quan hệ R. Một phụ thuộc hàm $X \rightarrow Y$ được gọi là suy diễn từ tập phụ thuộc hàm F được xác định trên R. Nếu $X \rightarrow Y$ đúng mỗi trạng thái quan hệ r.

Với X, Y, Z, W là tập con của R^+, r là quan hệ bất kỳ R.

1. Luật phản xạ (reflexivity)

Nếu $Y \subseteq X \Rightarrow X \rightarrow Y$

Chứng minh:

Giả sử $Y \subseteq X$ và hai bộ t_1, t_2 trên một quan hệ r của R sao cho $t_1.X = t_2.X$ khi đó $t_1.Y = t_2.Y$ bởi $Y \subseteq X$ như vậy $X \rightarrow Y$ phải xảy ra trên r.

2. Luật thêm vào (augmentation)

Cho $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$

Chứng minh:

Giả sử có $X \rightarrow Y$ đúng trên một quan hệ r của R nhưng $XZ \rightarrow YZ$ không đúng khi đó phải có hai bộ t_1, t_2 trên r sao cho.

$t_1.X = t_2.X$ (1)

$t_1.Y = t_2.Y$ (2)

$t_1.XZ = t_2.XZ$ và (3)

$t_1.YZ \neq t_2.YZ$ (4) Điều này là không đúng bởi vì (1) và (3) chúng ta suy ra $t_1.Z = t_2.Z$ (5) Từ (2) và (5) ta suy ra $t_1.YZ = t_2.YZ$ mâu thuẫn với (4)

3. Luật bắt cầu (transitivity)

Cho $\{X \to Y,\ Y \to Z\} \Rightarrow X \to Z$

Chứng minh:

Giả sử có $X \to Y$ (1)

$Y \to Z$ (2)

Cả hai đúng trên một quan hệ r của R. Khi đó với mọi bộ t_1, t_2 trên r sao cho

$t_1.X = t_2.X$ ta phải có $t_1.Y = t_2.Y$ (3) theo giả thiết (1). Vậy chúng ta cũng có

$t_1.Z = t_2.Z$ (4) theo (3) và giả thiết (2) suy ra $X \to Z$ phải đúng trên r.

4. Luật hợp (union)

Cho $\{X \to Y, X \to Z\} \Rightarrow X \to YZ$

Chứng minh:

Giả sử có: t1.X = t2.X (1)

$\Rightarrow$ t1.X = t2.X vaø t1.Z = t2.Z

$\Rightarrow$ t1.XZ = t2.XZ (2)

$\Rightarrow X \to YZ$ (do (1) $\Rightarrow$ (2))

5. Luật phân ra (decomposition)

Cho $X \to YZ \Rightarrow X \to Y$ và $X \to Z$

Chứng minh:

Giả sử có: t1.X = t2.X (1)

$\Rightarrow$ t1.YZ = t2.YZ (do $X \to YZ$)

$\Rightarrow$ t1.Y = t2.Y (2)

$\Rightarrow X \to Y$ (do (1) $\Rightarrow$ (2))

6. Luật bắt cầu giả (pseudotransitivity)

Cho $X \to Y$, $YZ \to W \Rightarrow XZ \to W$

Chứng minh:

Giả sử có: t1.XZ = t2.XZ (1)

$\Rightarrow t_1.X = t_2.X$ và $t_1.Z = t_2.Z$ (2)

$\Rightarrow t_1.Y = t_2.Y$ (do $X \to Y$) (3)

$\Rightarrow t_1.YZ = t_2.YZ$ (Kết hợp (2) và (3))

$\Rightarrow t_1.W = t_2.W$ (do $YZ \rightarrow W$) (4)

$\Rightarrow XZ \rightarrow W$

Armstrong đã chứng minh các luật suy diễn từ luật 1 đến luật 3 đúng và đầy đủ. Nghĩa là cho trước một tập phụ thuộc hàm F trên một lược đồ quan hệ R, bất kỳ một phụ thuộc hàm nào suy diễn bằng cách áp dụng các luật suy diễn từ luật 1 đến luật 3 cũng đúng trên mỗi trạng thái quan hệ r của R, thỏa mãn các phụ thuộc hàm trên F.

7.4. Bao đóng và thuật toán tìm bao đóng

7.4.1. Định nghĩa bao đóng

Cho lược đồ quan hệ $R(A_1, A_2, \ldots, A_n)$ cho $X \subseteq R$

F là tập phụ thuộc hàm trên $R(A_1, A_2, \ldots, A_n)$

Bao đóng của X trên F ký hiệu X^+ (hoặc X_F^+)

$X^+=\{A_i \mid A_i \in R$ và $F \models X \rightarrow A_i\}$

$F \models X \rightarrow A_i \equiv X \rightarrow A_i \in F^+$

$X \rightarrow A_i$ là phụ thuộc hàm được suy diễn từ F nhờ hệ tiên đề Armstrong.

7.4.2. Thuật toán tìm bao đóng

Input: Quan hệ $R(A_1, A_2, \ldots, A_n)$

F là tập phụ thuộc hàm trên $R(A_1, A_2, \ldots, A_n)$

$X \subseteq R$

Output: X_F^+

Tính các tập thuộc tính X_i theo các bước sau.

Bước 1: $X_0 = X$

Bước 2: lần lược xét các phụ thuộc hàm của F

Nếu $Y \rightarrow Z$ có $Y \subseteq X_i$ thì $X_{i+1} = X_i \cup Z$

Loại phụ thuộc hàm $Y \rightarrow Z$ ra khỏi F

Bước 3: Nếu ở bước 2 không tính được X_{i+1} thì X_i chính là bao đóng của X.

Ngược lại quay lại **Bước 2**

Ví dụ:

Cho lược đồ quan hệ R(U)=(ABCDEG) với U=(ABCDEG)

Tập phụ thuộc hàm F = {AB→C, C→D, A→E, BC→EG}

Tìm bao đóng $(AB)^+$

Bước 1: Đặt $X_0 = AB$

Bước 2: Lần lược xét các phụ thuộc hàm của F

- Xét $AB \rightarrow C$ thuộc F mà $AB \in X_0$, $C \in U$ vậy $X_1 = X_0 \cup C = ABC$

  Loại $AB \rightarrow C$ ra khỏi F

- Xét $A \rightarrow E$ thuộc F mà $A \in X_1$, $E \in U$ vậy $X_2 = X_1 \cup E = ABCE$

  Loại $A \rightarrow E$ ra khỏi F

- Xét $C \rightarrow D$ thuộc F mà $C \in X_2$, $D \in U$ vậy $X_3 = X_2 \cup D$ $=ABCED$

  Loại $C \rightarrow D$ ra khỏi F

- Xét $BC \rightarrow EG$ thuộc F mà $BC \in X_3$, $EG \in U$ vậy $X_4 = X_3 \cup G$ $=ABCEDG$

  Loại $BC \rightarrow EG$ ra khỏi F

Bước 3: Đã xét toàn bộ các phụ thuộc hàm trên F.

Vậy bao đóng của $(AB)^+ = X_4 = ABCEDG$

7.5. Khóa và thuật toán tìm khóa của lược đồ quan hệ

7.5.1. Định nghĩa khóa của lược đồ quan hệ

Cho lược đồ quan hệ $R(A_1, A_2, \ldots, A_n)$ và tập phụ thuộc hàm F trên $R(A_1, A_2, \ldots, A_n)$

$K \subseteq A_1, A_2, \ldots, A_n$. Ta nói K là một khóa của R

Nếu thỏa mãn hai điều kiện sau:

1. $K \rightarrow A_1, A_2, \ldots, A_n \in F^+$
2. $\nexists K' \subset K$ nếu $K' \rightarrow A_1, A_2, \ldots, A_n \in F^+$

(Nếu K thỏa mãn điều kiện **1** thì K gọi là siêu khóa)

7.5.2. Thuật toán tìm khóa của lược đồ quan hệ

7.5.2.1. Thuật toán tìm một khóa

Input: Cho lược đồ quan hệ $R(A_1, A_2, \ldots, A_n)$

F là tập phụ thuộc hàm trên $R(A_1, A_2, \ldots, A_n)$

Output: Một khóa của lược đồ quan hệ R.

Tìm K theo các bước sau:

Bước 1:

- Gán $K_0=U^+$ (U^+ là tập thuộc tính của U)

Bước 2: Ta có A là thuộc tính của U.

- Tính bao đóng của $(K_{i-1}\backslash A)^+ = U^+$ thì loại bỏ A ra khỏi K

tức là $K_i = (K_{i-1}\backslash A)$. nếu $(K_{i-1}\backslash A)^+ != U^+$ thì $K_i = K_{i-1}$.

Lặp lại **Bước 2** n lần

Bước n: Kết quả $K=K_n$

Ví dụ:

Cho lược đồ quan hệ R(U), U=(A, B, C, D, E) và tập phụ thuộc hàm F={AB→C, AC→B, BC→DE}.

Tìm một khóa của lược đồ quan hệ R xác định trên U và F.

Bước 1:

- K=U tức là K=ABCDE

Bước 2:

Lần 1: Tính bao đóng của $(K\backslash A)^+$ nghĩa là tính $(BCDE)^+$ = BCDE ta thấy kết quả tính bao đóng không bằng U^+ nên không loại bỏ A ra khỏi tập K vậy K=ABCDE

Lần 2: Tính bao đóng của $(K\backslash B)^+$ nghĩa là tính $(ACDE)^+$ = ABCDE ta thấy kết quả tính bao đóng bằng U^+ nên loại B ra khỏi tập K vậy K=ACDE.

Lần 3: Tính bao đóng của $(K\backslash C)^+$ nghĩa là tính $(ADE)^+$ = ADE ta thấy kết quả tính bao đóng không bằng U^+ nên không loại bỏ C ra khỏi tập K vậy K=ACDE.

Lần 4: Tính bao đóng của $(K\backslash D)^+$ nghĩa là tính $(ACE)^+$ = ACEBD ta thấy kết quả tính bao đóng bằng U^+ nên loại D ra khỏi tập K vậy K=ACE.

Lần 5: Tính bao đóng của $(K\backslash E)^+$ nghĩa là tính $(AC)^+$ = ACBDE ta thấy kết quả tính bao đóng bằng U^+ nên loại E ra khỏi tập K vậy K=AC.

Bước 3: Kết quả Khóa K=AC

7.5.2.2. Thuật toán tìm nhiều khóa

Một số khái niệm:

- Tập thuộc tính nguồn (TN): bao gồm các thuộc tính chỉ xuất hiện ở vế trái, không xuất hiện ở vế phải của phụ thuộc hàm và các thuộc tính không xuất hiện ở vế trái lẫn vế phải của phụ thuộc hàm.

- Tập thuộc tính đích (TĐ): bao gồm các thuộc tính chỉ xuất hiện ở vế phải không xuất hiện ở vế trái của phụ thuộc hàm.
- Tập thuộc tính trung gian (TG): Chứa thuộc tính ở vế trái lẫn vế phải của phụ thuộc hàm.

Thuật toán:

Input: Cho lược đồ quan hệ $R(A_1, A_2, \ldots, A_n)$

F là tập phụ thuộc hàm trên $R(A_1, A_2, \ldots, A_n)$

Output: Các khóa của lược đồ quan hệ R.

Tìm các khóa theo các bước sau:

Bước 1:

- Tạo tập nguồn TN và tập trung gian TG

Bước 2:

- Nếu TG=∅(rỗng) thì K=TN, kết thúc. ngược lại qua **Bước 3**.

Bước 3:

- Tìm tất cả tập con X_i của tập trung gian.

Bước 4:

- Tìm siêu khóa S_i bằng cách với mọi X_i, nếu $(TN \cup X_i)^+=R^+$ thì $S_i = TN \cup X_i$

Bước 5:

- Tìm khóa bằng cách loại bỏ các siêu khóa không tối thiểu với mọi S_i, S_j thuộc S
- Nếu S_i chứa trong S_j thì loại bỏ tập S_j ra khỏi siêu khóa S còn lại chính là tập khóa cần tìm.

Ví dụ:

Cho lược đồ quan hệ R(U), U=(C, S, Z) và tập phụ thuộc hàm

$F=\{CS \rightarrow Z, Z \rightarrow C\}$

Tìm tất cả các khóa của lược đồ quan hệ R.

Bước 1:

- TN={S}, TG={CZ}

Bước 2:

- TG khác rỗng nên qua **Bước 3**

Bước 3:

- tập con X_i của tập trung gian X={∅, C, Z, CZ}, (∅: là rỗng)

Bước 4:

- S^+=S Khác R có nghĩa không có siêu khóa.
- SC^+=CZS bằng với R nên siêu khóa SC.
- SZ^+=CZ bằng với Q nên siêu khóa là CZ
- SCZ^+= bằng với Q nên siêu khóa là CSZ

Bước 5:

- Vậy tập siêu khóa S={SC, CZ, CSZ} Vì SC chứa trong CSZ và CZ chứa trong CSZ nên loại bỏ siêu khóa CSZ khỏi tập siêu khóa.
- Kết quả khóa của lược đồ quan hệ trên là SC và CZ. K={SC, CZ}

7.6. Các dạng chuẩn

Trong thực tế, một ứng dụng cụ thể có thể được thiết kế thành nhiều lược đồ cơ sở dữ liệukhác nhau, và tất nhiên chất lượng thiết kế của các lược đồ CSDL này cũng khác nhau. Chất lượng thiết kế của một lược đồ CSDL có thể được đánh giá dựa trên nhiều tiêu chuẩntrong đó sự trùng lắp thông tin và chi phí kiểm tra các ràng buộc toàn vẹn là hai tiêu chuẩnquan trọng.

Sau đây là một số tiêu chuẩn để đánh giá độ tốt/xấu của một lược đồ quan hệ. Trước tiên ta tìm hiểu một số khái niệm liên quan:

7.6.1. Dạng Chuẩn Một (First Normal Form)

Một lược đồ quan hệ Q ở dạng chuẩn 1 nếu toàn bộ các thuộc tính của mọi bộ đều mang giá trị đơn.

Ví dụ 1: Xét quan hệ

MASV	HOVATEN	KHOA	TENMONHOC	DIEMTHI
99023	NGUYEN THI THU	CONG NGHE THONG TIN	KY THUAT LAP TRINH TOAN ROI RAC CO SO DU LIEU	6 8 4
99030	LE VAN THANH	DIEN TU	VI XULY	4

Quan hệ này không đạt chuẩn 1 vì các thuộc tính `TENMONHOC`, `DIEMTHI` của bộ thứnhất không mang giá trị đơn (chẳng hạn sinh viên `NGUYEN THI THU` có thuộc tính `TENMONHOC` là `KY THUAT LAP TRINH`, `TOAN ROI RAC`, `CO SO DU LIEU`).

Ta hoàn toàn có thể đưa quan hệ trên về dạng chuẩn 1 như sau:

MASV	HOVATEN	KHOA	TENMONHOC	DIEMTHI
99023	NGUYEN THI THU	CONG NGHE THONG TIN	KY THUAT LAP TRINH	6
99023	NGUYEN THI THU	CONG NGHE THONG TIN	TOAN ROI RAC	8
99023	NGUYEN THI THU	CONG NGHE THONG TIN	CO SO DU LIEU	4
99030	LE VAN THANH	DIEN TU	VI XULY	4

Chú ý: khi xét các dạng chuẩn, nếu không nói gì thêm thì hiểu dạng chuẩn đang xét ít nhất là đạt dạng chuẩn 1.

7.6.2. Dạng Chuẩn 2 (Second Normal Form)

Một lược đồ quan hệ Q ở dạng chuẩn 2 nếu Q đạt chuẩn 1 và mọi thuộc tính không khóa của Q đều phụ thuộc đầy đủ vào khóa.

Thuật toán kiểm tra dạng chuẩn 2

Vào: *lược đồ quan hệ Q, tập phụ thuộc hàm F*

Ra: *khẳng định Q đạt chuẩn 2 hay không đạt chuẩn 2.*

Bước 1*: Tìm tất cả khóa của Q*

Bước 2: *Với mỗi khóa K, tìm bao đóng của tất cả tập con thật sự S của K.*

Bước 3: *Nếu có bao đóng* S^+ *chứa thuộc tính không khóa thì Q không đạt chuẩn 2Ngược lại thì Q đạt chuẩn 2*

Ví dụ 2: Cho lược đồ quan hệ Q(A,B,C,D) và tập phụ thuộc hàm

F={AB→C; B→D; BC→A}.

Hỏi Q có đạt chuẩn 2 không?

Giải:

TN={B},TG={AC}

X_i	$(TN \cup X_i)$	$(TN \cup X_i)^+$	Siêu khóa	khóa
ϕ	B	BD		
A	AB	ABCD	AB	AB
C	BC	ABCD	BC	BC
AC	ABC	ABCD	ABC	

Khóa là K1=AB và K2=BC. Ta thấy B ⊂ K1, B→D, D là thuộc tính không khóa

⇒ thuộc tính không khóa không phụ thuộc đầy đủ vào khóa ⇒ Q không đạt chuẩn 2.

Ví dụ 3: Quan hệ sau đạt chuẩn 2.

Q(G,M,V,N,H,P)

F={G→M; G→N; G→H; G→P; M→V; NHP→M}

Giải:

TN={G} TG={M,N,H,P}

X_i	$(TN \cup X_i)$	$(TN \cup X_i)^+$	Siêu khóa	khóa
ϕ	G	Q^+	G	G
M	GM	Q^+	GM	
N	GN	Q^+	GN	
MN	GMN	Q^+	GMN	
H	GH	Q^+	GH	
MH	GMH	Q^+	GMH	
NH	GNH	Q^+	GNH	
MNH	GMNH	Q^+	GMNH	
P	GP	Q^+	GP	
MP	GMP	Q^+	GMP	
NP	GNP	Q^+	GNP	
MNP	GMNP	Q^+	GMNP	
HP	GHP	Q^+	GHP	
MHP	GMHP	Q^+	GMHP	
NHP	GNHP	Q^+	GNHP	
MNHP	GMNHP	Q^+	GMNHP	

Lược đồ quan hệ Q chỉ có một khóa và khóa chỉ có một thuộc tính nên mọi thuộctính đều phụ thuộc đầy đủ vào khóa ⇒ Q đạt chuẩn 2

Hệ quả:

- Nếu Q đạt chuẩn 1 và tập thuộc tính không khóa của Q bằng rỗng thì Q đạt chuẩn 2
- Nếu tất cả khóa của quan hệ chỉ gồm một thuộc tính thì quan hệ đó ít nhất

đạt chuẩn2.

Ví dụ 4: Cho lược đồ quan hệ và tập phụ thuộc hàm sau:

Q(A,B,C,D,E,H)

F={A → E; C → D; E → DH}

Giải:

TN={ACB} TG={E}

X_i	$(TN \cup X_i)$	$(TN \cup X_i)^+$	Siêu khóa	khóa
ϕ	ACB	ABCDEH	ACB	ACB
E	ACBE	ABCDEH	ACBE	

⇒ khóa của Q là K={ABC}.C ⊂ K, C→D, D là thuộc tính không khóa ⇒ D phụ thuộc không đầy đủ vào khóa nên Q không đạt chuẩn 2.

7.6.3. Dạng Chuẩn 3 (Third Normal Form)

Thuộc tính phụ thuộc bắc cầu: Q là lược đồ quan hệ, X,Y là hai tập con của Q^+, A là mộtthuộc tính. Nói rằng A phụ thuộc bắc cầu vào X nếu cả ba điều sau thỏa:

+ X → Y,Y → A

+ Y↛X

+ A ∉ XY

Định nghĩa 1:

Lược đồ quan hệ Q ở dạng chuẩn 3 nếu mọi phụ thuộc hàm X→ A ∈ F^+ với A∉X đều có:

- *Hoặc X là siêu khóa*
- *Hoặc A là thuộc tính khóa*

Định nghĩa 2:

Lược đồ quan hệ Q ở dạng chuẩn 3 nếu mọi thuộc tính không khóa của Q đều không phụ thuộc bắc cầu vào một khóa bất kỳ của Q

Hai định nghĩa trên là tương đương, tuy nhiên việc cài đặt thuật toán kiểm tra dạng chuẩn3 theo định nghĩa 1 thì hiệu quả hơn nhiều vì không phải kiểm tra tính phụ thuộc bắc cầu.

Ta chứng minh hai định nghĩa tương đương bằng cách:

- **Từ định nghĩa 1 ⇒ không có phụ thuộc bắc cầu vào một khóa bất kỳ của Q.**

Thật vậy:

Giả sử có phụ thuộc bắc cầu vào khóa nghĩa là có $K \to Y, Y \to A, Y \not\to K$ và $A \notin KY$.

$Y \to A$ là một phụ thuộc hàm nên theo định nghĩa 1 có hai trường hợp xảy ra cho Y:

+ Y là siêu khóa $\Rightarrow Y \to K$ điều này mâu thuẫn với $Y \not\to K$.
+ Y không là siêu khóa $\Rightarrow A$ là thuộc tính khóa $\Rightarrow$ điều này trái với giả thiết $A \notin KY$

- **Từ định nghĩa 2 $\Rightarrow$ nếu $X \to A \in F^+$ với $A \notin X$ thì X là siêu khóa hoặc A là thuộc tính khóa.**

Nếu $X \to A \in F^+$ với $A \notin X$ có X không là siêu khóa và A không là thuộc tính khóa thì dẫn đếnmột số điều sau:

A không là thuộc tính khóa $\Rightarrow A \notin K$

X không là siêu khóa $\Rightarrow X \not\to K$

Tóm lại ta có $K \to X$, $X \to A, X \not\to K$ và $A \notin KX$ $\Rightarrow A$ phụ thuộc bắc cầu vào K điều này mâu thuẫn với định nghĩa 2.

Hệ quả 1: Nếu Q đạt chuẩn 3 thì Q đạt chuẩn 2

Hệ quả 2: Nếu Q không có thuộc tính không khóa thì Q đạt chuẩn 3.

Chứng minh:

Hệ quả 1: Giả sử Q đạt dạng chuẩn 3 và có thuộc tính không khóa A không phụ thuộc hàm đầy đủ vào khóa K $\Rightarrow K' \subset K$ sao cho $K' \to A$ như vậy ta có $K \to K', K' \to A, K' \not\to K, A \notin KK' \Rightarrow Q$ có phụ thuộc bắc cầu.

Hệ quả 2: mọi phụ thuộc hàm trong Q đều có vế phải là thuộc tính khóa $\Rightarrow$ Q đạt dạng chuẩn 3

7.6.4. Định lý

Q là lược đồ quan hệ

F là tập các phụ thuộc hàm có vế phải một thuộc tính.

Q đạt chuẩn 3 nếu và chỉ nếu mọi phụ thuộc hàm $X \to A \in F$ với $A \notin X$ đều có X là siêu khóa hay A là thuộc tính khóa.

Chứng minh:

Q đạt dạng chuẩn 3 theo định nghĩa ta suy ra mọi phụ thuộc hàm $X \rightarrow A \in F$ với $A \notin X$ có X là siêu khóa hoặc A là thuộc tính khóa.

Ngược lại ta phải chứng minh nếu mọi phụ thuộc hàm $X \rightarrow A \in F$ với $A \notin X$ có X là siêu khóa hoặc A là thuộc tính khóa thì mọi phụ thuộc hàm $X \rightarrow A \in F^+$ với $A \notin X$ cũng có X là siêu khóa hoặc A là thuộc tính khóa

Giả sử có phụ thuộc hàm $X \rightarrow A \in F^+$ với $A \notin X$ sao cho X không là siêu khóa và A không là thuộc tính khóa sẽ dẫn đến $A \in X^+ \subseteq X \cup$ {các thuộc tính khóa} điều này mâu thuẫn với $A \notin K$. Trước khi chứng minh $A \in X^+ \subseteq X \cup$ {các thuộc tính khóa} ta có nhận xét sau:

X không là siêu khóa $\Rightarrow X^+$ cũng không là siêu khóa. Theo thuật toán tìm bao đóng, X^+ được hình thành từ các X_i $\Rightarrow$ ở mỗi bước X_i cũng không là siêu khóa.

Bước cơ sở: $X_0 = X \Rightarrow X_0 \subseteq X \cup$ {các thuộc tính khóa}

Bước qui nạp: Giả sử có $X_{i-1} \subseteq X \cup$ {các thuộc tính khóa}. Bao đóng X_i được hình thành do có $f_j = X_j \rightarrow Y_j$ để $X_{i-1} \supseteq X_j$ và $X_i = X_{i-1} \cup Y_j$ $\Rightarrow f_j = X_j \rightarrow Y_j$ là phụ thuộc hàm có X_j không là siêu khóa $\Rightarrow f_j = X_j \rightarrow Y_j$ là phụ thuộc hàm có Y_j là thuộc tính khóa $\Rightarrow X_i = X_{i-1} \cup Y_j \subseteq X \cup$ {các thuộc tính khóa}

Qua chứng minh trên $\Rightarrow A \in X^+ \subseteq X \cup$ {các thuộc tính khóa}

$\Rightarrow A \in X \cup$ {các thuộctính khóa} $\Rightarrow A \in$ {các thuộc tính khóa} điều này nghịch lý với điều $A \notin K$.

Thuật toán kiểm tra dạng chuẩn 3

Vào*: lược đồ quan hệ Q, tập phụ thuộc hàm F*

Ra: *khẳng định Q đạt chuẩn 3 hay không đạt chuẩn 3.*

Bước 1: *Tìm tất cả khóa của Q*

Bước 2: *Từ F tạo tập phụ thuộc hàm tương đương F_{1tt} có vế phải một thuộc tính.*

Bước 3: *Nếu mọi phụ thuộc hàm $X \rightarrow A \in F_{1tt}$ với $A \notin X$ đều có X là siêu khóa hoặc A là thuộc tính khoá thì Q đạt chuẩn 3 ngược lại Q không đạt chuẩn 3.*

Ví dụ 5:

Cho lược đồ quan hệ Q(A,B,C,D) F={AB→C; D→B; C→ABD}.

Hỏi Q có đạt chuẩn 3 không?

Giải:

TN=∅ TG={ABCD}

X_i	$(TN \cup X_i)$	$(TN \cup X_i)^+$	Siêu khóa	khóa
ϕ	ϕ	ϕ		
A	A	A		
B	B	B		
AB	AB	ABCD	AB	AB
C	C	ABCD	C	C
AC	AC	ABCD	AC	
BC	BC	ABCD	BC	
ABC	ABC	ABCD	ABC	
D	D	BD		
AD	AD	ABCD	AD	AD
BD	BD	BD		
ABD	ABD	ABCD	ABD	
CD	CD	ABCD	CD	
ACD	ACD	ABCD	ACD	
BCD	BCD	ABCD	BCD	
ABCD	ABCD	ABCD	ABCD	

K1={AB}; K2={AD}; K3={C} là các khóa ⇒ mọi phụ thuộc hàm X→A∈F đều có A là thuộc tính khóa. Vậy Q đạt chuẩn 3

Ví dụ 6: Quan hệ sau đạt chuẩn 3.

Q(N,G,P,M)

F = {NGP→M,M→P}

7.6.5. Dạng Chuẩn BC (Boyce-Codd Normal Form)

Một quan hệ Q ở dạng chuẩn BC nếu mọi phụ thuộc hàm $X \rightarrow A \in F^+$ với $A \notin X$ đều có X là siêu khóa.

Hệ quả 1: Nếu Q đạt chuẩn BC thì Q đạt chuẩn 3 (hiển nhiên do định nghĩa)

Hệ quả 2: Mỗi lược đồ có hai thuộc tính đều đạt chuẩn BC (xét phụ thuộc hàm có thể có của Q)

a. Định lý:

Q là lược đồ quan hệ

F là tập các phụ thuộc hàm có vế phải một thuộc tính.

Q đạt chuẩn BC nếu và chỉ nếu mọi phụ thuộc hàm $X \rightarrow A \in F$ *với* $A \notin X$ *đều có* X *là siêu khóa*

Chứng minh:

Q đạt dạng chuẩn BC theo định nghĩa ta suy ra mọi phụ thuộc hàm $X \rightarrow A \in F$ với $A \notin X$ có X là siêu khóa.

Ngược lại ta phải chứng minh nếu mọi phụ thuộc hàm $X \rightarrow A \in F$ với $A \notin X$ có X là siêu khóa thì mọi phụ thuộc hàm $Z \rightarrow B \in F^+$ với $B \notin Z$ cũng có Z là siêu khóa. Thật vậy, do $Z \rightarrow B$ không là phụ thuộc hàm hiển nhiên nên theo thuật toán tìm bao đóng phải có $X \rightarrow A \in F$ sao cho $Z \supseteq X$ (X là siêu khóa)$\Rightarrow$ Z là siêu khóa.

b. Thuật toán kiểm tra dạng chuẩn BC

Vào: *lược đồ quan hệ Q, tập phụ thuộc hàm F*

Ra: *khẳng định Q đạt chuẩn BC hay không đạt chuẩn BC.*

Bước 1: *Tìm tất cả khóa của Q*

Bước 2: *Từ F tạo tập phụ thuộc hàm tương đương* F_{1tt} *có vế phải một thuộc tính*

Bước 3: *Nếu mọi phụ thuộc hàm* $X \rightarrow A \in F_{1tt}$ *với* $A \notin X$ *đều có X là siêu khóa thì Qđạt chuẩn BC ngược lại Q không đạt chuẩn BC.*

Ví dụ: Cho quan hệ Q và tập phụ thuộc hàm sau:

Q(A, B, C, D, E, I)

F={ACD→EBI; CE→AD}.

Hỏi Q có đạt chuẩn BC không?

Giải:

TN={C}

TG={ADE}

X_i	$(TN \cup X_i)$	$(TN \cup X_i)^+$	Siêu khóa	khóa
ϕ	C	C		
A	AC	AC		
D	CD	CD		
AD	ACD	ABCDEI	ACD	ACD
E	CE	ABCDEI	CE	CE
AE	ACE	ABCDEI	ACE	
DE	CDE	ABCDEI	CDE	
ADE	ACDE	ABCDEI	ACDE	

$F \equiv F_{1tt}$={ACD→E,ACD→B,ACD→I,CE→A,CE→D}

Mọi phụ thuộc hàm của F_{1tt} đều có vế trái là siêu khóa ⇒ Q đạt dạng chuẩn BC

Ví dụ: Cho quan hệ Q và tập phụ thuộc sau:

Q(SV,MH,THAY)

F = {SV,MH → THAY; THAY → MH}

Quan hệ trên đạt chuẩn 3 nhưng không đạt chuẩn BC.

Ví dụ: Chẳng hạn cho

Q(A,B,C,D) và F={AB → C; D → B; C → ABD} thì Q là 3NF nhưng không là BCNF.

Nếu F={B → D,A → C,C → ABD} là 2 NF nhưng không là 3 NF.

c. Thuật toán kiểm tra dạng chuẩn của một lược đồ quan hệ.

Vào*: Lược đồ quan hệ Q, tập phụ thuộc hàm F*

Ra: *Khẳng định Q đạt chuẩn gì?*

Bước 1: *Tìm tất cả khóa của Q*

Bước 2: *Kiểm tra chuẩn BC nếu đúng thì Q đạt chuẩn BC, kết thúc thuật toán ngược lại qua bước.*

Bước 3: *Kiểm tra chuẩn 3 nếu đúng thì Q đạt chuẩn 3, kết thúc thuật toán* ngược lại qua bước 4

Bước 4: *Kiểm tra chuẩn 2 nếu đúng thì Q đạt chuẩn 2, kết thúc thuật toán. ngược lại Q đạt chuẩn 1.*

d. Định nghĩa

Dạng chuẩn của một lược đồ cơ sở dữ liệu là dạng chuẩn thấp nhất trong các dạng chuẩn của các lược đồ quan hệ con.

TRƯỜNG ĐẠI HỌC VĂN HIẾN
1997
VAN HIEN UNIVERSITY

BÀI TẬP CHƯƠNG 7

1. Cho biết dạng chuẩn của các lược đồ quan hệ sau:

a) Q(ABCDEG); F={A→BC,C→DE,E→G}

b) Q(ABCDEGH); F={C→AB,D→E,B→G}

c) Q(ABCDEGH; F={A→BC,D→E,H→G}

d) Q(ABCDEG); F={AB→C,C→B,ABD→E,G→A}

e) Q(ABCDEGHI);

F={AC→B,BI→ACD,ABC→D,H→I,ACE→BCG,CG→AE}

2. Kiểm tra sự bảo toàn thông tin?

Q(ABCDE)

R1(AD); R2(AB); R3(BE); R4(CDE); R5(AE)

F={A → C; B → C; C → D; DE → C; CE → A}

3. Kiểm tra dạng chuẩn Q(C, S, Z) F={CS→Z; Z→C}

4 . Phân rã Q(G, H, A, B, C, D)

F={GH→AD; AG→B; CD→GH; C→A; BH→C}

5. Cho lược đồ CSDL

Kehoach(NGAY, GIO, PHONG, MONHOC, GIAOVIEN)

F={NGAY, GIO, PHONG→MONHOC

MONHOC, NGAY→GIAOVIEN

NGAY, GIO, PHONG→GIAOVIEN

MONHOC→GIAOVIEN}

a) Xác định dạng chuẩn cao nhất của Kehoach

b) Nếu Kehoach chưa đạt dạng chuẩn 3, hãy phân rã Kehoach thành lược đồ CSDLdạng chuẩn 3 vừa bảo toàn phụ thuộc hàm vừa bảo toàn thông tin.

6. Giả sử ta có lược đồ quan hệ Q(C, D, E, G, H, K) và tập phụ thuộc hàm F như sau;

F = {CK→H; C→D; E→C; E→G; CK→E}

a) Từ tập F, hãy chứng minh EK→DH

b) Tìm tất cả các khóa của Q.

c) Xác định dạng chuẩn của Q.

d) Hãy tìm cách phân rã Q thành một lược đồ cơ sở dữ liệu đạt dạng chuẩn BC (hoặc dạngchuẩn 3). tìm tập phụ thuộc hàm và khóa cho mỗi lược đồ quan hệ con.

7. Cho lược đồ quan hệ Q(S,I,D,M)

F = {f1:SI→DM; f2:SD→M; f3:D→M}

a) Tính bao đóng D^+, SD^+, SI^+

b) Tìm tất cả các khóa của Q

c) Tìm phủ tối thiểu của F

d) Xác định dạng chuẩn cao nhất của Q

e) Nếu Q chưa đạt dạng chuẩn 3, hãy phân rã Q thành lược đồ CSDL dạng chuẩn 3vừa bảo toàn phụ thuộc hàm vừa bảo toàn thông tin.

f) Nếu Q chưa đạt dạng chuẩn BCNF, hãy phân rã Q thành lược đồ CSDL dạng BCNF

g) Kiểm tra phép tách Q thành các lược đồ con (SID,SIM) có bảo toàn thông tin ?

h) Kiểm tra phép tách Q thành các lược đồ con (SID,SIM) có bảo toàn phụ thuộc hàm ?

8. Cho lược đồ quan hệ R(W, A, Z, Y, Q, P)

R1(A,Z); R2(W,Y,Q,P) R3(Y,Q,P,A)

F = {W→AYQP, A→Z, YQP →A}

MONHOC,NGAY→GIAOVIEN

NGAY,GIO,PHONG→GIAOVIEN

MONHOC→GIAOVIEN}

a) Xác định dạng chuẩn cao nhất của Kehoach

b) Nếu Kehoach chưa đạt dạng chuẩn 3, hãy phân rã Kehoach thành lược đồ CSDLdạng chuẩn 3 vừa bảo toàn phụ thuộc hàm vừa bảo toàn thông tin.

TÀI LIỆU THAM KHẢO

[1]. Dương Tuấn Anh, Nguyễn Trung Trực, Hệ cơ sở dữ liệu, NXB Đại học Quốc gia Tp.HCM, 2006.

[2]. Đồng Thị Bích Thủy, Nguyễn Trần Minh Thư, Phạm Thị Bạch Huệ, "Cơ sở dữ liệu", NXB Khoa Học và Kỹ Thuật, 2010.

[3]. Đồng Thị Bích Thủy, Nguyễn An Tế, "Giáo trình nhập môn Cơ sở dữ liệu", Đại học KHTN ĐH QG TPHCM, 2003.

[4]. R. Elmasri & S.B. Navathe. Fundamentals of Database Systems (6nd Edition), Addison-Wesley, 2011

[5]. Silberschatz,Korth and Sudarshan. Database System Concepts (6th Edition), McGraw -Hill, 2010